

Diseño y fabricación de un sistema de telemetría en tiempo real con conectividad móvil

Escuela de Ingenierías Industriales

Autor: Julián Samblás Caballero

Tutor: Javier Pérez Fernández

Co-tutor: Francisco José Sánchez Pacheco

Ingeniería Electrónica Industrial

A 25 de Mayo de 2024, Málaga

Resumen

Este proyecto se encarga del desarrollo y fabricación de un dispositivo de telemetría que sea capaz de llevar al usuario común del motociclismo las ventajas que posee obtener información de los diferentes parámetros que afectan a la precisión en la conducción. El dispositivo tiene como objetivos medir la posición, velocidad y aceleración, que sufre el vehículo cuando se desarrolla la conducción. Estos datos han de ser mostrados por una pantalla, y enviados a una nube a través de internet. La grabación y visualización de estos forma parte del proyecto SpeedSense.

Por ello se ha hecho una introducción a la importancia que ha adquirido la telemetría a día de hoy en muchos aspectos de la vida, pero sobre todo en el mundo de la competición automovilística. Se realiza un estudio detallado de los componentes disponibles en el mercado y cuales son los que mejor se adaptan a las necesidades del proyecto. A continuación, se han diseñado los prototipos de placas PCB, al obtener el dispositivo físico final, se ha procedido a la programación del ESP32-S3 para que sea funcional, y por último se han realizado diferentes pruebas para verificar su funcionamiento.

El desarrollo de este dispositivo ha sido trabajado en diferentes plataformas como EasyEDA (para el desarrollo de la placa PCB), Square Line Studio (para la interfaz del display) y Arduino IDE (para la programación y correcto funcionamiento de los diferentes sensores).

Abstract

This Project focuses on the development and manufacturing of a telemetry device capable of bringing the advantages of obtaining information about various parameters affecting driving precisión to the common motocycle user. The device aims to measure the position, speed, and acceleration experienced by the vhicle during driving. This data is to be displayed on a screen and sent to the cloud via internet. Recording and visualization of this data are parto f the SpeedSense Project.

For this reason, an introduction to the importance of telemetry in various aspects of life today has been made, particularly in the world of automotive competition. A detailed study of the components avaiable on the market and which ones best suit the project´s needs has been conducted. Subsequently, prototypes of PCB boards have been designed. Upon obtaining the final physical devide, the ESP32-S3 has been programmed to make it functional, and finally, various test have been carried out to verify its operation.

The development of this device has been carried out on different platforms such as EasyEDA (for PCB board developmente), Square Line Studio (for display interface), and Arduino IDE (for programming and proper functioning of different sensors).

DOCUMENTO I: MEMORIA

DOCUMENTO I: MEMORIA

ÍNDICE DE ILUSTRACIONES 8

CAPÍTULO 0: INTRODUCCIÓN 1

0.1. Antecedentes 1

0.2. Objetivos 1

CAPÍTULO 1: TELEMETRÍA 3

1.1. ¿Qué es la telemetría y adquisición de datos? 3

1.1.1. Estado del arte 3

1.1.2. Importancia de la adquisición de datos en la actualidad 5

1.1.3. Importancia de la adquisición de datos en el mundo del motor 6

1.2. Telemetría aplicada a la competición 12

1.3. Objetivos del sistema proyectado 14

CAPÍTULO 2: COMPONENTES DEL DISPOSITIVO 14

2.1. Módulo GPS 15

2.1.1. Funcionalidades del módulo GPS 15

2.1.2. Opciones en el mercado 16

2.2. IMU 17

2.2.1. Opciones de mercado 18

2.3. Módulo SIM 21

2.4. Display 23

CAPÍTULO 3: PROTOTIPOS 25

3.1. Primer prototipo 25

3.2. Segundo prototipo 28

CAPÍTULO 4: MÉTODOS DE ADQUISICIÓN Y VISUALIZACIÓN DE DATOS 33

4.1. Protocolos de comunicación utilizados 33

4.1.1. UART 33

4.1.2. I2C 34

4.2. Pines de conexión 34

4.3. Programación en Square Line Studio y Arduino IDE 35

4.3.1. Square Line Studio 35

4.3.2. Arduino IDE 39

CAPÍTULO 5: MONTAJE Y PRUEBAS REALIZADAS AL DISPOSITIVO EN FUNCIONAMIENTO. 51

5.1. Montaje del dispositivo 51

5.2. Pruebas realizadas al dispositivo en funcionamiento 53

CAPÍTULO 6: CONCLUSIONES Y FUTURO DEL PROYECTO. 60

6.1. Conclusiones 60

6.2. Futuro del proyecto 60

BIBLIOGRAFÍA 63

ÍNDICE DE ILUSTRACIONES

Ilustración 1.Visualización de una telemetría [6] 5
Ilustración 2. Diferentes sensores en un automóvil [12] 7
Ilustración 3. Diferentes sensores en un vehículo [13] 9
Ilustración 4. Centralita programable de competición [14] 10
Ilustración 5. Ángulo de inclinación máximo en los diferentes tipos de motos [15] 11
Ilustración 6. Caída del piloto Jorge Lorenzo en el GP Shangai 2008 [16] 12
Ilustración 7. Asignación de pines MAX-M10S [17] 17
Ilustración 8. BNO055 [18] 21
Ilustración 9. T-Display-S3 [19] 23
Ilustración 10. Esquemático prototipo 1 26
Ilustración 11. Vista frontal PCB primer prototipo 27
Ilustración 12. Esquemático segundo prototipo 28
Ilustración 13. Vista trasera PCB segundo prototipo 29
Ilustración 14. Detalle de conexionado 29
Ilustración 15. Vista fronto/lateral PCB del segundo prototipo 30
Ilustración 16. Vista superior PCB segundo prototipo 30
Ilustración 17. Funcionamiento protocolo UART [11] 33
Ilustración 18. Conexión de pines al ESP32-S3 35
Ilustración 19.Interfaz Square Line Studio 36
Ilustración 20. Diferentes opciones square line studio 37
Ilustración 21. Configurador de diseños de square line studio 37
Ilustración 22. Explicación square line studio 38
Ilustración 23 . Configurador de diseños square line studio 2 38
Ilustración 24.Configuración para exportar archivos square line studio 39
Ilustración 25. Parte delantera dispositivo separado 51
Ilustración 26. Parte trasera de ambas placas 52
Ilustración 27.Unión de ambas placas 53
Ilustración 28. Pantalla principal en prueba estática en casa. 54
Ilustración 29. Segunda pantalla del dispositivo en prueba estática en casa. 55
Ilustración 30. Prueba en movimiento del dispositivo(Pantalla principal) 56
Ilustración 31. Prueba en funcionamiento (Pantalla secundaria) 56
Ilustración 32. Verificación GPS 57
Ilustración 33. Prueba de trayecto realizado 59
Ilustración 34. Captura página principal de SpeedSense.es 61

Ilustración 35.Simulación de lo que se verá con los datos enviados 61

CAPÍTULO 0: INTRODUCCIÓN

0.1. Antecedentes

El proyecto SynRide es un proyecto que pertenece al programa de maduración de proyectos de la UMA, el cual promueve el desarrollo de ideas basadas en tesis doctorales. El SynRide, nace con la finalidad de traer la tecnología del automovilismo de competición al usuario amateur del automovilismo. La idea base del proyecto siempre ha sido la de medir parámetros que se desarrollan durante la conducción (ya sea en una motocicleta o en un automóvil).

En el modelo anterior se optó por la eficiencia energética y un uso en ciudad del dispositivo, este diseño fue el de un prototipo, por lo que el costo de los materiales es muy diferente al costo que tendrán en el presente proyecto, ya que ahora lo que se busca es una serie corta de producción. Este giro del proyecto se ha producido debido a que se comprobó que entrar en un mercado genérico de usuarios es más complicado que entrar en un mercado específico, por lo que se aborda la idea de entrar en el mercado de los circuitos a pequeña escala (karting, escuelas de motociclismo, etc.).

En el dispositivo actual, se ha optado por la simpleza del dispositivo economizando al máximo posible los costes, pero manteniendo la idea de medir diversos parámetros dentro de la conducción, además de darle un enfoque más centrado hacia el mundo de la competición.

0.2. Objetivos

El objetivo principal del presente proyecto es construir y programar un dispositivo capaz de medir los siguientes parámetros:

- Localización. Mediante un localizador GPS, capaz de obtener una frecuencia de muestreo similar a la que se obtiene en la competición.
- Aceleración, velocidad y ángulos de giro. Se utiliza una IMU capaz de medir hasta en 9 ejes, para así obtener la máxima información posible.
- Transmisión de datos. Gracias a una tarjeta SIM la cual estará conectada a internet se enviarán los datos recogidos en tiempo real a una nube situada en una página web. Esta página web tiene de nombre Speedsense, y se está desarrollando dentro de una spinoff de la Universidad de Málaga.

Gracias a estos tres parámetros, los datos que se obtienen podrán ser entendidos por cualquier usuario común, y no hará falta de tener grandes conocimientos acerca de la telemetría para mejorar la conducción del usuario.

CAPÍTULO 1: TELEMETRÍA

1.1. ¿Qué es la telemetría y adquisición de datos?

La telemetría es la tecnología que hace posible la medición a distancia de magnitudes físicas que afectan a un objeto o cuerpo con el objetivo de recopilar información para enviarla o almacenarla y permitir su estudio desde un centro de control capacitado.

Por norma general la información telemétrica es enviada utilizando una comunicación inalámbrica, aunque es posible emplear teléfonos, redes de ordenadores o cableado de distinto tipo. Un sistema de telemetría, por lo general está constituido por un transductor como dispositivo de entrada, un medio de transmisión en forma de líneas de cable u ondas de radio, dispositivos de procesamiento de señales y dispositivos de grabación o visualización de datos.

Componentes del sistema de telemetría:

- El dispositivo de entrada se puede distinguir como conjunto de dos partes fundamentales: el sensor, que es el elemento sensible primario que responde a las variaciones de estado de las magnitudes físicas de estudio, y el transductor, que es el que se encarga de convertir el valor medido en la señal eléctrica correspondiente, una vez detectada la variable a seguir, como puede ser velocidad, revoluciones del motor o la inclinación del vehículo.
- El medio de transmisión puede establecerse de forma guiada por medios como redes de telefonía clásica, redes de ordenadores o enlaces de fibra óptica. O de forma no guiada, como ondas de radio, comunicación por bluetooth o wifi o incluso por redes de telefonía móvil, que es la opción más adecuada para el presente proyecto.
- El dispositivo de procesamiento de la señal está compuesto por un servidor remoto encargado de analizar y transformar los datos, según sea conveniente, para almacenar toda la información en una página web a la cual se podrá acceder libremente para ver los resultados obtenidos.

1.1.1. Estado del arte

A lo largo de este apartado se pretende, continuando de manera aproximada con el anterior, explicar y detallar como se encuentra el mundo de la telemetría en el motociclismo actualmente. Hay que señalar que, esta no es tarea fácil debido a las razones que a continuación se exponen.

La primera de ellas es que en términos de competición de alto nivel (como el Campeonato del mundo de MotoGP) es prácticamente imposible conocer las capacidades, características y especificaciones del sistema desde fuera.

Además, cada empresa guarda con extremo recelo su forma de adquirir los datos, las operaciones que se realizan y la manera de guardar y transmitir los valores medidos. A lo que hay que añadirle que, cada fabricante (sea de motos en general, o de telemetría particular) suele realizar su propia telemetría y siguiendo su propia estrategia en la alta competición.

Es decir, hay marcas que compran sistemas de telemetría a empresas del sector (como AIM), realizan colaboraciones creando partes de la marca y subcontratando otras o incluso las más grandes realizan su propio sistema completo.

Otra de las razones y sin salir del mundo de las carreras, es que los sistemas que se utilizan y aplican pueden variar muchísimo de un tipo de motocicleta a otra, o de un tipo de competición a otra. Por ilustrar un ejemplo, no se necesitan los mismos datos para estudiar el comportamiento de la moto si es una competición sobre asfalto en circuito,(que no presenta irregularidades, por ejemplo) o si es una competición de rally como el Dakar (dónde si interesa medir cómo se comporta en terreno irregular el vehículo).

La telemetría comúnmente se refiere a los sistemas de adquisición de datos, incluso si estos datos no se envían a distancia. Aunque la definición original de telemetría implica la transmisión de datos a una ubicación remota, en la práctica, el término también se utiliza para describir sistemas que simplemente recogen y almacenan datos localmente(o en una nube como en este caso) sin necesidad de transferirlos a otra ubicación.

Tanto la opacidad de los sistemas utilizados en competición, como la amplia diversidad de sistemas posibles de encontrar en el mercado, motivan y ratifican la decisión del proyecto Synride.



Ilustración 1.Visualización de una telemetría [6]

1.1.2. Importancia de la adquisición de datos en la actualidad

La adquisición de datos, como se ha especificado con anterioridad, se trata de la medida de parámetros a distancia generalmente en directo, pero, se ha extendido el concepto incluyendo entornos en los que las medidas se leen en diferido.

La sensorística, como disciplina general, está completamente extendida en nuestros días y presente en cada uno de los productos y máquinas que se utilizan. Con el progreso de la industria, y más concretamente de la electrónica, la necesidad de medir parámetros físicos y la aplicación de sensores ha sido una consecuencia directa.

Generalmente, esto ha sido en busca de dos propósitos diferentes: la mejora del rendimiento y capacidades de la máquina, por un lado, e interactuar con el entorno o el usuario, por otro.

El ejemplo más destacado de esto aparece en la telefonía móvil, en la que el dispositivo medio tiene aproximadamente 10 sensores diferentes que son:

1. Acelerómetro
2. Barómetro
3. Giroscopio
4. GPS
5. Lector de huella dactilar
6. Magnetómetro (o brújula electrónica)
7. Sensor de proximidad
8. Sensor de luz ambiental

9. Sensor infrarrojo
10. Termómetro (por seguridad de los componentes y batería)

Obviamente, si se retrocede hasta uno de los primeros modelos de telefonía móvil más conocidos, el Nokia 3310 lanzado en el año 2000, la lista de sensores de los que dispone es nula. Solo pudiendo considerarse como sensores el micrófono y el encargado de calcular el nivel de batería del dispositivo.

Y observando los modelos más avanzados, se tiene la lista anterior y a la que pueden añadir sensores como el podómetro, lector de iris o pulsómetro, entre otros.

Este cambio exponencial en términos de sensores, lo han sufrido todos los dispositivos electrónicos que utiliza el usuario medio, televisores, planchas e incluso frigoríficos están hoy en día repletos de sensores que miden los parámetros que definen el comportamiento de la máquina. Llegando, en aplicaciones como la domótica, a ser telemetría en toda su definición, dado que los valores de dichos sensores se observan y controlan a distancia.

1.1.3. Importancia de la adquisición de datos en el mundo del motor

Dentro de esta revolución no se puede ignorar ni dejar de lado el mundo del motor, en el que se va a entrar en más detalle.

Los primeros vehículos de calle en incorporar distintas clases de sensores fueron los automóviles, teniendo hoy en día hasta 24 tipos diferentes en un mismo coche. Se sobre entiende, que se está hablando exclusivamente de tipología, siendo la cantidad numérica de sensores muy superior. Al igual que en los casos anteriores, la introducción de esta gran cantidad de transductores (y de la electrónica asociada), ha sido en pos del aumento del rendimiento del vehículo y de su integración con el usuario, en forma de confort y seguridad. Atendiendo a estas tres familias, se van a diferenciar los tipos de sensores que se pueden encontrar en cualquiera de los vehículos que ocupan a día de hoy las carreteras.

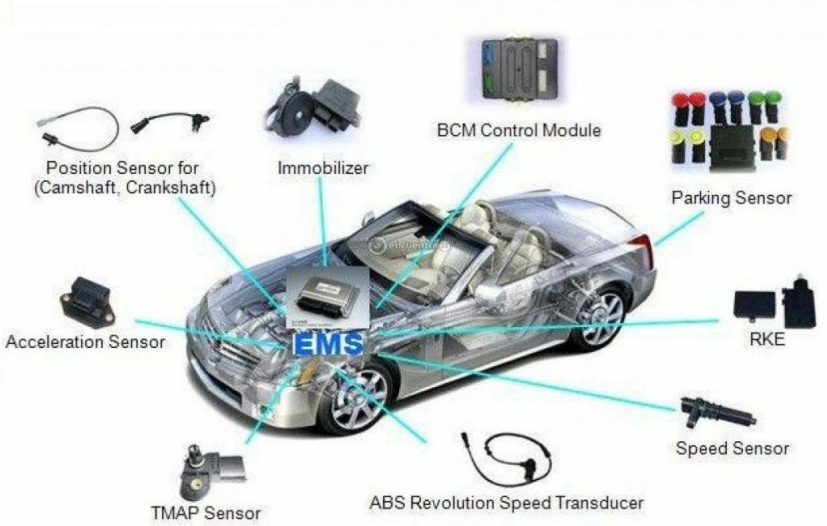


Ilustración 2. Diferentes sensores en un automóvil [12]

1. Encargados de evaluar, mejorar e interactuar con el rendimiento y la parte ciclo del motor, algunos ejemplos son:
   - Señor de presión
   - Sensor de sobrealimentación
   - Sensor de masa de aire
   - Sensor de presión ambiente
   - Sensor de alta presión (en la inyección de combustible)
   - Sonda Lambda (mide la pureza de los gases de escape)
   - Sensor de velocidad de rotación
   - Sensor de presión en el depósito
   - Sensor de posición de los pedales
   - Sensor de ángulo de posición del árbol de levas
   - Termómetros para el aceite, motor y agua

2. Su misión es la de evitar accidentes, adelantarse a sucesos peligrosos y mejorar la conducción en condiciones de riesgo:

   - Radar telemétrico (para evitar colisiones)

   - Sensor de inclinación (para regular correctamente faros o incluso suspensiones)

   - Sensor de par

- Sensores de alta presión y ángulo de volante de dirección (para el ESP)

- Sensores de ocupación de asientos

- Sensor de aceleración transversal (también para el ESP)

- Sensor de inclinación

- Sensor de vuelco

- Sensor de velocidad y giro de ruedas

3. Involucrados en hacer de la conducción una mejor experiencia y más disfrutable al usuario:

- Sensor de viraje (para la navegación)

- Sensor de calidad del aire

- Sensor de lluvia

- Termómetros (exterior y habitáculo)

- Sensores de ultrasonido (para la localización de personas o aparcamiento)

Simplemente observando el tamaño de la lista anterior y comparándola con la que se ha redactado anteriormente sobre el mismo tema en smartphones, se puede ver que, esta revolución ha sido aún mayor en los vehículos.

Una vez se ha introducido el mundo del motor, encuadrados en el avance y la importancia de la telemetría, se va a caracterizar el tema en el mundo de la telemetría en motocicletas, y más concretamente en la competición.

En el caso de las dos ruedas, existió un enorme punto de inflexión cuando se introdujo la inyección electrónica en las motocicletas (tanto de competición como de calle). La gestión de la potencia siempre se ha realizado controlando el ratio de la mezcla (relación aire/combustible introducida en el cilindro) y la cantidad de esa mezcla que se introduce en el motor, pero antiguamente se realizaba a través de carburantes, sistemas mucho menos flexibles y mucho más rudimentarios que la inyección electrónica.

Más tarde, se introdujeron los sistemas que, a través de una centralita electrónica que controla los inyectores de combustible, calculaban al milímetro y de manera

preprogramada la mezcla y su inyección.

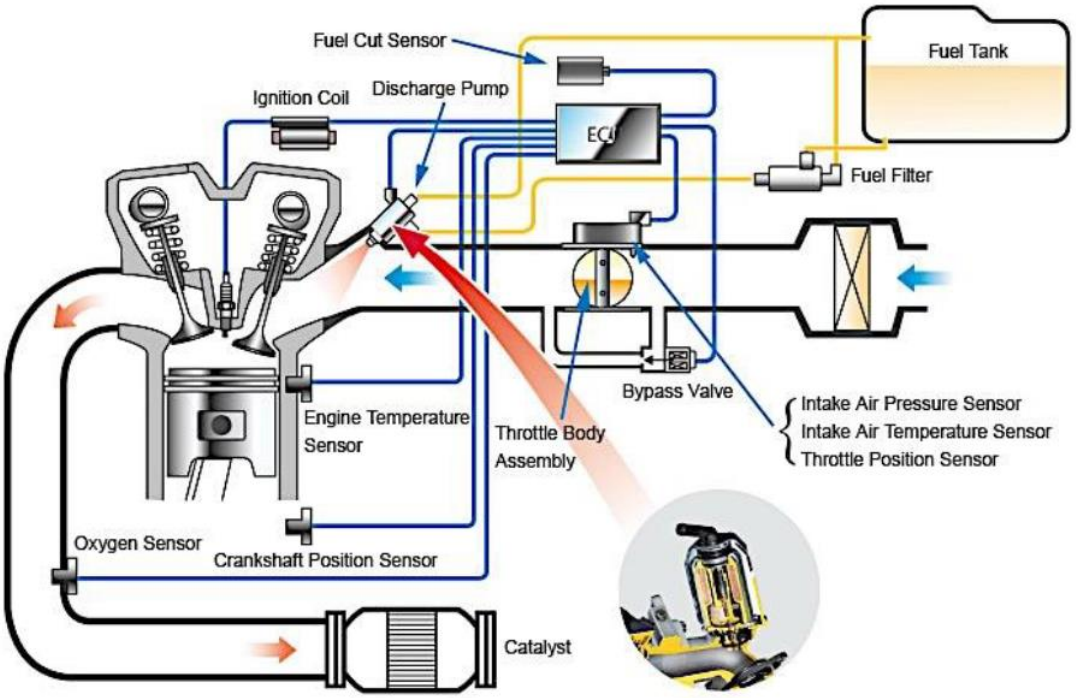


Ilustración 3. Diferentes sensores en un vehículo [13]

La centralita de la moto o ECU (Engine Control Unit) puede ser programada por el usuario de innumerables formas diferentes, pudiendo variar parámetros como el avance de la chispa, el tiempo de inyección, la pureza de la mezcla, etc. Pero, para poder realizar todo este tipo de actuaciones diferentes y ser programadas según ciertos sucesos o acciones del piloto, la ECU necesita una gran realidad de información de la motocicleta y de como está siendo operada. Es aquí donde surge la necesidad de añadirle sensores electrónicos a la moto que “le dicen” a la centralita lo que está ocurriendo.



Ilustración 4. Centralita programable de competición [14]

Estos sensores además son vitales para los sistemas de seguridad del vehículo, que en las motocicletas no son tan numerosos como en los coches, pero son más importantes (aún si cabe), dado que la peligrosidad es más elevada. Los avances más importantes y que han requerido explícitamente de sensorística para su implementación son: el ABS y el Control de Tracción.

El ABS es una implementación y variación directa del que se ha realizado en coches, siendo el funcionamiento exactamente el mismo: cuando la rueda se bloquea por frenada máxima, el sistema acciona y libera el freno intermitentemente para sacar la rueda de la posición de equilibrio que significa el bloqueo, haciendo que vuelva a frenar correctamente.

El Control de Tracción, sin embargo, es una revolución electrónica muy diferente a su implementación para automóviles, dado que requiere un sistema de sensores más complejo.

Esto es porque la motocicleta se mueve en 3 dimensiones (al inclinar además de girar) mientras que un coche o camión se mueve exclusivamente en dos (realmente son 3 pero lo que bascula la suspensión es despreciable en comparación con los grados de inclinación de una motocicleta). El control de tracción observa la velocidad de ambas ruedas y el ángulo de inclinación del vehículo, permitiendo la aceleración proporcional según esté el piloto inclinando y si las ruedas están derrapando o no (velocidades diferentes). Esto ha supuesto un descenso drástico de un tipo de caídas muy

frecuentes y peligrosas en el mundo de las ruedas, el denominado highside, que se va a mostrar y detallar.

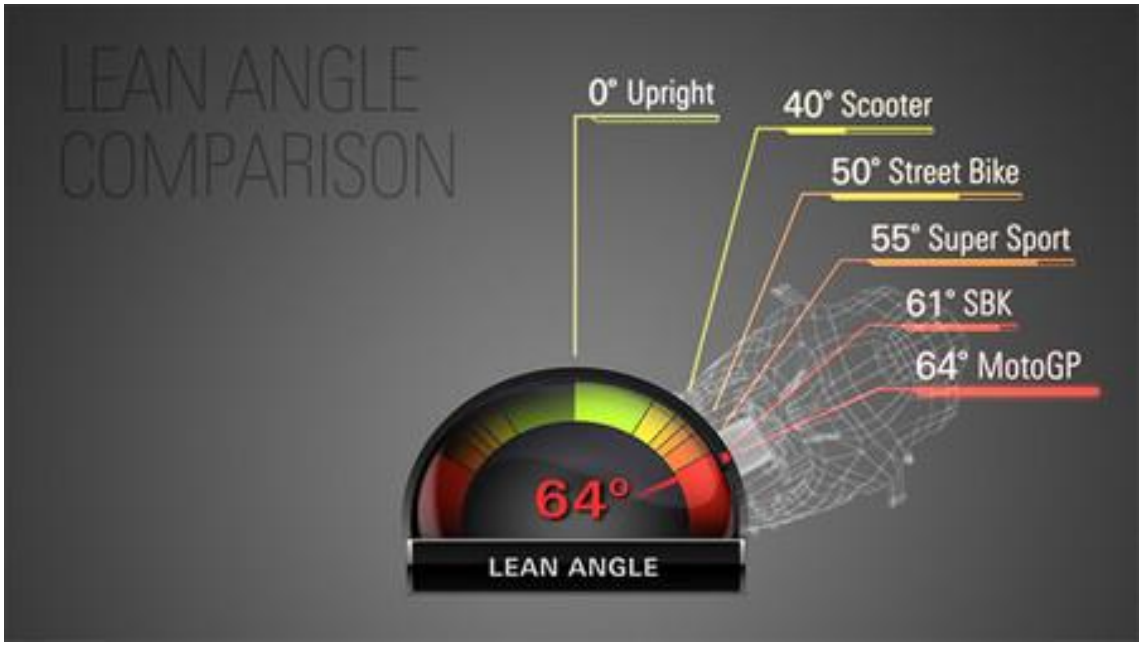


Ilustración 5. Ángulo de inclinación máximo en los diferentes tipos de motos [15]

En la secuencia de la *Imagen 6* se puede ver un highside que sufrió el piloto Jorge Lorenzo en el circuito de Shangai en 2008 por fallo de los sensores y, como consecuencia, la inclinación del Control de Tracción asociado. En el paso 1 la motocicleta está entrando en curva, en los pasos 2 y 3 se puede ver como la rueda trasera (la tractora), al tener mayor velocidad que la delantera, pierde adherencia. En menos de un segundo y antes de que el piloto pueda reaccionar, ocurre el paso 4, la rueda trasera recupera adherencia de golpe y al ir mucho más rápido que la delantera causa la contracción repentina del amortiguador trasero. En el paso 5, que ocurre escasas décimas de segundo más tarde, el amortiguador (por inercia) se estira de golpe, catapultando (paso 6) al piloto.

Todo esto no hubiera ocurrido de funcionar correctamente el sistema de sensores, dado que, al notar la disparidad de velocidades entre ruedas delantera y trasera, y además un cierto grado de inclinación, el sistema hubiera cortado la potencia del motor de inmediato. Evitándose de este modo el accidente antes descrito.



Ilustración 6. Caída del piloto Jorge Lorenzo en el GP Shangai 2008 [16]

Son ejemplos como el anterior, el motivo de la tremenda importancia de los sensores, y en concreto de la telemetría dado que evita caídas muy peligrosas en circuitos y fatales en la vía pública. Este sistema no es puramente telemetría hoy en día, pero vino heredado de ella, pues toda la información que necesita la ECU para "detectar" esta situación fue consultada y observada primero por los ingenieros con sistemas como el que (en estado prototipo) ocupa a este trabajo.

1.2. Telemetría aplicada a la competición

La adquisición de datos aplicada al mundo de la competición es una herramienta fundamental para obtener datos en tiempo real y mejorar el rendimiento en diversos ámbitos. A continuación, se enumeran algunas de las principales funciones de la telemetría:

- Monitoreo del rendimiento del vehículo. La telemetría proporciona información detallada sobre el rendimiento del vehículo, incluyendo datos como la velocidad, la aceleración, la frenada, la temperatura del motor y otros parámetros clave. Esto ayuda a los equipos a entender cómo se comporta el vehículo en diferentes situaciones y a realizar ajustes para optimizar su rendimiento.
- Análisis del comportamiento del piloto. La telemetría no solo recopila datos del vehículo, sino también del piloto. Puede medir aspectos como el estilo de conducción, la eficiencia en el uso de combustible, la consistencia en las

vueltas y la capacidad para aprovechar al máximo las características del circuito. Estos datos son valiosos para mejorar las habilidades de conducción y la estrategia durante la competición.

- Optimización de estrategias de carrera. Los datos de telemetría permiten a los equipos desarrollar estrategias de carrera más efectivas. Pueden analizar el rendimiento del vehículo y del piloto en tiempo real, lo que facilita la toma de decisiones sore cuándo ingresar a boxes, cuándo cambiar neumáticos y cómo ajustar la configuración del vehículo para adaptarse a las condiciones de la pista. Aunque se ha de destacar que en algunas competiciones la telemetría en tiempo real no está permitida. Por ejemplo, en el caso del motociclismo esto lo rige la Federación Internacional de Motociclismo (FIM).
- Diagnóstico de problemas. La telemetría ayuda a identificar y diagnosticar problemas mecánicos o eléctricos en el vehículo. Al monitorear constantemente los parámetros del motor y otros componentes, los equipos pueden detectar posibles fallas antes de que se conviertan en problemas graves, lo que permite intervenciones preventivas y minimiza el riesgo de averías durante la competición.
- Desarrollo y ajuste de la configuración del vehículo. Los datos de telemetría proporcionan información valiosa para el desarrollo y ajuste de la configuración del vehículo. Los ingenieros pueden analizar cómo diferentes configuraciones afectan al rendimiento y la aerodinámica, lo que ayuda en la optimización de la puesta a punto del vehículo para adaptarse a las características específicas de cada circuito.
- Mejora continua. La telemetría no solo se utiliza durante las competiciones, sino también en sesiones de prueba. Esto permite a los equipos recopilar datos en diversas condiciones y realizar análisis detallados para mejorar continuamente el rendimiento del vehículo y del piloto.

Para aplicaciones con requisitos elevados, como por ejemplo los campeonatos de velocidad, los diferentes equipos que forman parte de ellos pueden tomar dos caminos: comprar un sistema comercial profesional o desarrollar el suyo propio.

Los sistemas comerciales son muy variados y complejos, pero como ocurre con cualquier tipo de necesidad, existe una amplia gama tanto de marcas como de productos capaces de cumplir los requisitos del usuario adaptándose a él.

Una de las marcas comerciales con una mayor presencia y reputación en el mundo de la telemetría de competición (tanto para vehículos de dos como de cuatro ruedas) es *AiM Sportline*. El gran éxito de esta marca reside en su amplia gama de productos que cubren desde los niveles más altos de competición hasta servicios casi de *amateurs*, y su elevado rendimiento calidad-precio. Además, consiguió a nivel técnico innovaciones como hacer un todo en uno con el *display* y el *datalogger* y que sea completamente un dispositivo portátil. Otras marcas de telemetría comerciales son: *Beracing, Magneti Marelli, Dellorto, etc.* Para este tipo de sistemas, no es posible encontrar mucha información sobre la frecuencia de muestreo que utilizan, pero se ha podido conocer que aproximadamente toman una muestra cada 0,05 segundos.

En términos de sistemas de telemetría desarrollados íntegramente por equipos o marcas, es completamente imposible saber a ciencia cierta el número de sensores, la velocidad de adquisición, etc. En algunos textos divulgativos se habla de un número aproximado de 25-30 sensores diferentes para la competición de MotoGP (que es la que cuenta con los prototipos más complejos), pero es inviable encontrar información en términos de frecuencia de muestreo, resolución, etc.

1.3. Objetivos del sistema proyectado

Principalmente el objetivo del proyecto ha sido igualar la frecuencia de captación de datos de un sistema de telemetría de competición, pero intentando reducir los costes del producto todo lo posible para que sea accesible para prácticamente cualquier usuario. Lo que diferencia este sistema de telemetría de uno convencional es que los datos serán enviados a una nube en línea en la que se almacenan, y serán accesibles desde una página web. Y además gracias a la pantalla que posee el dispositivo pueden estos datos ser mostrados al piloto en tiempo real.

CAPÍTULO 2: COMPONENTES DEL DISPOSITIVO

En este apartado se realizará la búsqueda de los componentes necesarios para la medición de las variables requeridas para la funcionalidad del dispositivo, se realiza una búsqueda en el mercado comparando las diferentes características de los módulos, para su posterior selección del más adecuado para la tarea a realizar.

2.1. Módulo GPS

Un módulo GPS (Sistema de Posicionamiento Global) es un componente esencial en un sistema de telemetría, ya que proporciona información precisa sobre la ubicación geográfica de un objeto en movimiento.

El Sistema de Posicionamiento Global (GPS) opera mediante una constelación de 24 satélites en órbita alrededor de la Tierra, con trayectorias cuidadosamente sincronizadas para abarcar la totalidad de la superficie terrestre. El método principal utilizado para localizar dispositivos receptores, como los módulos GPS, es la triangulación. Cuando se busca determinar la posición, el receptor identifica automáticamente tres satélites de la constelación y recibe señales que contienen información sobre la identificación de cada satélite y la hora de emisión de las señales.

Basándose en estas señales, el dispositivo sincroniza su reloj con el del sistema GPS y calcula el tiempo que tardan en llegar las señales desde los satélites al módulo receptor. Este tiempo de viaje se utiliza para medir las distancias entre el receptor y cada uno de los satélites. Conocidas estas distancias y la posición relativa de cada satélite, derivada de la señal que emiten, el módulo GPS puede calcular con precisión las coordenadas reales o la posición absoluta del punto de medición. Este proceso de triangulación permite una determinación precisa de la ubicación geográfica del dispositivo receptor en cualquier lugar de la tierra.

2.1.1. Funcionalidades del módulo GPS

Una de las principales funciones del módulo GPS es el registro de trayectoria, ya que los módulos GPS pueden almacenar datos de trayectoria a lo largo del tiempo, permitiendo el registro de la tura seguida por el objeto en movimiento. Esta información es valiosa para el análisis post-evento y para evaluar patrones de movimiento.

Otra de las funcionalidades a destacar es el tiempo sincronizado, ya que los módulos GPS también proporcionan información precisa sobre la hora, y esta función es esencial para la sincronización de datos en sistemas de telemetría. El obtener una marca de tiempo en e el momento que se obtienen los datos de manera precisa, facilita la correlación de eventos específicos con la ubicación y otros parámetros.

Además, se ha de destacar que gracias a la integración con otros sensores, los datos del módulo GPS se complementan con acelerómetros, giroscopios o sensores de temperatura, para obtener una visión más completa del rendimiento del objeto en

movimiento. De esta manera se obtienen con más exactitud los datos, por ejemplo, de velocidad y posicionamiento dentro de la pista.

2.1.2. Opciones en el mercado

Se elaboró un estudio de mercado en el cual se estudiaron las diferentes posibilidades que existen dentro del mundo de los módulos GPS en la página web "Mouser", ya que este es el lugar donde se adquirirán los materiales necesarios para la placa PCB. En la siguiente tabla se pueden observar los modelos que fueron sujetos de estudio, se estudió minuciosamente diferentes parámetros entre los cuales se destacan: frecuencia de rastreo, consumo en funcionamiento, protocolos de comunicación, dimensiones y precio de cada uno de los diferentes módulos.

Comparación módulos GPS:

Modelo	Precio unitario (€)	Precio x10 (€)	Consumo en funcionamiento modo tracking (continuous mode)(mW)	IMU	Frecuencia de rastreo (Hz)
MAX-M10S	19,53	16,24	25	No	25
NEO-M9N-00B	25,11	22,25	108 (Variable*)	No	25
EVA-M8M	26,97	13,67	66	No	18
SAM-M10Q	29,30	20,88	30	No	25
MIA-M10Q	26,97	13,67	25	No	25
Teseo-VIC3DA	20,50	18,95	193	Si	30

Tabla 1. Comparación módulos GPS

El consumo ha sido calculado suponiendo una alimentación de una pila de 3V.

Variable*(NEO-M9N-00B NEO-M9N-00B): El consumo de este módulo de GPS es variable dependiendo a las constelaciones a las que se conecte, se ha de destacar que está diseñado para que el consumo de potencia sea el mínimo posible en cada caso.

De los módulos GPS que se han encontrado, el que más se ajusta a las necesidades tanto económicas, como de frecuencia de rastreo y consumo mínimo es el MAX-M10S.

El MAX-M10S admite la recepción simultánea de cuatro GNSS (sistema global de navegación por satélite) GPS, GLONASS, Galileo y BeiDou. Esto maximiza la disponibilidad de la posición del dispositivo. Gracias al consumo extremadamente bajo de 25mW en modo de seguimiento contínuo permite una gran autonomía energética para el dispositivo ya que funcionará con batería sin comprometer el rendimiento GNSS.

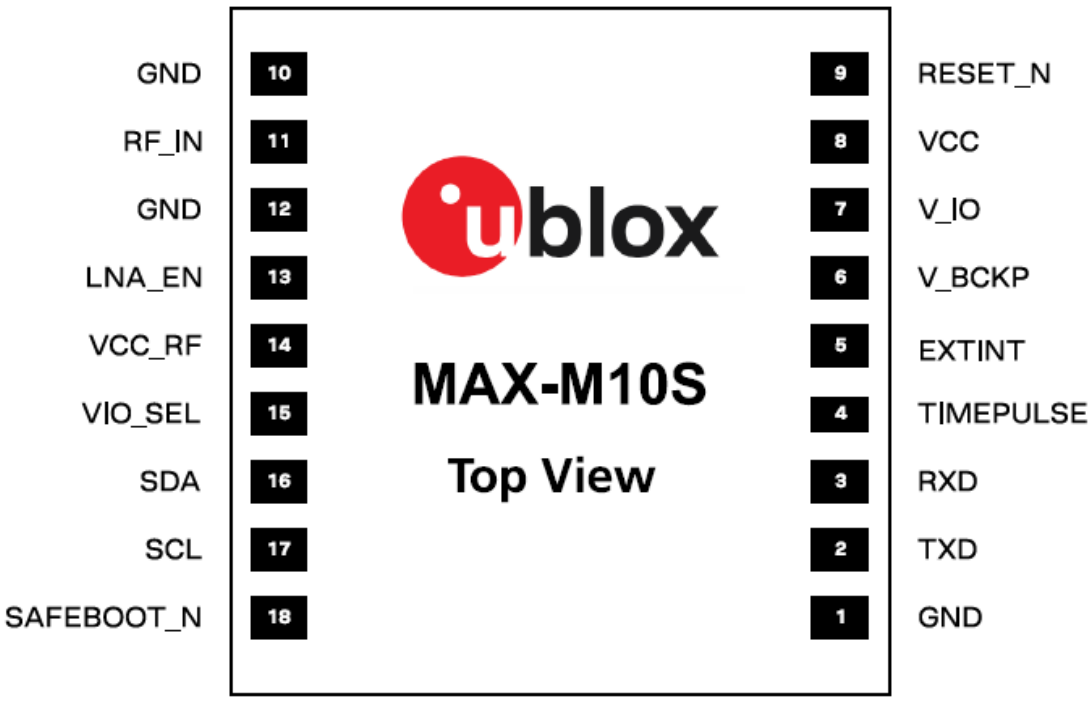


Ilustración 7. Asignación de pines MAX-M10S [17]

2.2. IMU

La Unidad de Medición Inercial, el IMU, desempeña un papel crucial en un sistema de telemetría para motocicletas o cualquier vehículo al proporcionar datos precisos sobre la dinámica del vehículo. Se destacan las siguientes destrezas de este sensor:

1. Medición de aceleración y velocidad angular:

La IMU mide la aceleración lineal y la velocidad angular del objeto a medir. Estos datos son esenciales para comprender la dinámica de movimiento, incluyendo aceleración frenado y cambios de dirección.

2. Determinación de la orientación:

La IMU ayuda a determinar la orientación del objeto en el espacio tridimensional. Esto es fundamental para conocer la inclinación y la posición del objeto, especialmente en curvas y cambios de dirección.

3. Estabilidad y control dinámico:

La información de la IMU se utiliza para evaluar la estabilidad y el control dinámico una motocicleta. Esto es crucial para ajustar parámetros como la suspensión electrónica y otros sistemas de control para optimizar el rendimiento y la seguridad.

4. Análisis de inclinación (cabeceo, alabeo y guiñada):

La IMU permite medir la inclinación en los ejes de cabeceo (inclinación lateral), alabeo (inclinación longitudinal) y guiñada (rotación alrededor del eje vertical). Estos datos son fundamentales para comprender el comportamiento dinámico de la motocicleta en diferentes situaciones.

5. Sistemas de estabilización electrónica:

La IMU se utiliza en sistemas de estabilización electrónica, como el control de tracción y el sistema de frenado antibloqueo (ABS). Estos sistemas se ajustan en tiempo real para adaptarse a las condiciones de conducción.

6. Detección de ángulo de inclinación en curvas:

La IMU es esencial para detectar y medir el ángulo de inclinación de la motocicleta al tomar curvas. Estos datos son críticos para ajustar la electrónica y mejorar la estabilidad durante la conducción en curvas.

7. Generación de datos para análisis posterior:

Los datos recopilados por la IMU se almacenan para su posterior análisis. Esto permite a ingenieros y pilotos revisar el rendimiento de la motocicleta, identificar áreas de mejora y ajustar configuraciones para optimizar el rendimiento.

2.2.1. Opciones de mercado

Se elaboró un estudio de mercado en el cual se estudiaron las diferentes posibilidades que existen dentro del mundo de los módulos IMU en la página web “Mouser”, ya que este es el lugar donde se adquirirán los materiales necesarios para la placa PCB. En la siguiente tabla se pueden observar los modelos que fueron sujetos de estudio, se estudió minuciosamente diferentes parámetros entre los cuales se destacan: frecuencia de rastreo, consumo en funcionamiento, protocolos de comunicación, dimensiones en los diferentes ejes que puedan medir y precio de cada uno de los diferentes módulos.

Comparación IMUs:

Modelo	Precio unitario (€)	Precio x10 (€)	Ejes axiales
BMI323	4,46	2,86	6
BHI360	6,08	4,87	6
ASM330LHHXTR	13,02	10,30	6
LSM6DSV16XTR	6,29	5,03	6
BNO055	11,56	8,65	9

Tabla 2. Comparación IMUs

ASM330LHHXTR:

Características más importantes:

- AEC-Q100 qualified: es un certificado que asegura la viable utilización de esta Unidad de Medida Inercial en el sector automotriz. Se utiliza para establecer requisitos y pruebas de calidad y confiabilidad.
- Funcionamiento en un rango de temperatura entre -40 y 105ºC.
- Compatible con compilador Android.
- Capacidad de seleccionar el rango de medición del acelerómetro según las necesidades. Puede medir hasta +-16g. Un “g” representa la aceleración debida a la gravedad terrestre, por lo tanto, el acelerómetro puede medir aceleraciones en un rango que va desde -157,44m/s^2 hasta +157,44m/s^2.
- El giroscopio tiene un rango de medición que se puede configurar para medir tasas de cambio angular en un rango que va desde -125dps (grados por segundo) hasta +4000dps.
- Protocolos de comunicación I2C, MIPI I3C y SPI.
- Tamaño: 2,5 x 3,0 x 0,83 mm3.

BMI323:

Características más importantes:

- El acelerómetro es capaz de medir hasta +-16g, teniendo la capacidad de ser configurable desde +-2g. (16 bit digitales)
- Giroscopio con rango configurable desde +-125ºdps hasta 2000dps. (16 bit digitales)
- Sensor de temperatura y rango de funcionamiento en temperaturas desde -40º hasta 85º. (16 bit digitales)
- Tamaño: 2,5 x 3,0 x 0,83 mm3.

- Protocolos de comunicación I2C, MIPI I3C y SPI.
- Compensación de erros de offset de acelerómetro y giroscopio.
- Sincronización mediante hardware de acelerómetro, giroscopio y temperatura.
- Filtro de paso bajo programable.

BNO055:

Características más importantes:

- El acelerómetro es capaz de medir hasta +-16g, teniendo la capacidad de ser configurable desde +-2g. Con filtro de paso bajo programable con un ancho de banda desde 1kHz -< 8Hz. (14 bit)
- Giroscopio con rango configurable desde +-125ºdps hasta 2000dps. (16 bit digitales). Con filtro de paso bajo programable con un ancho de banda desde 512Hz -< 12Hz.
- Magnetómetro con un campo magnético con un rango de +-1300µT, en ejes x-y, y +-2500 en el eje z.
- Tamaño: 3,8x5,2x1x13 mm3.
- Rango de funcionamiento en temperaturas desde -40º hasta 85º.
- Protocolo de comunicación I2C.

LSM6DSV16X:

Características más importantes:

- El acelerómetro es capaz de medir hasta +-16g, teniendo la capacidad de ser configurable desde +-2g.
- Tamaño: 2,5 x 3,0 x 0,83 mm3.
- Giroscopio con rango configurable desde +-125ºdps hasta 4000dps. (16 bit digitales).

- Protocolos de comunicación I2C, MIPI I3C y SPI.
- Sensor de temperatura y rango de funcionamiento en temperaturas desde -40º hasta 85º.

La elección por la que se ha optado ha sido por el dispositivo BNO055, ya que, aunque sea más caro que algunos de los que se han expuesto anteriormente, nos brinda una calidad mucho mayor y una capacidad de medición de 9 ejes frente a los 6 ejes de los demás dispositivos.



Ilustración 8. BNO055 [18]

2.3. Módulo SIM

Al ser la idea principal buscar un módulo que ya viniera integrado con el ESP32 y el módulo SIM. Se buscaron en la página de la compañía china Xinyuan-LilyGO los siguientes modelos, los cuales se estudiaron para ver cual era el que mejor cumplía los requerimientos del dispositivo que se presenta.

Comparación módulo SIM:

Modelo	Tamaño(cm)	Batería	MCU	Precio (Dólares)	Conectividad

T-SIM7600E/G-H	10,5x3,3	18650 (Posibilidad de carga solar)	ESP32	49,78	2G-3G-4G
T-A7670E/G/SAR2	10,5x3,3	18650	ESP32	32,89	2G-3G-4G
T-SIM7080G-S3	11x3,2	Módulo de pila externa	ESP32-S3	29,60	2G-3G-4G
T-SIM7000G (solo nano SIM)	10,6x3	18650 (Posibilidad de carga solar)	ESP32	35,14	2G
T-SIM7070G (solo nano SIM)	11x3,3	18650 (Posibilidad de carga solar)	ESP32-WROVER-E	26,59	2G
T-SIM7600E-L1 (solo nano SIM)	10,5x3,3	18650 (Posibilidad de carga solar)	ESP32	40,98	2G-3G-4G
T-SIM7000E (solo nano SIM)	10,6x3	18650 (Posibilidad de carga solar)	ESP32-WROVER-B	35,87	2G-3G-4G

Tabla 3. Comparación módulos SIM

Teniendo en cuenta los datos de cada uno de los módulos SIM, se podría concluir que el módulo T-SIM7080G-S3 es el más conveniente calidad-precio, ya que posee el MCU más potente, lleva la especificación S3 del modelo ESP32, y la capacidad de conectarse a la red 2G-3G-4G. En cambio, si se requiere menos velocidad de transferencia de datos, el modelo T-SIM7070G podrá cumplir con las especificaciones necesarias también, ya que equipa el MCU ESP32-WROVER-E posee más memoria flash y RAM que un ESP32 común, además incluye diversos conectores para antenas externas, puertos GPIO, periféricos… también tiene la ventaja de que se puede programar en el entorno de Arduino. El módulo SIM7070 al funcionar en redes más

antiguas como el 2G, consume menos energía que si tuviera que conectarse a una red 4G LTE.

También se debe tener en cuenta la condición de que la red 2G, al ser una red antigua puede quedar sin soporte en algún momento.

El módulo elegido es el T-SIM7080G-S3.

2.4. Display

Para la parte visual del proyecto, se va a utilizar la placa T-Display-S3 de la compañía china Xinyuan-LilyGO, dedicada a los componentes y sistemas electrónicos.

Basada en un microprocesador ESP32-S3R8 de doble núcleo y pensada para su uso en proyectos de electrónica y robótica, la T-Display-S3 incluye una pantalla táctil LCD de 1,9 pulgadas que tiene una resolución de 170x320 píxeles y una interfaz paralela de 8 bits para su conexión.

El chip de controlador utilizado para la pantalla es el ST7789V, lo que garantiza una calidad de imagen nítida y colores vibrantes en todos los ángulos de visualización.

Además, dicha pantalla es compatible con STEMMA QT/Qwiic y el conector JST-SH de cuatro pines de 1mm, lo que facilita su integración en distintos proyectos.



Ilustración 9. T-Display-S3 [19]

El microprocesador ESP32-S3R8 dispone de dos núcleos LX7, ofreciendo una gran potencia de procesamiento y una excelente conectividad inalámbrica, compatible con los estándares Wi-Fi 802.11, Bluetooh de bajo consumo (BLE por sus siglas en inglés) 5, con capacidad de red mallada (mesh), lo cual facilita su empleo en aplicaciones de la IoT y proyectos de automatización doméstica.

También dispone de una memoria flash de 16 MB y una memoria PSRAM de 8 MB, lo cual constituye una gran capacidad de almacenamiento para proyectos de mayor complejidad.

- Manejo y programación.

Para su programación, deberemos usar el entorno de desarrollo (IDE) de Arduino, o Micropython, lo que hace que sea más accesible para cualquier persona con conocimiento básicos de programación.

En cuanto a la alimentación, funciona con una fuente de alimentación de 3,3V y cuenta con configuraciones adicionales como arranque+reinicio+botón IO14, que facilitan su uso y configuración.

CAPÍTULO 3: PROTOTIPOS

3.1. Primer prototipo

Se realiza un primer prototipo basado en la placa PBC T-SIM7080G-S3 perteneciente a la página web de Lilygo. Se eligió esta placa debido a la potencia del procesador que trae y a la capacidad de conexión del dispositivo SIM7080.

Acoplando a esta placa los siguientes dispositivos:

- Dos sistemas de posicionamiento. Un GPS convencional (el MAX-M10S) que es un GPS muy potente y con una gran frecuencia de muestreo. Además de un dispositivo llamado DWM3000 que se basa en un transceptor IC de banda ultra ancha, que permite la localización del objeto con una precisión de 10 cm. El módulo tiene soporte UWB mundial a través de su operación en el canal UWB 5 (6,5 GHz) y el canal 9 (8 GHz). Su funcionamiento se basa en recibir una señal emitida por un dispositivo situado en un sitio el cual emite una señal la cual recibe el DWM3000 y así es capaz de realizar la localización.
- Dos unidades de medida inercial (IMU). La unidad BNO055 con un coste superior y con unas características más premium, y la unidad BMI323 que satisface unas necesidades generales, pero posee peores características.

Esto unido al SIM7080G que posee la placa de base, permite que se obtenga un prototipo a priori eficiente para las tareas que se requieren.

Este prototipo se quedó en esta versión y no pasó a la siguiente fase, la cual sería la de añadir una display capaz de mostrar por pantalla algunos datos interesantes y hacer el dispositivo más completo y funcional. A continuación, se muestra el esquemático del circuito de la PCB realizado en la plataforma EasyEDA:

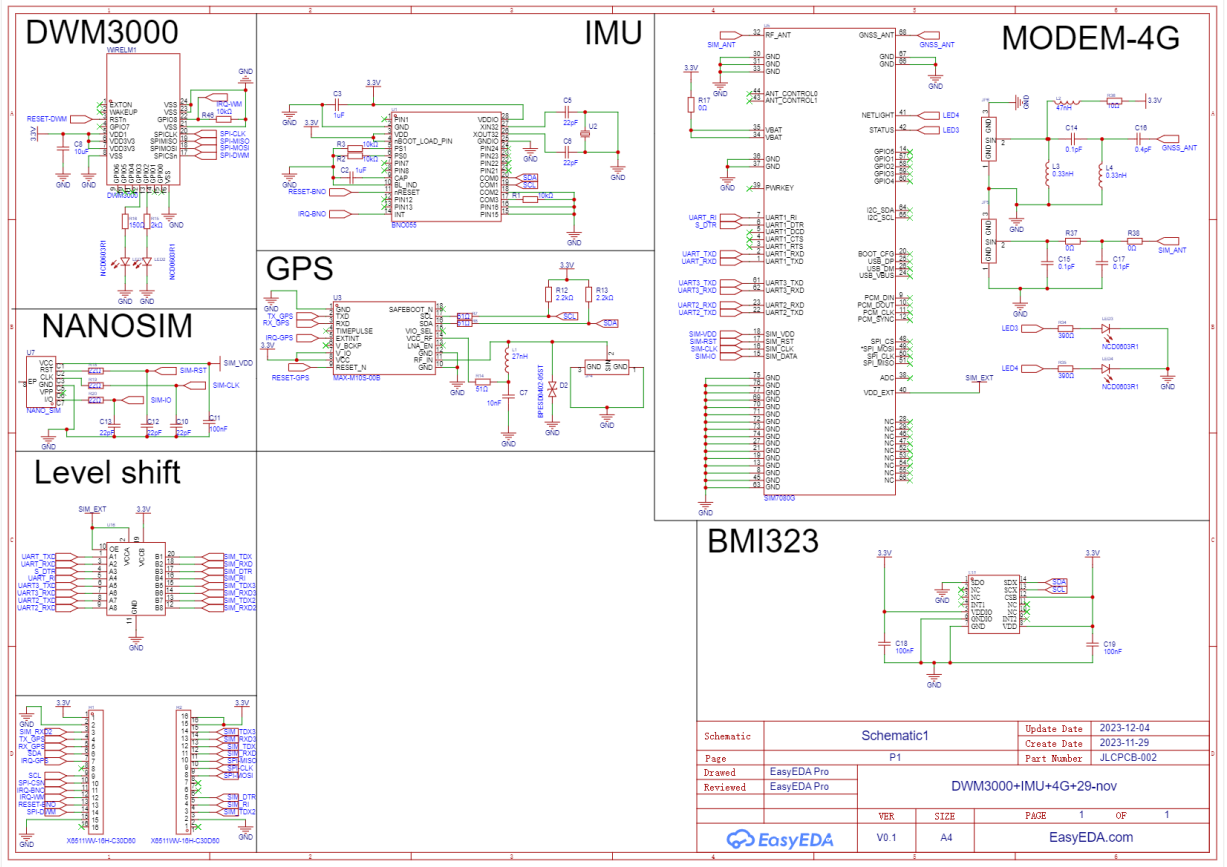


Ilustración 10. Esquemático prototipo 1

Una vez realizado el esquemático se procede a asignar las dimensiones de la placa para que coincidan con las de la T-SIM7080G-S3. Se realizó una placa con 4 capas de cobre, ya que así se puede tener una capa para los componentes que van conectados a tierra y una capa para los componentes que van conectados al VCC que en este caso será una pila de 3.3V.

En el ANEXO X se puede observar como fue el ruteo de todos los componentes de la PCB que se muestra a continuación en la imagen 3D.

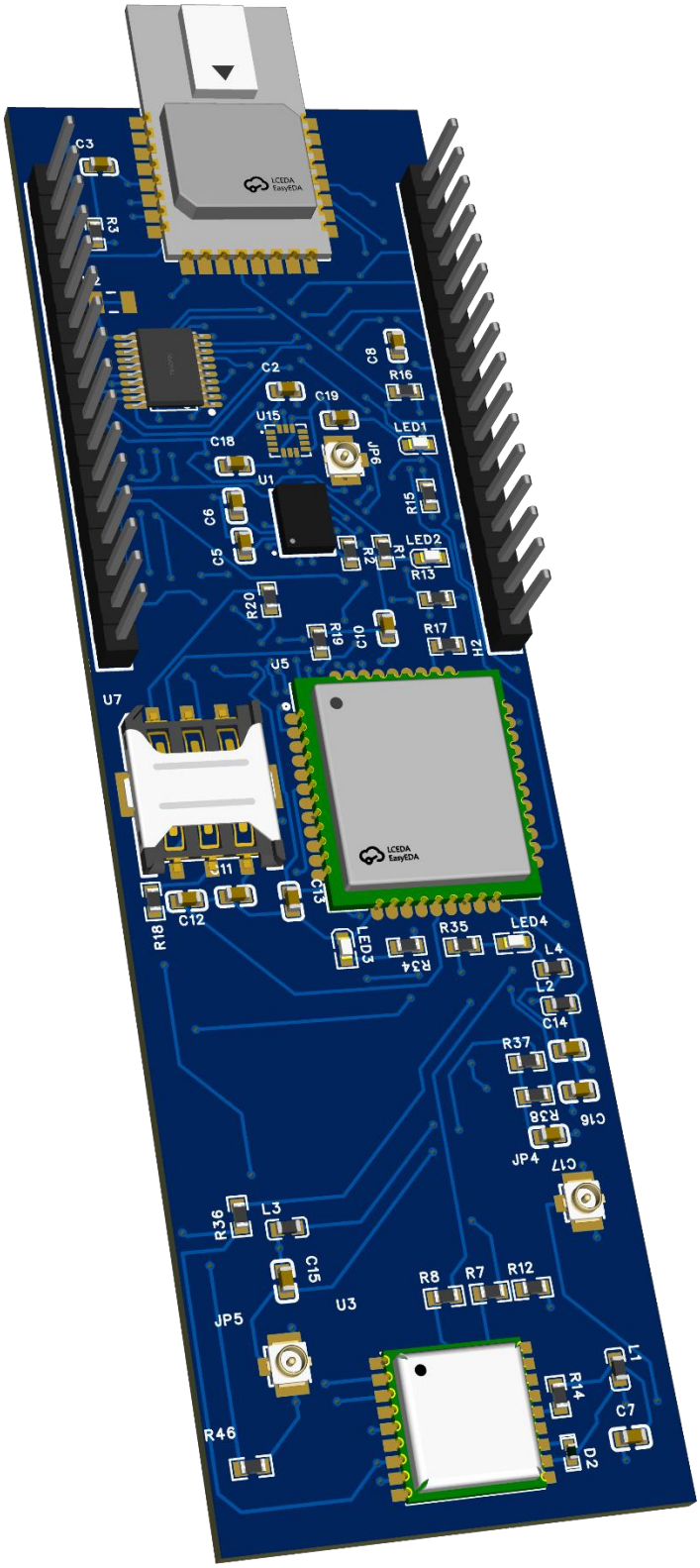


Ilustración 11. Vista frontal PCB primer prototipo

3.2. Segundo prototipo

En este avance del proyecto se optó por una versión más simple que la primera, priorizando que los componentes que lleve la PCB sean los siguientes.

- GPS. Se deja como único dispositivo localizador el MAX-M10S, ya que con su gran frecuencia de muestreo de 25Hz es suficiente para los requerimientos que se le exigen a la telemetría deseada.
- IMU. Se opta por utilizar únicamente la BNO055 debido a sus mejores prestaciones.
- SIM7080. Se utiliza el módulo STAMP Cat-M, el cual es una versión global compacta del módulo de comunicación inalámbrica de doble modo CatM & NB-IoT basado en la solución SIM7080G.

En vez de basar la nueva PCB en la T-SIM7080G-S3, se optó por ir directamente a realizar el diseño de esta nueva placa en base a la siguiente fase del proyecto que es la de acoplar un display para visualizar diversos datos que recopile el dispositivo. Por lo que se eligió la placa T-DISPLAY-S3, y en base a las dimensiones y el conexionado de pines de esta placa, se crea un nuevo diseño el cual se colocará encima de esta como se mostrará a continuación. Primero se realizó el nuevo esquemático de este segundo prototipo:

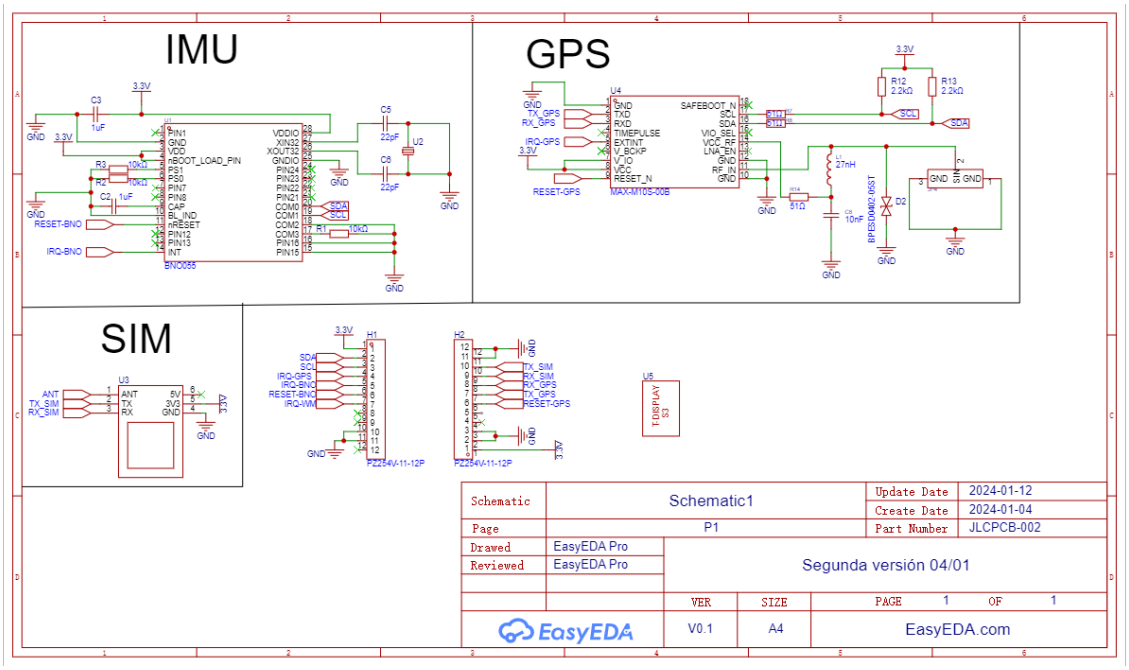


Ilustración 12. Esquemático segundo prototipo

Y posteriormente se llevó a cabo el ruteo correspondiente para el conexionado de los componentes, de nuevo la PCB se compone de 4 capas de cobre como la

anteriormente diseñada. A continuación, se puede observar como sería la placa vista en 3D desde diferentes perspectivas:

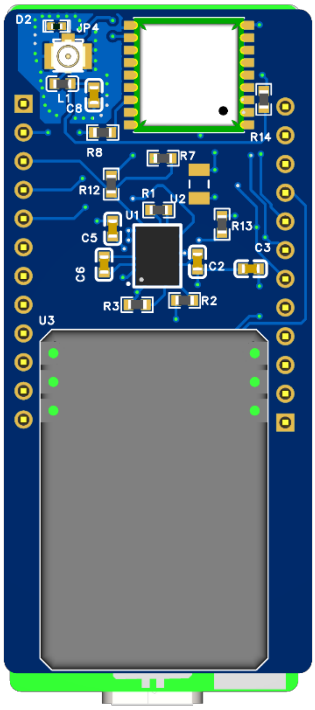


Ilustración 13. Vista trasera PCB segundo prototipo

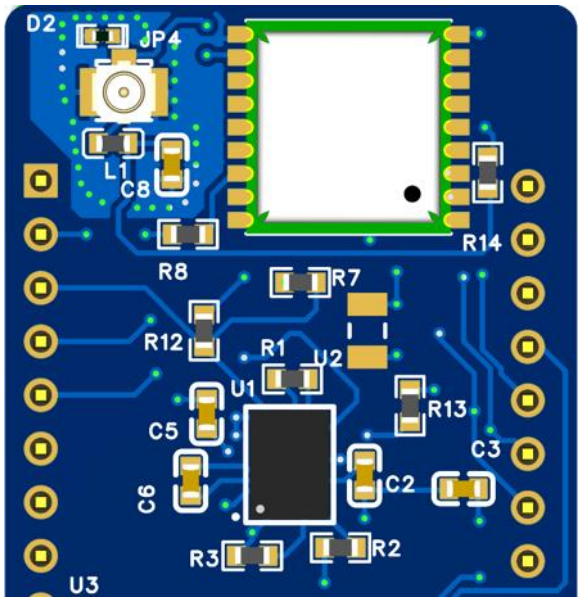


Ilustración 14. Detalle de conexionado

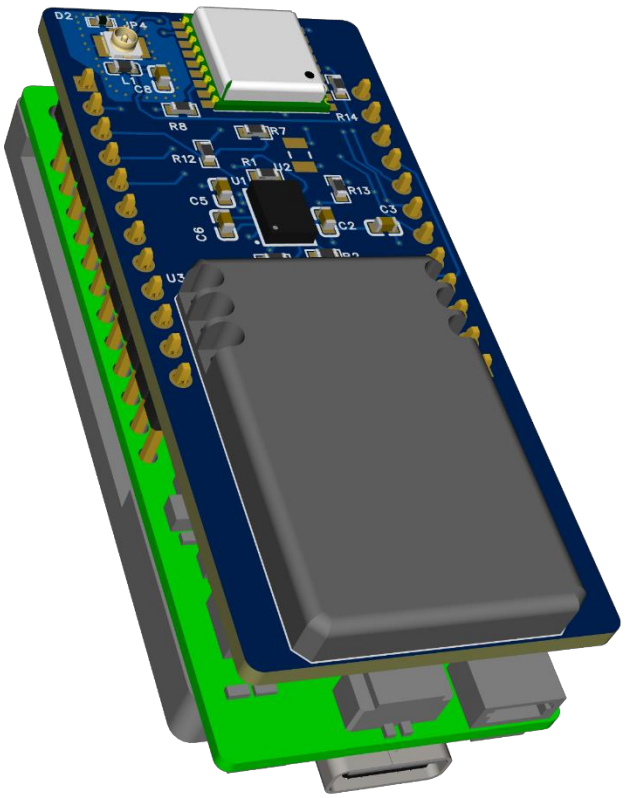


Ilustración 15. Vista fronto/lateral PCB del segundo prototipo

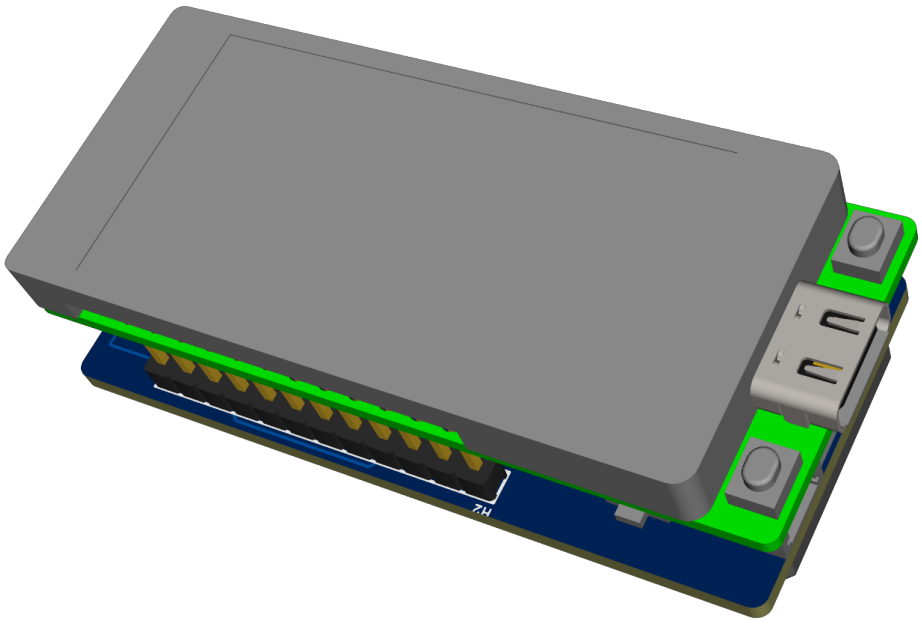


Ilustración 16. Vista superior PCB segundo prototipo

Para realizar el correcto montaje de la PCB del diseño que se ha realizado en EasyEDA, se siguieron los siguientes pasos:

- Estimación de las dimensiones que debe tener la placa para encajar a la perfección con la pantalla a la que se acopla.

- Disposición de pines para encajar correctamente con el conexionado con el ESP32-S3.

- Colocación de la IMU en el centro de la placa para obtener unos datos correctos a la hora de estar el dispositivo en movimiento.

- Alejar lo máximo posible el GPS de los demás elementos de la placa para que así no haya interferencias.

- Añadir una capa adicional de GND alrededor de los componentes pasivos del GPS y la antena, para así conseguir una aislación perfecta.

- Colocar el módulo SIM en la parte que queda libre.

Para el diseño de la placa PCB se han utilizado 4 capas de cobre:

- La primera superficial, la llamada capa Top Layer en la cual se encuentran los componentes pasivos, además de los módulos GPS, IMU y SIMCARD.

- La segunda capa que se menciona es la Bottom Layer, es la capa de cobre que se encuentra en la parte inferior de la placa. Esta ha sido utilizada para trazar circuitos y facilitar la conexión de algunos componentes entre sí ya que no fue sencillo encontrar las rutas correctas para la conexión de todos los componentes.

- Además, se añadieron una capa intermedias de GND(para proporcionar una referencia de bajo ruido) y VCC, para proporcionar planos de alimentación y tierra, y así mejorar la integridad del diseño del circuito.

Para el ruteado de la placa se han tenido en cuenta los siguientes condicionantes:

- Todos los cables han de evitar los ángulos rectos (90º), para evitar posibles problemas, como la generación de puntos calientes, acumulación de residuos durante la fabricación o la reflexión y discontinuidad de impedancias en los trazados.

- La utilización óptima del número de vías planificando cuidadosamente el ruteo de las señales, aprovechando las capas dedicadas GND y VCC, además de la Bottom para que no choquen las rutas de cables entre sí.

- Respetar las reglas de diseño como son el espaciado mínimo entre las trazas, vías y componentes siguiendo las especificaciones del fabricante. Además de que el ancho de traza sea el suficiente para transportar la corriente necesaria.

- Revisión exhaustiva del diseño utilizando herramientas de verificación DRC (Design Rule Check) y ERC (Electrical Rule Check) disponibles en EasyEDA.

- Documentación y etiquetado adecuado de todas las señales, componentes y conexiones para facilitar el ensamblaje y mantenimiento. Atendiendo además a los pines específicos que posee el ESP32-S3 para realizar la comunicación con los módulos, ya fuera por UART o I2C.

CAPÍTULO 4: MÉTODOS DE ADQUISICIÓN Y VISUALIZACIÓN DE DATOS

4.1. Protocolos de comunicación utilizados

4.1.1. UART

El nombre de este protocolo de comunicación corresponde a las siglas en inglés de Universal Asynchronous Receiver-Transmitter que es el dispositivo que controla los puertos y dispositivos serie del circuito.

Las funciones principales del UART son manejar interrupciones de dispositivos conectados mediante los puertos serie y convertir esos datos de formato paralelo a formato serie, realizando una transmisión de datos individual la cual es reensamblada en el UART receptor donde estos datos pueden ser utilizados.

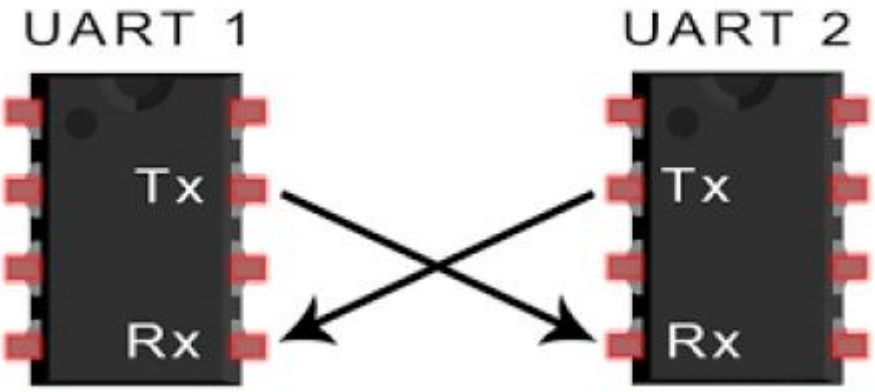


Ilustración 17. Funcionamiento protocolo UART [11]

Como se puede observar en la imagen, estos buses están compuestos por 2 pistas TX y RX (además de la alimentación y la masa) conectado el TX de un módulo al RX del otro y viceversa, consiguiendo de esta manera una comunicación bidireccional de ambos módulos donde ambos son capaces de enviar y recibir información.

Este tipo de protocolo de comunicación es bastante utilizado en programación de sistemas embebidos debido a que no es necesario que los microcontroladores implicados cuenten con un gestor de arranque o “bootloader” precargado.

Mediante este protocolo se programan el módulo GPS y el módulo SIM.

4.1.2. I2C

El protocolo de comunicación I2C recibe su nombre del inglés Inter-Integrated Circuit y es uno de los protocolos más utilizados para la comunicación interna de los circuitos, permitiendo el paso de información entre el controlador principal y los distintos periféricos o módulos que se integran.

El I2C está diseñado como un sistema maestro-esclavo donde la transferencia de datos es siempre iniciada por un maestro y se espera la reacción del sistema esclavo. Existe la posibilidad de configurar un sistema multimaestro y el control para el acceso al bus de datos, se realiza mediante la programación consiguiendo que todos los maestros puedan turnarse para su uso.

El uso del protocolo I2C necesita cuatro líneas de conexión, una de alimentación a VCC, otra de masa a GND y dos líneas de señal, el reloj CLK (Serial Clock) y la línea de datos SDA (Serial Data) siendo necesarias resistencias pull-up hacia VCC para ambas líneas de señal.

Es importante tener en cuenta que la comunicación mediante el bus I2C funciona con lógica positiva, siendo un nivel alto en la línea de datos un 1 lógico y un nivel bajo correspondiente a un 0.

El I2C utiliza un espacio de direccionamiento 1 byte donde los 7 primeros bits representan la dirección permitiendo hasta 112 nodos en un bus. El octavo bus es el que comunica al esclavo si debe recibir dato del maestro (nivel bajo) o enviarlos (nivel alto). Es importante tener en cuenta que todos los circuitos integrados tienen una dirección determinada por el fabricante en la cual los tres últimos bits pueden ser fijados como pines de control.

Tal y como se especifica en el párrafo anterior, el último bit de la cadena de datos enviados es el que define la función que desempeñará el esclavo.

Mediante este protocolo se programa el módulo IMU.

4.2. Pines de conexión

Para las conexiones con el ESP32-S3, se sitúan en cada uno de los pines de la siguiente manera, atendiendo a las funciones que tiene cada uno de los pines del ESP32-S3.

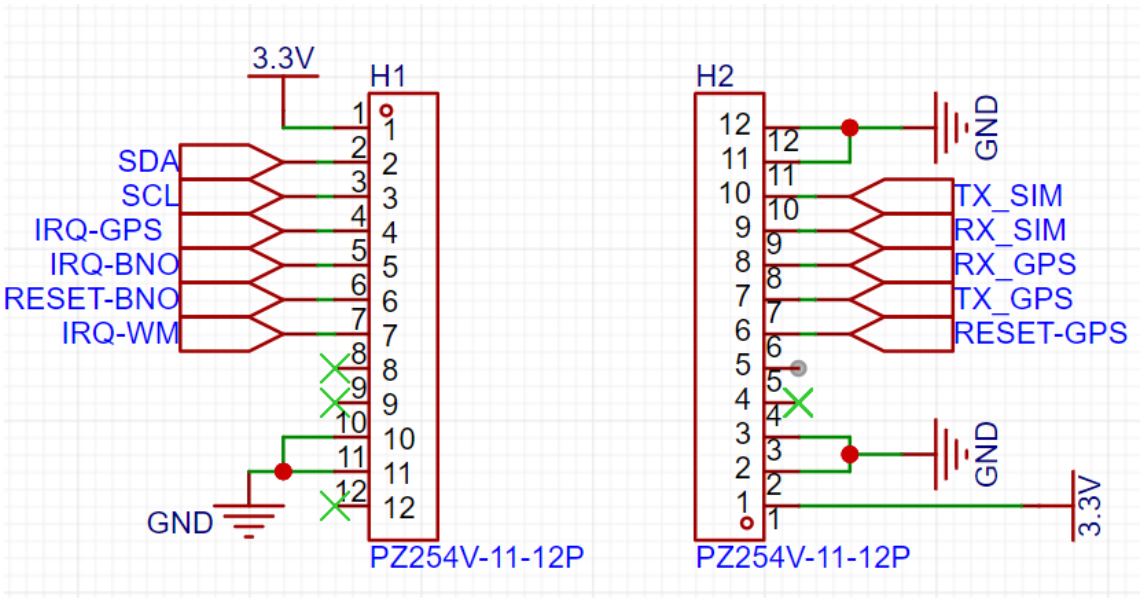


Ilustración 18. Conexión de pines al ESP32-S3

4.3. Programación en Square Line Studio y Arduino IDE

Gracias a la facilidad de programación que posee el ESP32-S3, permite que se realice su desarrollo con diferentes plataformas. Entre ellas está el propio IDE de Espressif, Micropython, el IDE de Arduino, entre otras. En este caso se ha optado por realizar el desarrollo en el IDE de Arduino ya que era el que más convenía para la facilidad de obtener las librerías necesarias ya que hay mayor cantidad de repositorios de librerías en él. Además, el desarrollo de la programación de la pantalla se realizó mediante el programa Square Line Studio, este programa permite realizar un desarrollo intuitivo mediante bloques a elegir en su entorno y al finalizar el diseño de la pantalla, da la opción de extraer el código de programación del entorno realizado para diferentes lenguajes, entre ellos el de Arduino.

4.3.1. Square Line Studio

Mediante este software se llevará a cabo el diseño de la interface de la pantalla T-Display-S3, la principal ventaja que ofrece este software frente a la programación convencional de pantallas mediante librerías como la “TFT_espi”, en la cual se debe ir

programando de una manera más específica y lenta cada parte de la pantalla. Es que este software proporciona una vez acabado el diseño realizado, el código de lo diseñado mediante los archivos “ui_x.c o ui_x.h” para poder adaptarlo al programa principal que se tenga simplemente sabiendo manejar la librería LVGL.



Ilustración 19.Interfaz Square Line Studio

A la izquierda de la pantalla en el panel llamado “Hierarchy”, se puede cambiar de una pantalla a la otra que se han creado, además de observar los componentes que posee la pantalla que se selecciona, en este caso la “Screen1”.

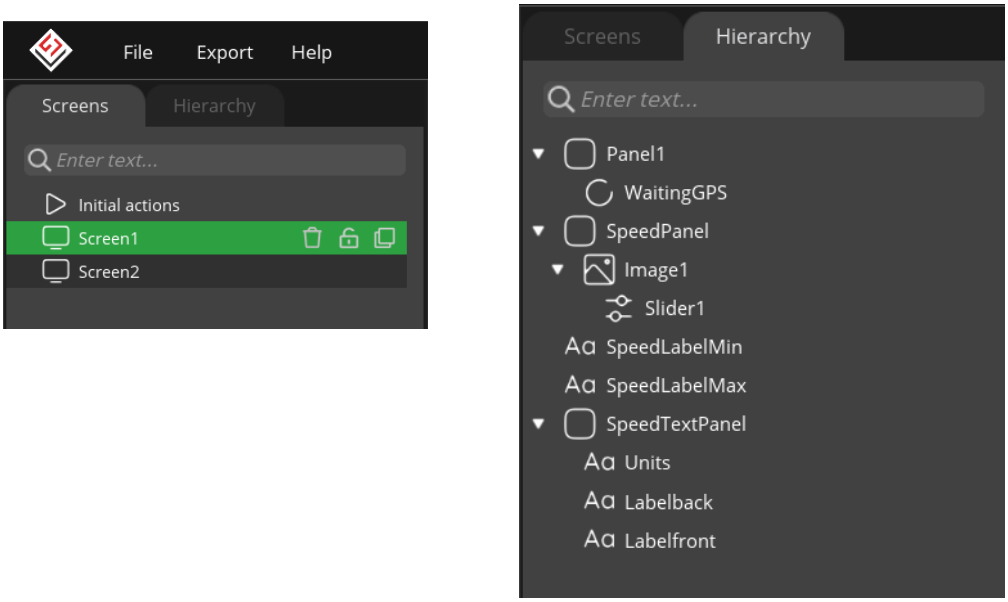


Ilustración 20. Diferentes opciones square line studio

En la parte de abajo a la izquierda aparecen los Widgets que el programa da la opción de añadir en la pantalla.

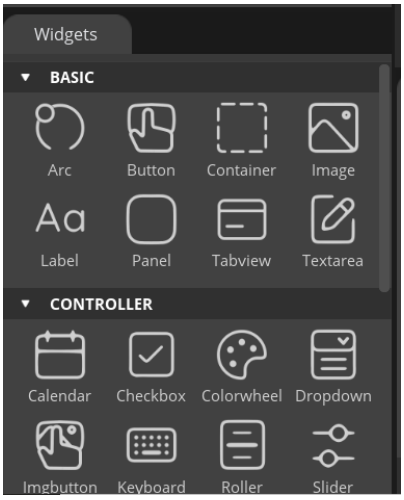


Ilustración 21. Configurador de diseños de square line studio

En la parte centrar se puede posicionar y ver como se personaliza la pantalla al gusto que el usuario quiere.

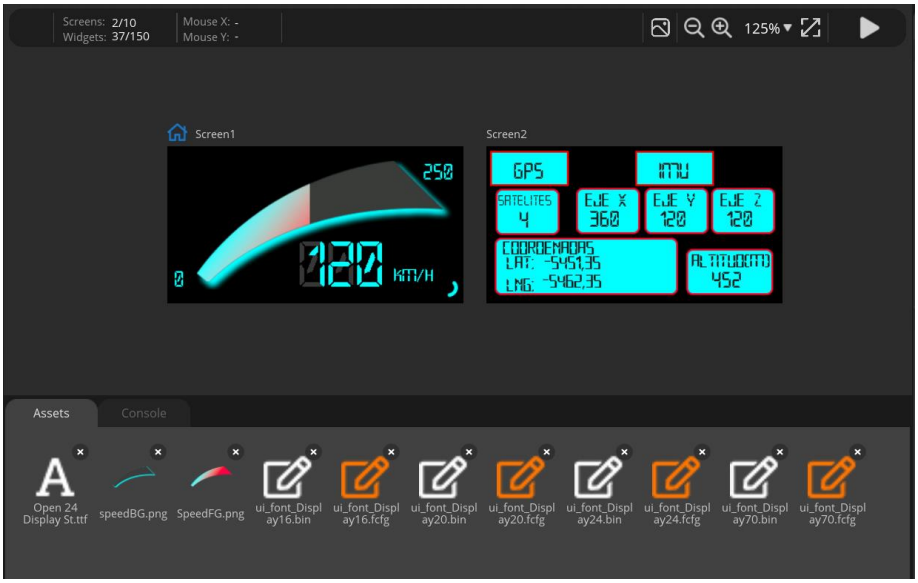


Ilustración 22. Explicación square line studio

Y en la derecha de la pantalla se pueden observar algunas de las muchas opciones de personalización que da a elegir el programa.

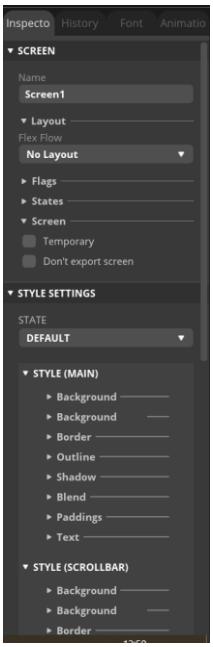


Ilustración 23 . Configurador de diseños square line studio 2

Como primer paso para realizar el diseño se configura la pantalla con las dimensiones que se desean y la profundidad del color de la pantalla, para que al exportar los archivos sean coherentes las configuraciones de la pantalla con las de la programación y de esta manera todo funcione correctamente. Una vez todo

configurado se trabaja en el diseño para que quede de la forma deseada, al finalizar este proceso se revisan los ajustes del proyecto en la pestaña “Proyect Setting” y se procede a exportar los archivos.

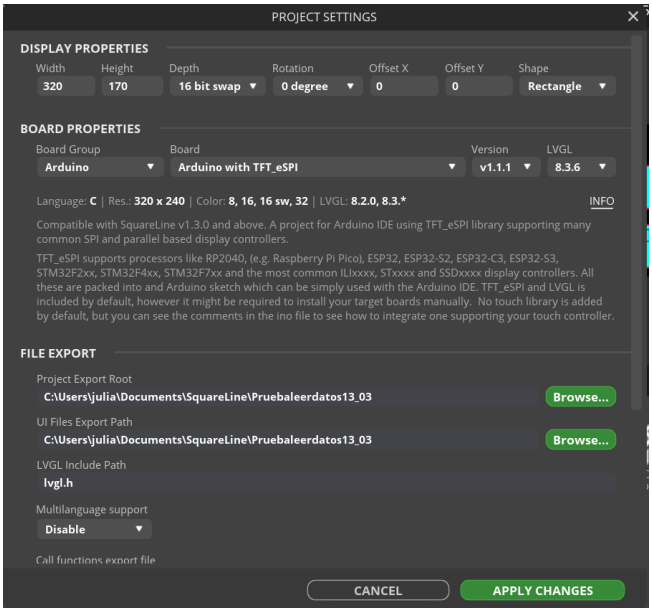


Ilustración 24.Configuración para exportar archivos square line studio

4.3.2. Arduino IDE

Una vez realizado la exportación de los archivos del Square Line Studio (se pueden observar en el Anexo A) se procede a acoplarlos al código principal que se ha creado en el IDE de Arduino. El cual se explica a continuación:

- Primero se incluyen las librerías necesarias para el funcionamiento del código:

```
#define LCD_MODULE_CMD_1
#define USE_UI

#include "OneButton.h" /* https://github.com/mathertel/OneButton.git */
#include "lvgl.h"      /* https://github.com/lvgl/lvgl.git */
#include <TFT_eSPI.h>
#include "Arduino.h"

#include "Wire.h"
#include "esp_lcd_panel_io.h"
#include "esp_lcd_panel_ops.h"
#include "esp_lcd_panel_vendor.h"
#include "ui.h"
```

```
#include "pin_config.h"
#include "esp_sntp.h"
#include <Adafruit_Sensor.h>
#include <Adafruit_BNO055.h>
#include <utility/imumaths.h>
```

- Se declaran distintas variables que serán necesarias para el correcto funcionamiento del programa, entre ellas están las variables que se encargarán de definir las lecturas de la IMU y el GPS.

```
int Screen=1;
double linearAccelDataX=0;
double linearAccelDataY=0;
double linearAccelDataZ=0;
double latitude;
int altitude;
//int speedAccEst;
double longitude;
double SIV;
int nedNorthVel;
int nedEastVel;
int nedDownVel;
uint16_t BNO055_SAMPLERATE_DELAY_MS = 100;
Adafruit_BNO055 bno = Adafruit_BNO055(55, 0x28, &Wire);
```

- Función que se encarga del botón para cambiar de una pantalla a la otra:

```
void IRAM_ATTR toggleLCD()
{
  if (Screen==1){
   lv_disp_load_scr(ui_Screen2);
   Screen=2;
  }else{
  lv_disp_load_scr(ui_Screen1);
  Screen=1;
  }
}
bool enabled=0;
OneButton button1(PIN_BUTTON_1, true);
OneButton button2(PIN_BUTTON_2, true);
```

- Para el correcto funcionamiento de la pantalla Lilygo T-Display es necesario añadir las siguientes variables y funciones:

```
esp_lcd_panel_io_handle_t io_handle = NULL;
static lv_disp_draw_buf_t disp_buf; // Contiene búferes gráficos internos
llamados búferes de dibujo
static lv_disp_drv_t disp_drv;      // Contiene funciones de retrollamada
static lv_color_t *lv_disp_buf;
static bool is_initialized_lvgl = false;
#if defined(LCD_MODULE_CMD_1)
typedef struct {
uint8_t cmd;
uint8_t data[14];
uint8_t len;
} lcd_cmd_t;

lcd_cmd_t lcd_st7789v[] = {
    {0x11, {0}, 0 | 0x80},
    {0x3A, {0X05}, 1},
    {0xB2, {0X0B, 0X0B, 0X00, 0X33, 0X33}, 5},
    {0xB7, {0X75}, 1},
    {0xBB, {0X28}, 1},
    {0xC0, {0X2C}, 1},
    {0xC2, {0X01}, 1},
    {0xC3, {0X1F}, 1},
    {0xC6, {0X13}, 1},
    {0xD0, {0XA7}, 1},
    {0xD0, {0XA4, 0XA1}, 2},
    {0xD6, {0XA1}, 1},
    {0xE0, {0XF0, 0X05, 0X0A, 0X06, 0X06, 0X03, 0X2B, 0X32, 0X43, 0X36,
0X11, 0X10, 0X2B, 0X32}, 14},
    {0xE1, {0XF0, 0X08, 0X0C, 0X0B, 0X09, 0X24, 0X2B, 0X22, 0X43, 0X38,
0X15, 0X16, 0X2F, 0X37}, 14},

};
#endif

static   bool   example_notify_lvgl_flush_ready(esp_lcd_panel_io_handle_t
panel_io, esp_lcd_panel_io_event_data_t *edata, void *user_ctx)
{
    if (is_initialized_lvgl) {
        lv_disp_drv_t *disp_driver = (lv_disp_drv_t *)user_ctx;
        lv_disp_flush_ready(disp_driver);
    }
    return false;
}

static  void  example_lvgl_flush_cb(lv_disp_drv_t  *drv,  const  lv_area_t
*area, lv_color_t *color_map)
{
```

```
    esp_lcd_panel_handle_t panel_handle = (esp_lcd_panel_handle_t)drv->user_data;
    int offsetx1 = area->x1;
    int offsetx2 = area->x2;
    int offsety1 = area->y1;
    int offsety2 = area->y2;
    // copy a buffer's content to a specific area of the display
    esp_lcd_panel_draw_bitmap(panel_handle, offsetx1, offsety1, offsetx2 + 1, offsety2 + 1, color_map);
}
```

- En la función setup() se prepara todo lo necesario antes de que el programa entre en el bucle loop(), donde se ejecutará el código principal repetidamente. En esta función setup(), se inician las configuraciones de la pantalla y se definen los pines a utilizar, una vez realizado esta parte del código dentro del mismo setup() se inicializan las funciones de los módulos GPS e IMU.

```
void setup()
{
    // (POWER ON)IO15 debe estar en HIGH antes de comenzar, de lo contrario, la pantalla no se mostrará cuando se use la batería.
    pinMode(PIN_POWER_ON, OUTPUT);
    digitalWrite(PIN_POWER_ON, HIGH);
    Serial.begin(115200);


    pinMode(PIN_LCD_RD, OUTPUT);
    digitalWrite(PIN_LCD_RD, HIGH);
    esp_lcd_i80_bus_handle_t i80_bus = NULL;
    esp_lcd_i80_bus_config_t bus_config = {
      .dc_gpio_num = PIN_LCD_DC,
      .wr_gpio_num = PIN_LCD_WR,
      .clk_src = LCD_CLK_SRC_PLL160M,
      .data_gpio_nums =
      {
          PIN_LCD_D0,
          PIN_LCD_D1,
          PIN_LCD_D2,
          PIN_LCD_D3,
          PIN_LCD_D4,
          PIN_LCD_D5,
          PIN_LCD_D6,
          PIN_LCD_D7,
      },
      .bus_width = 8,
```

```
        .max_transfer_bytes = LVGL_LCD_BUF_SIZE * sizeof(uint16_t),
        .psram_trans_align = 0,
        .sram_trans_align = 0
    };
    esp_lcd_new_i80_bus(&bus_config, &i80_bus);

    esp_lcd_panel_io_i80_config_t io_config = {
        .cs_gpio_num = PIN_LCD_CS,
        .pclk_hz = EXAMPLE_LCD_PIXEL_CLOCK_HZ,
        .trans_queue_depth = 20,
        .on_color_trans_done = example_notify_lvgl_flush_ready,
        .user_ctx = &disp_drv,
        .lcd_cmd_bits = 8,
        .lcd_param_bits = 8,
        .dc_levels =
        {
            .dc_idle_level = 0,
            .dc_cmd_level = 0,
            .dc_dummy_level = 0,
            .dc_data_level = 1,
        },
    };
          ESP_ERROR_CHECK(esp_lcd_new_panel_io_i80(i80_bus,    &io_config,
&io_handle));
    esp_lcd_panel_handle_t panel_handle = NULL;
    esp_lcd_panel_dev_config_t panel_config = {
        .reset_gpio_num = PIN_LCD_RES,
        .color_space = ESP_LCD_COLOR_SPACE_RGB,
        .bits_per_pixel = 16,
        .vendor_config = NULL
    };
    esp_lcd_new_panel_st7789(io_handle, &panel_config, &panel_handle);
    esp_lcd_panel_reset(panel_handle);
    esp_lcd_panel_init(panel_handle);

    esp_lcd_panel_invert_color(panel_handle, true);

    esp_lcd_panel_swap_xy(panel_handle, true);

    esp_lcd_panel_mirror(panel_handle, false, true);


    esp_lcd_panel_set_gap(panel_handle, 0, 35);
#if defined(LCD_MODULE_CMD_1)
     for (uint8_t i = 0; i < (sizeof(lcd_st7789v) / sizeof(lcd_cmd_t));
i++) {
```

```
                esp_lcd_panel_io_tx_param(io_handle,   lcd_st7789v[i].cmd,
lcd_st7789v[i].data, lcd_st7789v[i].len & 0x7f);
        if (lcd_st7789v[i].len & 0x80)
            delay(120);
    }
#endif
    /* Lighten the screen with gradient */
    ledcSetup(0, 10000, 8);
    ledcAttachPin(PIN_LCD_BL, 0);
    for (uint8_t i = 0; i < 0xFF; i++) {
        ledcWrite(0, i);
        delay(2);
    }

    lv_init();
      lv_disp_buf  =  (lv_color_t  *)heap_caps_malloc(LVGL_LCD_BUF_SIZE  *
sizeof(lv_color_t), MALLOC_CAP_DMA | MALLOC_CAP_INTERNAL);

            lv_disp_draw_buf_init(&disp_buf,      lv_disp_buf,      NULL,
LVGL_LCD_BUF_SIZE);
    /*Inicialización del display*/
    lv_disp_drv_init(&disp_drv);
    /*Ajuste de la resolución de la pantalla*/
    disp_drv.hor_res = EXAMPLE_LCD_H_RES;
    disp_drv.ver_res = EXAMPLE_LCD_V_RES;
    disp_drv.flush_cb = example_lvgl_flush_cb;
    disp_drv.draw_buf = &disp_buf;
    disp_drv.user_data = panel_handle;
    lv_disp_drv_register(&disp_drv);


    is_initialized_lvgl = true;
//Iniciación  de  la  los  archivos  exportados  del  programa  Square  Line
Studio.
    ui_init();
//Función que coloca a 0 de los valores de los sensores en la pantalla
    ui_reset();

attachInterrupt(PIN_BUTTON_2, toggleLCD, RISING);
  //GPSINIT
  GPSinit();
  //BNOINIT
  BNOinit();

}
```

- Función que coloca a 0 de los valores de los sensores en la pantalla:

```
void ui_reset() {
  lv_slider_set_value(ui_Slider1, 0, LV_ANIM_OFF);
  lv_label_set_text(ui_Labelfront, "0");
  lv_label_set_text(ui_Satelitesnumero, "0");
  lv_label_set_text(ui_Aceleracionxd, "0");
  lv_label_set_text(ui_aceleracionyd, "0");
  lv_label_set_text(ui_Aceleracionzd, "0");
  lv_label_set_text(ui_Gradoslatitudn, "0");
  lv_label_set_text(ui_Gradoslongitudn, "0");
  lv_label_set_text(ui_Altitudt, "0");
}
```

- A continuación se entra en la función principal del programa que es el loop(), bucle que se ejecutará repetidamente mientras el programa esté corriendo en el dispositivo. En este bucle hay varias funciones que se encargan de obtener los valores de los sensores (GPS,IMU) y las dos siguientes de actualizar estos valores en la pantalla (updateBNO(), updateGPS()).

```
void loop()
{
    lv_timer_handler();

    delay(10);

    if (millis()>2000 && enabled==0){

     enabled=1;
    }
  //GPSINIT
      //Consulta el módulo solo cada segundo, hacerlo más frecuentemente
solo causará tráfico en el bus I2C.
  //El módulo solo responde cuando hay una nueva posición disponible
  if (millis() - lastTime > 1000)
  {
    lastTime = millis(); //Update the timer

    latitude = myGNSS.getLatitude();
    Serial.print(F("Lat: "));
    Serial.print(latitude);

    longitude = myGNSS.getLongitude();
    Serial.print(F(" Long: "));
    Serial.print(longitude);
    Serial.print(F(" (degrees * 10^-7)"));
```

```
    altitude = myGNSS.getAltitude();
    Serial.print(F(" Alt: "));
    Serial.print(altitude);
    Serial.print(F(" (mm)"));

    SIV = myGNSS.getSIV();
    Serial.print(F(" SIV: "));
    Serial.print(SIV);

    nedNorthVel = myGNSS.getNedNorthVel();
    Serial.print(F(" VelN: "));
    Serial.print(nedNorthVel);
    Serial.print(F(" (mm/s)"));

    nedEastVel = myGNSS.getNedEastVel();
    Serial.print(F(" VelE: "));
    Serial.print(nedEastVel);
    Serial.print(F(" (mm/s)"));

    nedDownVel = myGNSS.getNedDownVel();
    Serial.print(F(" VelD: "));
    Serial.print(nedDownVel);
    Serial.print(F(" (mm/s)"));

    //Este comando es para obtener también la velocidad de una forma más
aproximada y menos exacta que el que se usa en el programa pero se deja
en el código por si se requiere en algún momento.
    //speedAccEst = myGNSS.getSpeedAccEst();
    //Serial.print(F(" SpeedAccEst: "));
    //Serial.print(speedAccEst);
    //Serial.print(F(" (mm/s)"));

    Serial.println();


  //BNOINIT
    //Se pueden añadir los vectores que se quieran dentro de las posibles
mediciones, como por ejemplo el de gravedad.
   sensors_event_t  orientationData  ,  angVelocityData  ,  linearAccelData,
magnetometerData, accelerometerData, gravityData;
  bno.getEvent(&orientationData, Adafruit_BNO055::VECTOR_EULER);
  bno.getEvent(&angVelocityData, Adafruit_BNO055::VECTOR_GYROSCOPE);
  bno.getEvent(&linearAccelData, Adafruit_BNO055::VECTOR_LINEARACCEL);
  bno.getEvent(&magnetometerData, Adafruit_BNO055::VECTOR_MAGNETOMETER);
                                        bno.getEvent(&accelerometerData,
Adafruit_BNO055::VECTOR_ACCELEROMETER);
  bno.getEvent(&gravityData, Adafruit_BNO055::VECTOR_GRAVITY);
```

```
  printEvent(&orientationData);
  printEvent(&angVelocityData);
  printEvent(&linearAccelData);
  printEvent(&magnetometerData);
  printEvent(&accelerometerData);
  printEvent(&gravityData);

  int8_t boardTemp = bno.getTemp();
  Serial.println();
  Serial.print(F("temperature: "));
  Serial.println(boardTemp);

  uint8_t system, gyro, accel, mag = 0;
  bno.getCalibration(&system, &gyro, &accel, &mag);
  Serial.println();
  Serial.print("Calibration: Sys=");
  Serial.print(system);
  Serial.print(" Gyro=");
  Serial.print(gyro);
  Serial.print(" Accel=");
  Serial.print(accel);
  Serial.print(" Mag=");
  Serial.println(mag);

  Serial.println("--");
  delay(BNO055_SAMPLERATE_DELAY_MS);


  }

  updateBNO();
  updateGPS();
  delay(10);
}
```

- Las siguientes funciones son llamadas en el setup() del programa pero las incluyo al final del código para que esté mas limpio y claro el programa. Estas se encargan de la comunicación entre los módulos GPS e IMU mediante I2C, seleccionando los puertos (1,2) los cuales están destinados para esta comunicación en el ESP32-S3:

```
void BNOinit(){
```

```
  Serial.println("Orientation Sensor Test"); Serial.println("");
  Wire.begin(1,2);
  /* Inicializa el sensor */
  if (!bno.begin())
  {
    /* Hay un problema conectando BNO055… revisa tus conexiones*/
    Serial.print("Ooops, no BNO055 detected ... Check your wiring or I2C
ADDR!");
    while (1);
  }

  delay(1000);
}


void GPSinit()
{
  Serial.println("SparkFun u-blox Example");

  Wire.begin(1,2);

  if (myGNSS.begin() == false) //Conecta al módulo u-blox utilizando el
puerto Wire.
  {
    Serial.println(F("u-blox GNSS not detected at default I2C address.
Please check wiring. Freezing."));
    while (1);
  }

  myGNSS.setI2COutput(COM_TYPE_UBX); //Configura el puerto I2C para que
solo emita UBX(apaga el ruido NMEA)
  myGNSS.setNavigationFrequency(2); //Produce dos soluciones por segundo
  myGNSS.setAutoPVT(true); //Indica al GPS que envíe cada solución
  myGNSS.saveConfiguration(); //Guarda la configuración que se ejecutará
en el GPS
}
```

- La siguiente función se encarga de obtener los valores deseados de la IMU y representarlos mediante el monitor serial. En este caso solo se representan mediante pantalla los valores de la aceleración en los ejes X, Y y Z.

```
void printEvent(sensors_event_t* event) {
```

```
  double x = -1000000, y = -1000000 , z = -1000000; //dumb values, easy
to spot problem
  if (event->type == SENSOR_TYPE_ACCELEROMETER) {
    Serial.print("Accl:");
    x = event->acceleration.x;
    linearAccelDataX=event->acceleration.x;
    y = event->acceleration.y;
    linearAccelDataY=event->acceleration.y;
    z = event->acceleration.z;
    linearAccelDataZ=event->acceleration.z;
  }
  else if (event->type == SENSOR_TYPE_ORIENTATION) {
    Serial.print("Orient:");
    x = event->orientation.x;
    y = event->orientation.y;
    z = event->orientation.z;
  }
  else if (event->type == SENSOR_TYPE_MAGNETIC_FIELD) {
    Serial.print("Mag:");
    x = event->magnetic.x;
    y = event->magnetic.y;
    z = event->magnetic.z;
  }
  else if (event->type == SENSOR_TYPE_GYROSCOPE) {
    Serial.print("Gyro:");
    x = event->gyro.x;
    y = event->gyro.y;
    z = event->gyro.z;
  }
  else if (event->type == SENSOR_TYPE_ROTATION_VECTOR) {
    Serial.print("Rot:");
    x = event->gyro.x;
    y = event->gyro.y;
    z = event->gyro.z;
  }
  else if (event->type == SENSOR_TYPE_LINEAR_ACCELERATION) {
    Serial.print("Linear:");
    x = event->acceleration.x;
    y = event->acceleration.y;
    z = event->acceleration.z;
  }
  else if (event->type == SENSOR_TYPE_GRAVITY) {
    Serial.print("Gravity:");
    x = event->acceleration.x;
    y = event->acceleration.y;
    z = event->acceleration.z;
  }
```

```
  else {
    Serial.print("Unk:");
  }

  Serial.print("\tx= ");
  Serial.print(x);
  Serial.print(" |\ty= ");
  Serial.print(y);
  Serial.print(" |\tz= ");
  Serial.println(z);
}
```

- Actualiza los valores de la IMU en la pantalla:

```
void updateBNO() {

  lv_label_set_text(ui_Aceleracionxd, String(linearAccelDataX).c_str());

  lv_label_set_text(ui_aceleracionyd, String(linearAccelDataY).c_str());

  lv_label_set_text(ui_Aceleracionzd, String(linearAccelDataZ).c_str());
}
```

- Actualiza los valores del GPS por la pantalla, como la velocidad el GPS la entrega en mm/s, primero se obtiene el vector velocidad de las tres componentes realizando el módulo de ellas y luego pasando ese vector a km/h.

```
void updateGPS(){
    int velocidad = sqrt(sq(nedNorthVel) + sq(nedEastVel) +
sq(nedDownVel));
  int velocidadkmh = velocidad*0.0036;
  lv_label_set_text(ui_Labelfront, String(velocidadkmh).c_str());
  lv_slider_set_value(ui_Slider1, int(velocidadkmh), LV_ANIM_ON);

  lv_label_set_text(ui_Satelitesnumero, String(SIV).c_str());

  lv_label_set_text(ui_Gradoslatitudn, String(latitude).c_str());

  lv_label_set_text(ui_Gradoslongitudn, String(longitude).c_str());
  int Altitude1=altitude/100;
  lv_label_set_text(ui_Altitudt, String(Altitude1).c_str());
}
```

CAPÍTULO 5: MONTAJE Y PRUEBAS REALIZADAS AL DISPOSITIVO EN FUNCIONAMIENTO.

5.1. Montaje del dispositivo

En las siguientes ilustraciones se muestran las distintas partes del dispositivo final:

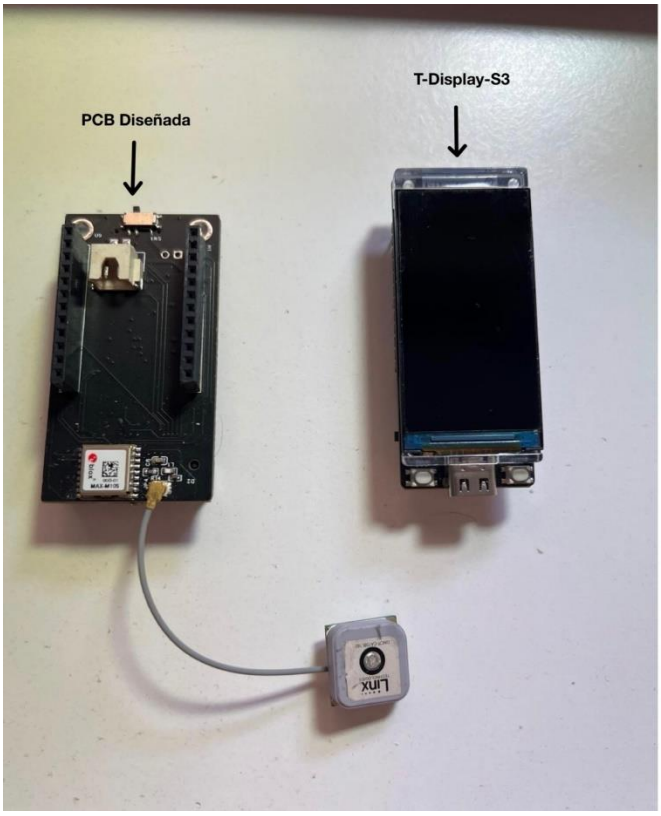


Ilustración 25. Parte delantera dispositivo separado

En la ilustración 25 se puede observar a la izquierda la placa PCB diseñada por el proyectista, en esta parte se encuentran el módulo GPS y la antena, además del puerto donde se incorporará la batería en un futuro. Y a la derecha la pantalla Lilygo T-Display-S3.

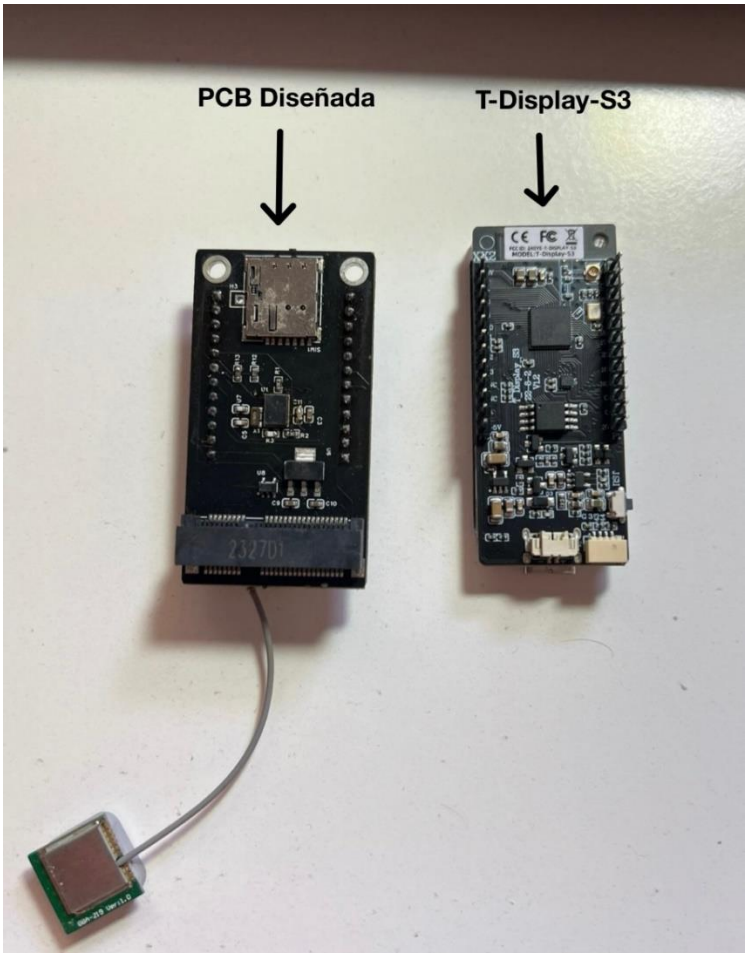


Ilustración 26. Parte trasera de ambas placas

En esta imagen se puede apreciar la parte trasera de ambas placas, en la PCB izquierda se pueden observar el módulo SIMCARD y el módulo IMU situado en el centro de la placa.

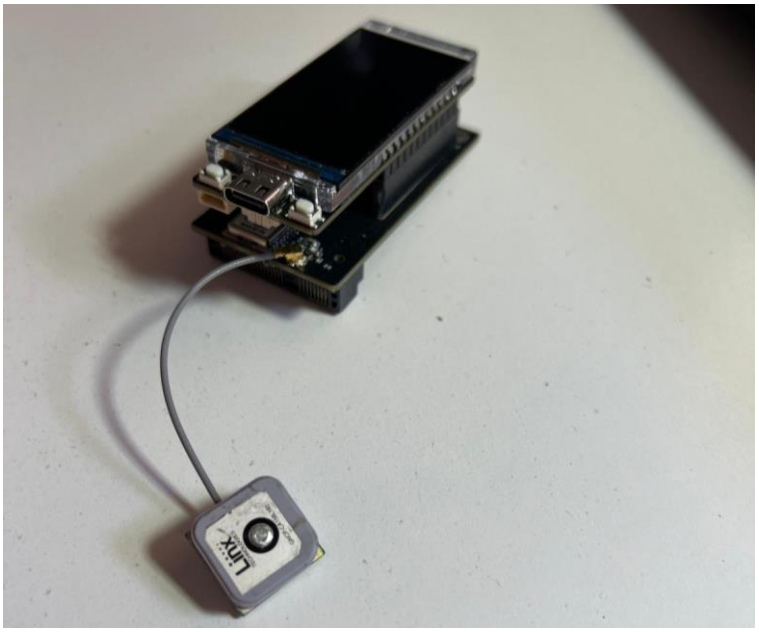


Ilustración 27.Unión de ambas placas

En la ilustración anterior se observa el dispositivo ya uniendo ambas placas a la perfección.

5.2. Pruebas realizadas al dispositivo en funcionamiento

Primero se realizó una prueba estática dentro de casa para comprobar el correcto funcionamiento de la pantalla y sus módulos, se sabe que dentro de un edificio el módulo GPS no es capaz de conectarse prácticamente a ningún satélite por lo que en ese primer momento no ofrece datos a la pantalla.

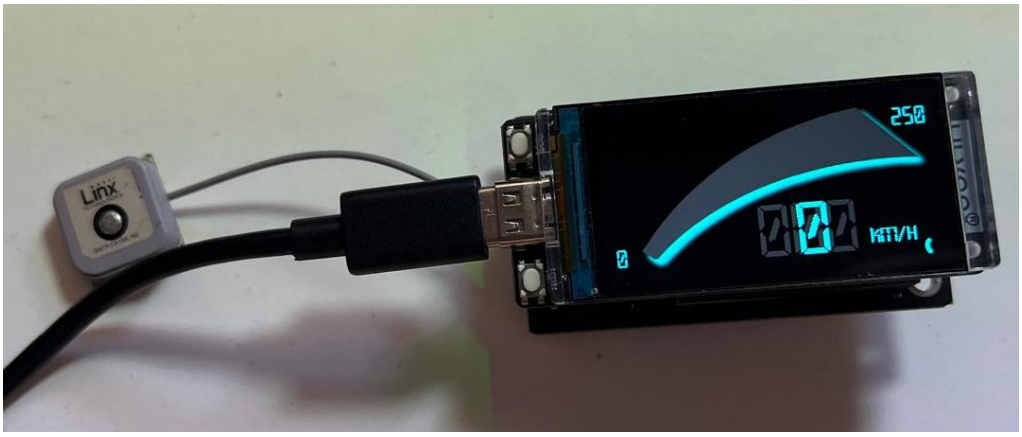


Ilustración 28. Pantalla principal en prueba estática en casa.

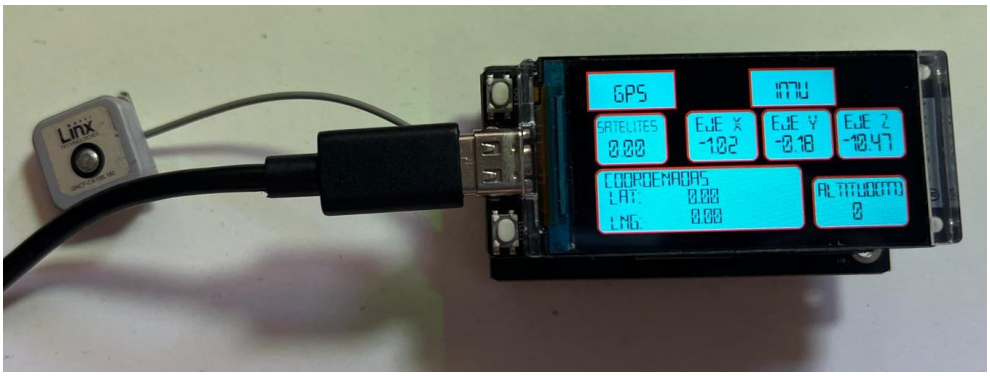


Ilustración 29. Segunda pantalla del dispositivo en prueba estática en casa.

Para poder visualizar esta segunda pantalla hay que accionar el botón inferior que se encuentra en la pantalla Lilygo T-Display-S3. En esta interface se pueden visualizar en tiempo real algunos de los datos que ofrece en ese instante de tiempo la IMU, y el GPS.

Para comprobar el correcto funcionamiento del dispositivo en movimiento se realizaron diferentes pruebas de funcionamiento subiendo el dispositivo a un coche y comprobando que efectivamente el GPS es capaz de conectarse a los satélites y dar la posición y la velocidad con una precisión bastante efectiva. En las siguientes imágenes que se adjuntan se puede observar su eficacia:



Ilustración 30. Prueba en movimiento del dispositivo(Pantalla principal)

En esta pantalla se puede observar la velocidad en km/h a la que se desplaza el vehículo y una animación que va rellenando la barra a la vez que la velocidad aumenta.



Ilustración 31. Prueba en funcionamiento (Pantalla secundaria)

Esta pantalla está dedicada para observar diferentes valores obtenidos por los módulos de adquisición de datos:

- Desde la IMU se proporcionan los valores de las aceleraciones que sufre el vehículo dependiendo de la inclinación y dirección de hacia donde se dirija el movimiento del vehículo.
- Desde el GPS se pueden observar la cantidad de satélites a la que está conectado (cuantos más satélites mayor precisión se obtendrá en los datos reflejados por pantalla), las coordenadas de latitud y longitud para obtener con precisión la ubicación exacta del dispositivo, y por último la altitud a la que se encuentra.

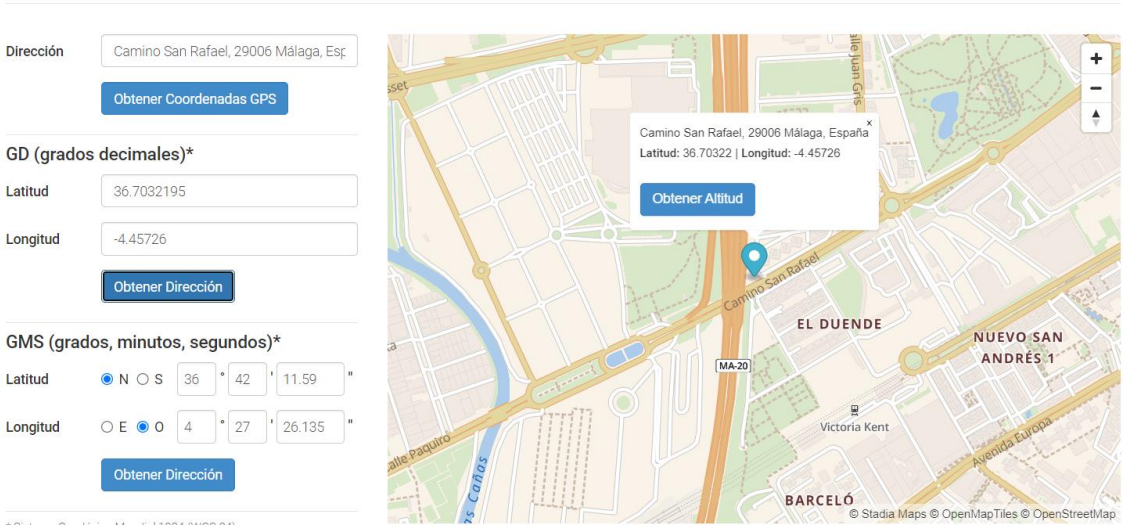


Ilustración 32. Verificación GPS

En la ilustración anterior se puede comprobar con las coordenadas que proporciona la pantalla son las que efectivamente estábamos situados en ese justo momento.

Entre las pruebas que se realizaron, se cotejaron los datos obtenidos por el monitor serial del Arduino IDE de latitud y longitud, y se elaboró una trayectoria mediante un programa en Matlab para comprobar que el dispositivo funcionaba correctamente. Se recogieron los datos de hora, latitud, longitud y número de satélites a los que estaba el dispositivo conectado, y se elaboró la siguiente tabla:

HORA	LAT	LONG	SIV
22:27:10.641	367378578.00	-46053442.00	11
22:27:10.900	367378635.00	-46053429.00	10
22:27:10.900	367378651.00	-46053423.00	11
22:27:10.900	367378665.00	-46053403.00	10
22:27:10.900	367378711.00	-46053378.00	10
22:27:11.922	367378639.00	-46053295.00	10
22:27:12.984	367378457.00	-46053191.00	11
22:27:13.996	367378233.00	-46053125.00	11

22:27:15.004	367378018.00	-46053112.00	11
22:27:16.004	367377809.00	-46053107.00	11
22:27:17.012	367377553.00	-46053104.00	11
22:27:18.008	367377244.00	-46053106.00	11
22:27:19.033	367376914.00	-46053110.00	10
22:27:20.034	367376530.00	-46053103.00	11
22:27:21.072	367376083.00	-46053093.00	11

Tabla 4. Valores recogidos en prueba en funcionamiento

Se siguieron recogiendo valores durante tres minutos y con el siguiente código en el software ***Matlab*** se consigue representar la trayectoria que siguió el vehículo:

```
->load('Trayectoria.mat')
->LAT=Trayectoria(:,2)/10^7;
->LON=Trayectoria(:,3)/10^7;
->name = 'openstreetmap';
->url = 'https://a.tile.openstreetmap.org/${z}/${x}/${y}.png';
->copyright = char(uint8(169));
->attribution = copyright + "OpenStreetMap contributors";
->addCustomBasemap(name,url,'Attribution',attribution)
->zoomLevel = 16;
->player = geoplayer(LAT(100),LON(100),zoomLevel);
->player.Basemap = 'openstreetmap';
->plotRoute(player,LAT(:),LON(:));
->fig = player.Parent;
->exportgraphics(fig,'Trayectoria.png','Resolution',600)
->for i = 1:length(LAT(:))
  ->plotPosition(player,LAT(i),LON(i))
->end
```

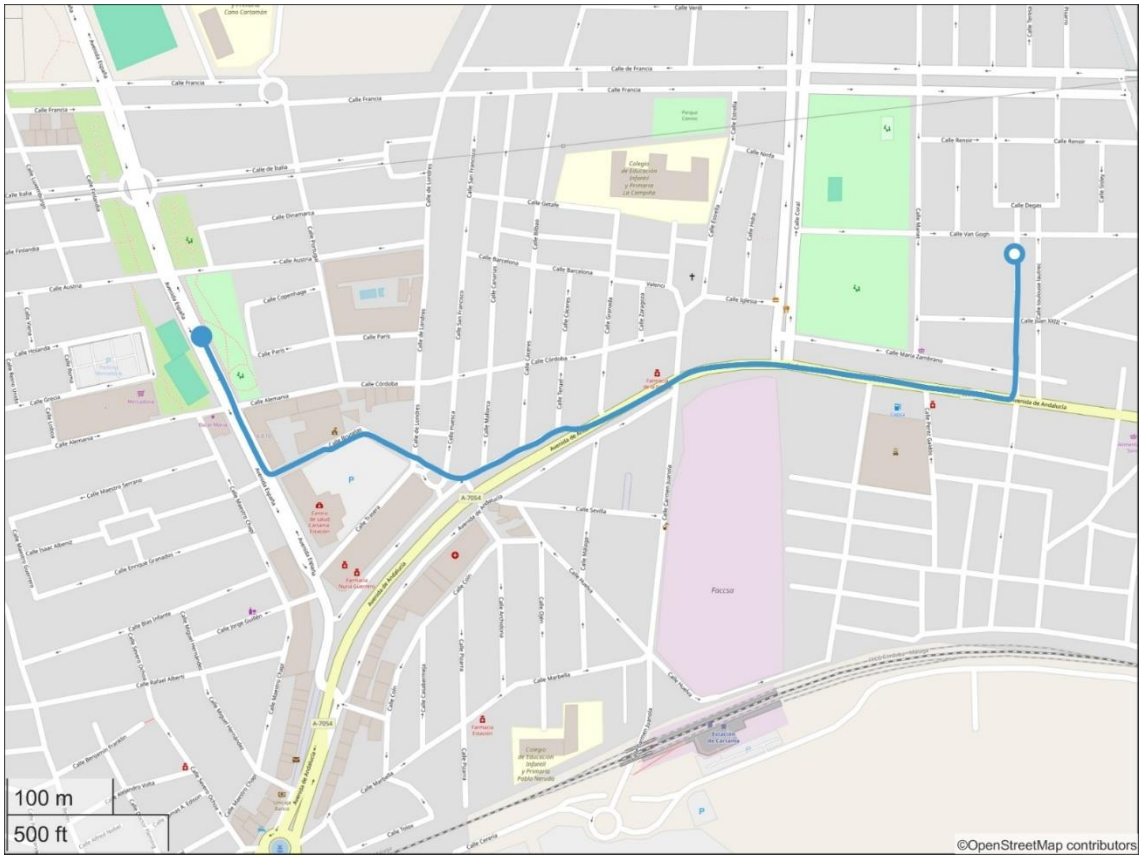


Ilustración 33. Prueba de trayecto realizado

CAPÍTULO 6: CONCLUSIONES Y FUTURO DEL PROYECTO.

6.1. Conclusiones

Respecto al objetivo del presente proyecto, el cual era diseñar y fabricar un dispositivo de telemetría capaz de obtener la precisión y la frecuencia suficiente como para unos resultados semiprofesionales, capaces de ser entendidos por cualquier usuario amateur sin necesidad de tener grandes conocimientos acerca de la telemetría.

En cuanto al diseño del dispositivo las principales objeciones se encontraron a la hora del primer prototipo, ya que era una placa con demasiados sensores muy agrupados entre sí. Por lo que, el enrutamiento fue complicado de realizar, aunque gracias a las 4 capas de cobre se pudo solventar ese impedimento utilizando cada espacio disponible para pasar rutas de cableados entre los diferentes módulos.

Gracias al segundo prototipo y la reducción de los sensores, se consiguieron reducir las medidas para la adaptación de esta placa a la pantalla T-Display del distribuidor Lilygo. En la siguiente fase del proyecto, que fue la de adaptación de la placa PCB diseñada a la pantalla, fue donde más impedimentos se encontraron, ya que la poca información acerca de la programación de la pantalla hizo muy complicado poder encontrar la manera más efectiva de realizar esta tarea. Mediante el programa Square Line Studio y la librería LVGL se consiguió que la programación fuera solventada de manera exitosa, con los ajustes necesarios para ella, se configuró un entorno muy intuitivo y llamativo para el usuario del dispositivo.

Respecto a la funcionalidad del dispositivo, se ha conseguido obtener una frecuencia de muestreo muy elevada, capaz de proporcionar valores en tiempo real muy exactos. La parte de telemetría depende de otra parte del proyecto SynRide, por lo que en este proyecto se ha limitado a comprobar que se pueden enviar los datos a la nube exclusivamente. Para la visualización de estos valores, dentro del proyecto SynRide se está desarrollando una página web la cual estará destinada a la visualización de los datos que obtiene el dispositivo. Los datos serán enviados mediante una tarjeta SIM a través de internet, con la visualización de estos datos se podrá completar el último objetivo de este proyecto, que es el análisis de los datos para la mejora de la conducción del usuario del dispositivo.

6.2. Futuro del proyecto

Para la siguiente fase del proyecto lo que se espera es que estos datos que se consiguen mostrar en tiempo real, sean enviados gracias a la preparación que se ha realizado en este proyecto a través del módulo SIMCARD incluido en la PCB. De esta

manera, se podrán visualizar en una página web que sigue en construcción, la cual adjunto el enlace(https://speedsense.es:61052/#featurettes), además de varias capturas de lo que será posible visualizar en ella en un futuro.

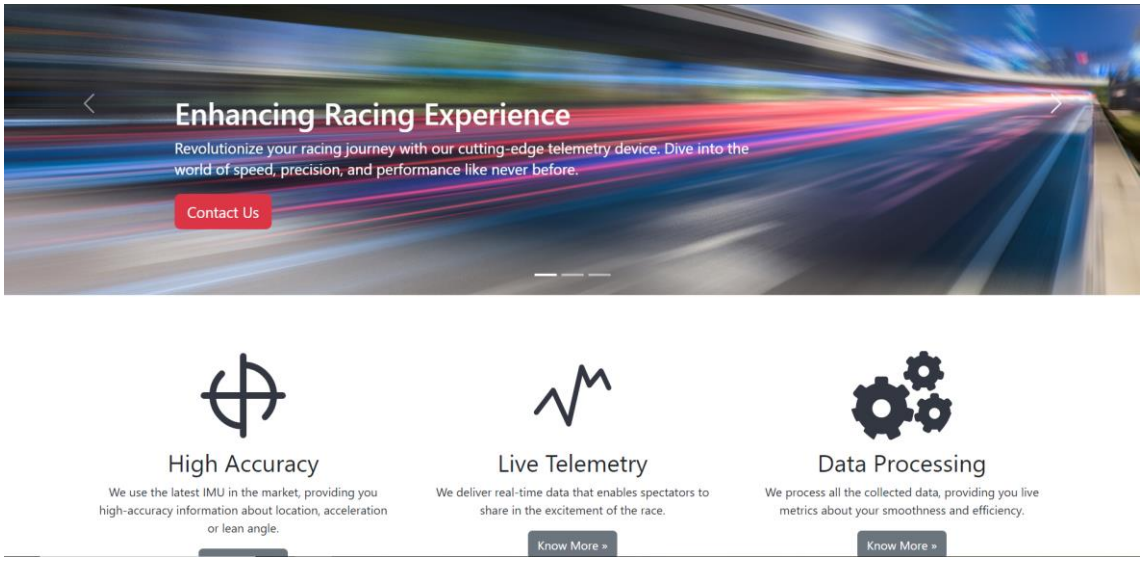


Ilustración 34. Captura página principal de SpeedSense.es

Dentro de esta página se podrá acceder a los parámetros que se han enviado a través del dispositivo y visualizarlos en su contexto para así poder cumplir con el objetivo de obtener esa telemetría la cual ayudará al usuario en mejorar sus habilidades en la conducción.



Ilustración 35.Simulación de lo que se verá con los datos enviados

En la ilustración 31 se puede observar como se verán los datos que se hayan enviado en la página web. Como son la velocidad, las aceleraciones que sufre el vehículo

debido a los cambios de dirección, ángulos de inclinación y demás datos como el tiempo por vuelta que se ha realizado en el circuito y la diferencia con la vuelta anterior.

BIBLIOGRAFÍA

[1] Sittrack. (22/05/2024). [Online]. Telemetría y sus aplicaciones. Recuperado de https://landing.sitrack.com/telemtria-y-sus-aplicaciones.

[2] Wikipedia. (22/05/2024). [Online]. Telemetría. En Wikipedia, La enciclopedia libre. Recuperado de https://es.wikipedia.org/wiki/Telemetr%C3%ADa

[3] Geotab. (22/05/2024). [Online]. ¿Qué es GPS? Recuperado de https://geotab.com/es/blog/que-es-gps/

[4] M5Stack. (22/05/2024). [Online]. Stamp CAT-M. Recuperado de https://docs.m5stack.com/en/stamp/stamp_catm

[5] Llamas,L. (22/05/2024). [Online]. ESP32-S3: Detalles hardware y pinout. Recuperado de https://www.luisllamas.es/esp-32-s3-detalles-hardware-pinout/

[6] Moto1Pro. (22/05/2024). [Online]. Convierte tu moto en una SBK: Yamaha R1 e-test. Recuperado de https://www.moto1pro.com/convierte-tu-moto-en-una-sbk-yamaha-r1e-etesta

[7] Industria Embebida Hoy. (22/05/2024). [Online]. Placa de desarrollo T-Display-S3 con pantalla táctil. Recuperado de https://www.industriaembebidahoy.com/placa-de-desarrollo-t-display-s3-con-pantalla-tactil/

[8] LVGL. (22/05/2024). [Online]. Recuperado de https://lvgl.io/

[9] Diario electrónico Hoy. (22/05/2024). [Online]. Introducción al I2C bus. Recuperado de https://www.diarioelectronicohoy.co/blog/introduccion-al-i2c-bus

[10] Wikipedia. (22/05/2024). I2C. [Online]. En Wikipedia, la enciclopedia libre. Recuperado de https://es.wikipedia.org/wiki/I%C2%B2C

[11] Rincón Ingenieril. (22/05/2024). [Online]. ¿Cómo funciona el puerto serie o la UART? Recuperado de https://www.rinconingenieril.es/funciona-puerto-serir-la-uart/

[12] Ruiz, J. (2017, noviembre 27). [Online]. Tipos de sensores en un vehículo. Nitro.pe. Recuperado de: https://www.nitro.pe/mecanico-nitro/tipos-de-sensores-en-un-vehiculo.html

[13] Ingeniería y Mecánica Automotriz. [Online]. (15/05/2024). Sensores Automotrices. Ingeniería y Mecánica automotriz. Recuperado de: https://www.ingenieriaymecanicaautomotriz.com/category/mecanica/sensores-automotrices/

[14] Maroto, J.G. (2012, septiembre 26). [Online]. La centralita única ya es una realidad. Motociclismo.es. Recuperado de: https://www.motociclismo.es/mundial-motogp/la-centralita-unica-ya-es-una-realidad_174149_102.html

[15] Domínguez, C. (2013, septiembre 28). [Online]. ¿Cuánto tumban una Supersport, una SBK y una MotoGP? Motorpasión Recuperado de: Moto. https://www.motorpasionmoto.com/motogp/cuanto-tumban-una-supersport-una-sbk-y-una-motogp

[16] Morrillu. (2008, mayo 2). [Online]. Jorge Lorenzo sufre una caída en el G.P. de China. Motorpasión Moto. Recuperado de: https://www.motorpasionmoto.com/motogp/jorge-lorenzo-sufre-una-caida-en-el-gp-de-china

[17] MAX-M10S_IntegrationManual_UBX-20053088. UBLOX. C1-Público. R04, (2023, junio 1)

[18] bst_bno055_ds000-3247202. BOSCH. C1-Público. R1.8, (2021, Octubre)

[19] LILYGO. (2024, mayo 5). T-Display-S3 [DE]. [Online]. LILYGO. Recuperado de: https://www.lilygo.cc/products/t-display-s3-de?_pos=7&_sid=7e8eb575e&_ss=r

DOCUMENTO II: ANEXOS

ÍNDICE

1. Configuración de pines de la pantalla T-Display (pin_config.h)..... 67

2. Declaración de variables y eventos que se producen en la pantalla exportados de Square Line Studio (ui.c, ui.h, ui_events.h, ui_helpers.c y ui_helpers.h) 68

3. Código que se encarga de los eventos y configuraciones, imágenes y tipos de letras de las pantallas 1 y 2. (ui_Screen1.c, ui_Screen2.c, ui_img_speedbg_png.c, ui_img_speedfg_png.c, ui_font_Display16.c, ui_font_Display20.c, ui_font_Display24.c, ui_font_Dislay70.c) 81

ANEXO A: CÓDIGOS COMPLEMENTARIOS AL CÓDIGO PRINCIPAL

1. CONFIGURACIÓN DE PINES DE LA PANTALLA T-DISPLAY (PIN_CONFIG.H)

```
#pragma once



/* LCD CONFIG */
// Too low or too high pixel clock may cause screen mosaic
#define EXAMPLE_LCD_PIXEL_CLOCK_HZ    (16 * 1000 * 1000)
// The pixel number in horizontal and vertical
#define EXAMPLE_LCD_H_RES              320
#define EXAMPLE_LCD_V_RES              170
#define    LVGL_LCD_BUF_SIZE                          (EXAMPLE_LCD_H_RES    *
EXAMPLE_LCD_V_RES)
#define EXAMPLE_PSRAM_DATA_ALIGNMENT 64

/*ESP32S3*/
#define PIN_LCD_BL                  38

#define PIN_LCD_D0                  39
#define PIN_LCD_D1                  40
#define PIN_LCD_D2                  41
#define PIN_LCD_D3                  42
#define PIN_LCD_D4                  45
#define PIN_LCD_D5                  46
#define PIN_LCD_D6                  47
#define PIN_LCD_D7                  48

#define PIN_POWER_ON                15

#define PIN_LCD_RES                 5
#define PIN_LCD_CS                  6
#define PIN_LCD_DC                  7
#define PIN_LCD_WR                  8
#define PIN_LCD_RD                  9

#define PIN_BUTTON_1                0
#define PIN_BUTTON_2                14
#define PIN_BAT_VOLT                4

#define PIN_IIC_SCL                 17
#define PIN_IIC_SDA                 18
```

```
#define PIN_TOUCH_INT           16
#define PIN_TOUCH_RES           21

/* External expansion */
#define PIN_SD_CMD              13
#define PIN_SD_CLK              11
#define PIN_SD_D0               12
```

2. DECLARACIÓN DE VARIABLES Y EVENTOS QUE SE PRODUCEN EN LA PANTALLA EXPORTADOS DE SQUARE LINE STUDIO (UI.C, UI.H, UI_EVENTS.H, UI_HELPERS.C Y UI_HELPERS.H)

- **ui.c:**

```
// This file was generated by SquareLine Studio
// SquareLine Studio version: SquareLine Studio 1.3.4
// LVGL version: 8.3.6

#include "ui.h"
#include "ui_helpers.h"

////////////////////// VARIABLES //////////////////////


// SCREEN: ui_Screen1
void ui_Screen1_screen_init(void);
void ui_event_Screen1(lv_event_t * e);
lv_obj_t * ui_Screen1;
lv_obj_t * ui_Panel1;
lv_obj_t * ui_WaitingGPS;
lv_obj_t * ui_SpeedPanel;
lv_obj_t * ui_Image1;
lv_obj_t * ui_Slider1;
lv_obj_t * ui_SpeedLabelMin;
lv_obj_t * ui_SpeedLabelMax;
lv_obj_t * ui_SpeedTextPanel;
lv_obj_t * ui_Units;
lv_obj_t * ui_Labelback;
lv_obj_t * ui_Labelfront;


// SCREEN: ui_Screen2
void ui_Screen2_screen_init(void);
void ui_event_Screen2(lv_event_t * e);
lv_obj_t * ui_Screen2;
lv_obj_t * ui_Panel2;
lv_obj_t * ui_TopPanel1;
```

```
lv_obj_t * ui_Titulo1;
lv_obj_t * ui_TopPanel2;
lv_obj_t * ui_Titulo2;
lv_obj_t * ui_Panel3;
lv_obj_t * ui_Satelitespanel;
lv_obj_t * ui_Satelitestittle;
lv_obj_t * ui_Satelitesnumero;
lv_obj_t * ui_Aceleracionx;
lv_obj_t * ui_Aceleracionxtitulo;
lv_obj_t * ui_Aceleracionxd;
lv_obj_t * ui_Grados1;
lv_obj_t * ui_Coordenadastitulo;
lv_obj_t * ui_Gradosnumero1;
lv_obj_t * ui_Gradoslatitudn;
lv_obj_t * ui_Gradoslongitudn;
lv_obj_t * ui_Aceleraciony;
lv_obj_t * ui_Aceleracionytitulo;
lv_obj_t * ui_aceleracionyd;
lv_obj_t * ui_Altitud;
lv_obj_t * ui_Altitudtitulo;
lv_obj_t * ui_Altitudt;
lv_obj_t * ui_Aceleracionz;
lv_obj_t * ui_Aceleracionztitulo;
lv_obj_t * ui_Aceleracionzd;
lv_obj_t * ui____initial_actions0;

///////////////////// TEST LVGL SETTINGS ////////////////////
#if LV_COLOR_DEPTH != 16
    #error "LV_COLOR_DEPTH should be 16bit to match SquareLine Studio's
settings"
#endif
#if LV_COLOR_16_SWAP !=1
    #error "LV_COLOR_16_SWAP should be 1 to match SquareLine Studio's
settings"
#endif

///////////////////// ANIMATIONS ////////////////////

///////////////////// FUNCTIONS ////////////////////
void ui_event_Screen1(lv_event_t * e)
{
    lv_event_code_t event_code = lv_event_get_code(e);
    lv_obj_t * target = lv_event_get_target(e);
    if(event_code == LV_EVENT_CLICKED) {
        _ui_screen_change(&ui_Screen2, LV_SCR_LOAD_ANIM_FADE_ON, 500, 0,
&ui_Screen2_screen_init);
    }
```

```
}
void ui_event_Screen2(lv_event_t * e)
{
    lv_event_code_t event_code = lv_event_get_code(e);
    lv_obj_t * target = lv_event_get_target(e);
    if(event_code == LV_EVENT_CLICKED) {
         _ui_screen_change(&ui_Screen1, LV_SCR_LOAD_ANIM_FADE_ON, 500, 0,
&ui_Screen1_screen_init);
    }
}

////////////////////// SCREENS //////////////////////

void ui_init(void)
{
    lv_disp_t * dispp = lv_disp_get_default();
          lv_theme_t     *     theme     =     lv_theme_default_init(dispp,
lv_palette_main(LV_PALETTE_BLUE), lv_palette_main(LV_PALETTE_RED),
                                              false, LV_FONT_DEFAULT);
    lv_disp_set_theme(dispp, theme);
    ui_Screen1_screen_init();
    ui_Screen2_screen_init();
    ui____initial_actions0 = lv_obj_create(NULL);
    lv_disp_load_scr(ui_Screen1);
}
```

- **ui.h:**

```
// This file was generated by SquareLine Studio
// SquareLine Studio version: SquareLine Studio 1.3.4
// LVGL version: 8.3.6

#ifndef _PRUEBA1_02_UI_H
#define _PRUEBA1_02_UI_H

#ifdef __cplusplus
extern "C" {
#endif

#include "lvgl.h"

#include "ui_helpers.h"
#include "ui_events.h"
// SCREEN: ui_Screen1
void ui_Screen1_screen_init(void);
void ui_event_Screen1(lv_event_t * e);
extern lv_obj_t * ui_Screen1;
extern lv_obj_t * ui_Panel1;
```

```
extern lv_obj_t * ui_WaitingGPS;
extern lv_obj_t * ui_SpeedPanel;
extern lv_obj_t * ui_Image1;
extern lv_obj_t * ui_Slider1;
extern lv_obj_t * ui_SpeedLabelMin;
extern lv_obj_t * ui_SpeedLabelMax;
extern lv_obj_t * ui_SpeedTextPanel;
extern lv_obj_t * ui_Units;
extern lv_obj_t * ui_Labelback;
extern lv_obj_t * ui_Labelfront;
// SCREEN: ui_Screen2
void ui_Screen2_screen_init(void);
void ui_event_Screen2(lv_event_t * e);
extern lv_obj_t * ui_Screen2;
extern lv_obj_t * ui_Panel2;
extern lv_obj_t * ui_TopPanel1;
extern lv_obj_t * ui_Titulo1;
extern lv_obj_t * ui_TopPanel2;
extern lv_obj_t * ui_Titulo2;
extern lv_obj_t * ui_Panel3;
extern lv_obj_t * ui_Satelitespanel;
extern lv_obj_t * ui_Satelitestittle;
extern lv_obj_t * ui_Satelitesnumero;
extern lv_obj_t * ui_Aceleracionx;
extern lv_obj_t * ui_Aceleracionxtitulo;
extern lv_obj_t * ui_Aceleracionxd;
extern lv_obj_t * ui_Grados1;
extern lv_obj_t * ui_Coordenadastitulo;
extern lv_obj_t * ui_Gradosnumero1;
extern lv_obj_t * ui_Gradoslatitudn;
extern lv_obj_t * ui_Gradoslongitudn;
extern lv_obj_t * ui_Aceleraciony;
extern lv_obj_t * ui_Aceleracionytitulo;
extern lv_obj_t * ui_aceleracionyd;
extern lv_obj_t * ui_Altitud;
extern lv_obj_t * ui_Altitudtitulo;
extern lv_obj_t * ui_Altitudt;
extern lv_obj_t * ui_Aceleracionz;
extern lv_obj_t * ui_Aceleracionztitulo;
extern lv_obj_t * ui_Aceleracionzd;
extern lv_obj_t * ui____initial_actions0;

LV_IMG_DECLARE(ui_img_speedbg_png);    // assets\speedBG.png
LV_IMG_DECLARE(ui_img_speedfg_png);    // assets\SpeedFG.png

LV_FONT_DECLARE(ui_font_Display16);
LV_FONT_DECLARE(ui_font_Display20);
```

```
LV_FONT_DECLARE(ui_font_Display24);
LV_FONT_DECLARE(ui_font_Display70);

void ui_init(void);

#ifdef __cplusplus
} /*extern "C"*/
#endif

#endif
```

- **ui_events.h:**

```
// This file was generated by SquareLine Studio
// SquareLine Studio version: SquareLine Studio 1.3.4
// LVGL version: 8.3.6

#ifndef _UI_EVENTS_H
#define _UI_EVENTS_H

#ifdef __cplusplus
extern "C" {
#endif

#ifdef __cplusplus
} /*extern "C"*/
#endif

#endif
```

- **ui_helpers.c:**

```
// This file was generated by SquareLine Studio
// SquareLine Studio version: SquareLine Studio 1.3.4
// LVGL version: 8.3.6

#include "ui_helpers.h"

void _ui_bar_set_property(lv_obj_t * target, int id, int val)
{
    if(id == _UI_BAR_PROPERTY_VALUE_WITH_ANIM) lv_bar_set_value(target,
val, LV_ANIM_ON);
     if(id == _UI_BAR_PROPERTY_VALUE) lv_bar_set_value(target, val,
LV_ANIM_OFF);
}

void _ui_basic_set_property(lv_obj_t * target, int id, int val)
{
```

```
    if(id == _UI_BASIC_PROPERTY_POSITION_X) lv_obj_set_x(target, val);
    if(id == _UI_BASIC_PROPERTY_POSITION_Y) lv_obj_set_y(target, val);
    if(id == _UI_BASIC_PROPERTY_WIDTH) lv_obj_set_width(target, val);
    if(id == _UI_BASIC_PROPERTY_HEIGHT) lv_obj_set_height(target, val);
}

void _ui_dropdown_set_property(lv_obj_t * target, int id, int val)
{
                    if(id        ==        _UI_DROPDOWN_PROPERTY_SELECTED)
lv_dropdown_set_selected(target, val);
}

void _ui_image_set_property(lv_obj_t * target, int id, uint8_t * val)
{
    if(id == _UI_IMAGE_PROPERTY_IMAGE) lv_img_set_src(target, val);
}

void _ui_label_set_property(lv_obj_t * target, int id, const char * val)
{
    if(id == _UI_LABEL_PROPERTY_TEXT) lv_label_set_text(target, val);
}

void _ui_roller_set_property(lv_obj_t * target, int id, int val)
{
               if(id      ==      _UI_ROLLER_PROPERTY_SELECTED_WITH_ANIM)
lv_roller_set_selected(target, val, LV_ANIM_ON);
    if(id == _UI_ROLLER_PROPERTY_SELECTED) lv_roller_set_selected(target,
val, LV_ANIM_OFF);
}

void _ui_slider_set_property(lv_obj_t * target, int id, int val)
{
                if(id       ==       _UI_SLIDER_PROPERTY_VALUE_WITH_ANIM)
lv_slider_set_value(target, val, LV_ANIM_ON);
    if(id == _UI_SLIDER_PROPERTY_VALUE) lv_slider_set_value(target, val,
LV_ANIM_OFF);
}

void _ui_screen_change(lv_obj_t ** target, lv_scr_load_anim_t fademode,
int spd, int delay, void (*target_init)(void))
{
    if(*target == NULL)
        target_init();
    lv_scr_load_anim(*target, fademode, spd, delay, false);
}

void _ui_screen_delete(lv_obj_t ** target)
```

```
{
    if(*target == NULL) {
        lv_obj_del(*target);
        target = NULL;
    }
}

void _ui_arc_increment(lv_obj_t * target, int val)
{
    int old = lv_arc_get_value(target);
    lv_arc_set_value(target, old + val);
    lv_event_send(target, LV_EVENT_VALUE_CHANGED, 0);
}

void _ui_bar_increment(lv_obj_t * target, int val, int anm)
{
    int old = lv_bar_get_value(target);
    lv_bar_set_value(target, old + val, anm);
}

void _ui_slider_increment(lv_obj_t * target, int val, int anm)
{
    int old = lv_slider_get_value(target);
    lv_slider_set_value(target, old + val, anm);
    lv_event_send(target, LV_EVENT_VALUE_CHANGED, 0);
}

void _ui_keyboard_set_target(lv_obj_t * keyboard, lv_obj_t * textarea)
{
    lv_keyboard_set_textarea(keyboard, textarea);
}

void _ui_flag_modify(lv_obj_t * target, int32_t flag, int value)
{
    if(value == _UI_MODIFY_FLAG_TOGGLE) {
            if(lv_obj_has_flag(target,  flag))  lv_obj_clear_flag(target,
flag);
        else lv_obj_add_flag(target, flag);
    }
    else if(value == _UI_MODIFY_FLAG_ADD) lv_obj_add_flag(target, flag);
    else lv_obj_clear_flag(target, flag);
}
void _ui_state_modify(lv_obj_t * target, int32_t state, int value)
{
    if(value == _UI_MODIFY_STATE_TOGGLE) {
          if(lv_obj_has_state(target,  state))  lv_obj_clear_state(target,
state);
```

```
        else lv_obj_add_state(target, state);
    }
      else  if(value  ==  _UI_MODIFY_STATE_ADD)  lv_obj_add_state(target,
state);
    else lv_obj_clear_state(target, state);
}

void scr_unloaded_delete_cb(lv_event_t * e)
{
    lv_obj_t ** var = lv_event_get_user_data(e);
    lv_obj_del(*var);
    (*var) = NULL;
}

void _ui_opacity_set(lv_obj_t * target, int val)
{
    lv_obj_set_style_opa(target, val, 0);
}

void _ui_anim_callback_free_user_data(lv_anim_t * a)
{
    lv_mem_free(a->user_data);
    a->user_data = NULL;
}

void _ui_anim_callback_set_x(lv_anim_t * a, int32_t v)
{
    ui_anim_user_data_t * usr = (ui_anim_user_data_t *)a->user_data;
    lv_obj_set_x(usr->target, v);
}

void _ui_anim_callback_set_y(lv_anim_t * a, int32_t v)
{
    ui_anim_user_data_t * usr = (ui_anim_user_data_t *)a->user_data;
    lv_obj_set_y(usr->target, v);
}

void _ui_anim_callback_set_width(lv_anim_t * a, int32_t v)
{
    ui_anim_user_data_t * usr = (ui_anim_user_data_t *)a->user_data;
    lv_obj_set_width(usr->target, v);
}

void _ui_anim_callback_set_height(lv_anim_t * a, int32_t v)
{
    ui_anim_user_data_t * usr = (ui_anim_user_data_t *)a->user_data;
    lv_obj_set_height(usr->target, v);
```

```
}

void _ui_anim_callback_set_opacity(lv_anim_t * a, int32_t v)
{
    ui_anim_user_data_t * usr = (ui_anim_user_data_t *)a->user_data;
    lv_obj_set_style_opa(usr->target, v, 0);
}

void _ui_anim_callback_set_image_zoom(lv_anim_t * a, int32_t v)
{
    ui_anim_user_data_t * usr = (ui_anim_user_data_t *)a->user_data;
    lv_img_set_zoom(usr->target, v);
}

void _ui_anim_callback_set_image_angle(lv_anim_t * a, int32_t v)
{
    ui_anim_user_data_t * usr = (ui_anim_user_data_t *)a->user_data;
    lv_img_set_angle(usr->target, v);
}

void _ui_anim_callback_set_image_frame(lv_anim_t * a, int32_t v)
{
    ui_anim_user_data_t * usr = (ui_anim_user_data_t *)a->user_data;
    usr->val = v;
    if(v < 0) v = 0;
    if(v >= usr->imgset_size) v = usr->imgset_size - 1;
    lv_img_set_src(usr->target, usr->imgset[v]);
}

int32_t _ui_anim_callback_get_x(lv_anim_t * a)
{
    ui_anim_user_data_t * usr = (ui_anim_user_data_t *)a->user_data;
    return lv_obj_get_x_aligned(usr->target);
}

int32_t _ui_anim_callback_get_y(lv_anim_t * a)
{
    ui_anim_user_data_t * usr = (ui_anim_user_data_t *)a->user_data;
    return lv_obj_get_y_aligned(usr->target);
}

int32_t _ui_anim_callback_get_width(lv_anim_t * a)
{
    ui_anim_user_data_t * usr = (ui_anim_user_data_t *)a->user_data;
    return lv_obj_get_width(usr->target);
}
```

```
int32_t _ui_anim_callback_get_height(lv_anim_t * a)
{
    ui_anim_user_data_t * usr = (ui_anim_user_data_t *)a->user_data;
    return lv_obj_get_height(usr->target);
}

int32_t _ui_anim_callback_get_opacity(lv_anim_t * a)
{
    ui_anim_user_data_t * usr = (ui_anim_user_data_t *)a->user_data;
    return lv_obj_get_style_opa(usr->target, 0);
}

int32_t _ui_anim_callback_get_image_zoom(lv_anim_t * a)
{
    ui_anim_user_data_t * usr = (ui_anim_user_data_t *)a->user_data;
    return lv_img_get_zoom(usr->target);
}

int32_t _ui_anim_callback_get_image_angle(lv_anim_t * a)
{
    ui_anim_user_data_t * usr = (ui_anim_user_data_t *)a->user_data;
    return lv_img_get_angle(usr->target);
}

int32_t _ui_anim_callback_get_image_frame(lv_anim_t * a)
{
    ui_anim_user_data_t * usr = (ui_anim_user_data_t *)a->user_data;
    return usr->val;
}

void _ui_arc_set_text_value(lv_obj_t * trg, lv_obj_t * src, const char *
prefix, const char * postfix)
{
    char buf[_UI_TEMPORARY_STRING_BUFFER_SIZE];
            lv_snprintf(buf,     sizeof(buf),     "%s%d%s",     prefix,
(int)lv_arc_get_value(src), postfix);
    lv_label_set_text(trg, buf);
}

void _ui_slider_set_text_value(lv_obj_t * trg, lv_obj_t * src, const char
* prefix, const char * postfix)
{
    char buf[_UI_TEMPORARY_STRING_BUFFER_SIZE];
            lv_snprintf(buf,     sizeof(buf),     "%s%d%s",     prefix,
(int)lv_slider_get_value(src), postfix);
    lv_label_set_text(trg, buf);
}
```

```
void _ui_checked_set_text_value(lv_obj_t * trg, lv_obj_t * src, const
char * txt_on, const char * txt_off)
{
      if(lv_obj_has_state(src, LV_STATE_CHECKED)) lv_label_set_text(trg,
txt_on);
    else lv_label_set_text(trg, txt_off);
}

void _ui_spinbox_step(lv_obj_t * target, int val)
{
    if(val > 0) lv_spinbox_increment(target);
    else lv_spinbox_decrement(target);

    lv_event_send(target, LV_EVENT_VALUE_CHANGED, 0);
}
```

- **ui_helpers.h:**

```
// This file was generated by SquareLine Studio
// SquareLine Studio version: SquareLine Studio 1.3.4
// LVGL version: 8.3.6

#ifndef _PRUEBA1_02_UI_HELPERS_H
#define _PRUEBA1_02_UI_HELPERS_H

#ifdef __cplusplus
extern "C" {
#endif

#include "ui.h"

#define _UI_TEMPORARY_STRING_BUFFER_SIZE 32
#define _UI_BAR_PROPERTY_VALUE 0
#define _UI_BAR_PROPERTY_VALUE_WITH_ANIM 1
void _ui_bar_set_property(lv_obj_t * target, int id, int val);

#define _UI_BASIC_PROPERTY_POSITION_X 0
#define _UI_BASIC_PROPERTY_POSITION_Y 1
#define _UI_BASIC_PROPERTY_WIDTH 2
#define _UI_BASIC_PROPERTY_HEIGHT 3
void _ui_basic_set_property(lv_obj_t * target, int id, int val);

#define _UI_DROPDOWN_PROPERTY_SELECTED 0
void _ui_dropdown_set_property(lv_obj_t * target, int id, int val);

#define _UI_IMAGE_PROPERTY_IMAGE 0
```

```
void _ui_image_set_property(lv_obj_t * target, int id, uint8_t * val);

#define _UI_LABEL_PROPERTY_TEXT 0
void _ui_label_set_property(lv_obj_t * target, int id, const char * val);

#define _UI_ROLLER_PROPERTY_SELECTED 0
#define _UI_ROLLER_PROPERTY_SELECTED_WITH_ANIM 1
void _ui_roller_set_property(lv_obj_t * target, int id, int val);

#define _UI_SLIDER_PROPERTY_VALUE 0
#define _UI_SLIDER_PROPERTY_VALUE_WITH_ANIM 1
void _ui_slider_set_property(lv_obj_t * target, int id, int val);

void _ui_screen_change(lv_obj_t ** target, lv_scr_load_anim_t fademode,
int spd, int delay, void (*target_init)(void));

void _ui_screen_delete(lv_obj_t ** target);

void _ui_arc_increment(lv_obj_t * target, int val);

void _ui_bar_increment(lv_obj_t * target, int val, int anm);

void _ui_slider_increment(lv_obj_t * target, int val, int anm);

void _ui_keyboard_set_target(lv_obj_t * keyboard, lv_obj_t * textarea);

#define _UI_MODIFY_FLAG_ADD 0
#define _UI_MODIFY_FLAG_REMOVE 1
#define _UI_MODIFY_FLAG_TOGGLE 2
void _ui_flag_modify(lv_obj_t * target, int32_t flag, int value);

#define _UI_MODIFY_STATE_ADD 0
#define _UI_MODIFY_STATE_REMOVE 1
#define _UI_MODIFY_STATE_TOGGLE 2
void _ui_state_modify(lv_obj_t * target, int32_t state, int value);

void scr_unloaded_delete_cb(lv_event_t * e);

void _ui_opacity_set(lv_obj_t * target, int val);

/** Describes an animation*/
typedef struct _ui_anim_user_data_t {
    lv_obj_t * target;
    lv_img_dsc_t ** imgset;
    int32_t imgset_size;
    int32_t val;
} ui_anim_user_data_t;
```

```
void _ui_anim_callback_free_user_data(lv_anim_t * a);

void _ui_anim_callback_set_x(lv_anim_t * a, int32_t v);

void _ui_anim_callback_set_y(lv_anim_t * a, int32_t v);

void _ui_anim_callback_set_width(lv_anim_t * a, int32_t v);

void _ui_anim_callback_set_height(lv_anim_t * a, int32_t v);

void _ui_anim_callback_set_opacity(lv_anim_t * a, int32_t v);

void _ui_anim_callback_set_image_zoom(lv_anim_t * a, int32_t v);

void _ui_anim_callback_set_image_angle(lv_anim_t * a, int32_t v);

void _ui_anim_callback_set_image_frame(lv_anim_t * a, int32_t v);

int32_t _ui_anim_callback_get_x(lv_anim_t * a);

int32_t _ui_anim_callback_get_y(lv_anim_t * a);

int32_t _ui_anim_callback_get_width(lv_anim_t * a);

int32_t _ui_anim_callback_get_height(lv_anim_t * a);

int32_t _ui_anim_callback_get_opacity(lv_anim_t * a);

int32_t _ui_anim_callback_get_image_zoom(lv_anim_t * a);

int32_t _ui_anim_callback_get_image_angle(lv_anim_t * a);

int32_t _ui_anim_callback_get_image_frame(lv_anim_t * a);

void _ui_arc_set_text_value(lv_obj_t * trg, lv_obj_t * src, const char *
prefix, const char * postfix);

void _ui_slider_set_text_value(lv_obj_t * trg, lv_obj_t * src, const char
* prefix, const char * postfix);

void  _ui_checked_set_text_value(lv_obj_t  *  trg,  lv_obj_t  *  src,  const
char * txt_on, const char * txt_off);

void _ui_spinbox_step(lv_obj_t * target, int val)
;

#ifdef __cplusplus
```

```
} /*extern "C"*/
#endif

#endif
```

3. CÓDIGO QUE SE ENCARGA DE LOS EVENTOS Y CONFIGURACIONES, IMÁGENES Y TIPOS DE LETRAS DE LAS PANTALLAS 1 Y 2. (UI_SCREEN1.C, UI_SCREEN2.C, UI_IMG_SPEEDBG_PNG.C, UI_IMG_SPEEDFG_PNG.C, UI_FONT_DISPLAY16.C, UI_FONT_DISPLAY20.C, UI_FONT_DISPLAY24.C, UI_FONT_DISPLAY70.C)

- **ui_Screen1.c:**

```
// This file was generated by SquareLine Studio
// SquareLine Studio version: SquareLine Studio 1.3.4
// LVGL version: 8.3.6

#include "ui.h"

void ui_Screen1_screen_init(void)
{
    ui_Screen1 = lv_obj_create(NULL);
    lv_obj_clear_flag(ui_Screen1, LV_OBJ_FLAG_SCROLLABLE);      /// Flags

    ui_Panel1 = lv_obj_create(ui_Screen1);
    lv_obj_set_width(ui_Panel1, 320);
    lv_obj_set_height(ui_Panel1, 170);
    lv_obj_set_align(ui_Panel1, LV_ALIGN_CENTER);
    lv_obj_clear_flag(ui_Panel1, LV_OBJ_FLAG_SCROLLABLE);      /// Flags
          lv_obj_set_style_radius(ui_Panel1,    0,    LV_PART_MAIN    |
LV_STATE_DEFAULT);
         lv_obj_set_style_bg_color(ui_Panel1,    lv_color_hex(0x000000),
LV_PART_MAIN | LV_STATE_DEFAULT);
         lv_obj_set_style_bg_opa(ui_Panel1,    255,    LV_PART_MAIN    |
LV_STATE_DEFAULT);
       lv_obj_set_style_border_color(ui_Panel1,   lv_color_hex(0x000000),
LV_PART_MAIN | LV_STATE_DEFAULT);
        lv_obj_set_style_border_opa(ui_Panel1,    0,    LV_PART_MAIN    |
LV_STATE_DEFAULT);
        lv_obj_set_style_border_width(ui_Panel1,    0,    LV_PART_MAIN    |
LV_STATE_DEFAULT);

    ui_WaitingGPS = lv_spinner_create(ui_Panel1, 1000, 90);
    lv_obj_set_width(ui_WaitingGPS, 20);
```

```
    lv_obj_set_height(ui_WaitingGPS, 20);
    lv_obj_set_x(ui_WaitingGPS, 7);
    lv_obj_set_y(ui_WaitingGPS, 6);
    lv_obj_set_align(ui_WaitingGPS, LV_ALIGN_BOTTOM_RIGHT);
     lv_obj_clear_flag(ui_WaitingGPS,  LV_OBJ_FLAG_CLICKABLE);         ///
Flags
      lv_obj_set_style_arc_color(ui_WaitingGPS,   lv_color_hex(0x00FDFE),
LV_PART_MAIN | LV_STATE_DEFAULT);
       lv_obj_set_style_arc_opa(ui_WaitingGPS,    0,    LV_PART_MAIN    |
LV_STATE_DEFAULT);
       lv_obj_set_style_arc_width(ui_WaitingGPS,    4,    LV_PART_MAIN    |
LV_STATE_DEFAULT);

      lv_obj_set_style_arc_color(ui_WaitingGPS,   lv_color_hex(0x00FDFE),
LV_PART_INDICATOR | LV_STATE_DEFAULT);
     lv_obj_set_style_arc_opa(ui_WaitingGPS,  255,  LV_PART_INDICATOR  |
LV_STATE_DEFAULT);
     lv_obj_set_style_arc_width(ui_WaitingGPS,  4,  LV_PART_INDICATOR  |
LV_STATE_DEFAULT);

    ui_SpeedPanel = lv_obj_create(ui_Screen1);
    lv_obj_set_height(ui_SpeedPanel, 162);
    lv_obj_set_width(ui_SpeedPanel, lv_pct(100));
    lv_obj_set_align(ui_SpeedPanel, LV_ALIGN_CENTER);
     lv_obj_clear_flag(ui_SpeedPanel,  LV_OBJ_FLAG_SCROLLABLE);         ///
Flags
      lv_obj_set_style_bg_color(ui_SpeedPanel,   lv_color_hex(0xFFFFFF),
LV_PART_MAIN | LV_STATE_DEFAULT);
       lv_obj_set_style_bg_opa(ui_SpeedPanel,    0,    LV_PART_MAIN    |
LV_STATE_DEFAULT);
    lv_obj_set_style_border_color(ui_SpeedPanel,  lv_color_hex(0x000000),
LV_PART_MAIN | LV_STATE_DEFAULT);
      lv_obj_set_style_border_opa(ui_SpeedPanel,   0,   LV_PART_MAIN   |
LV_STATE_DEFAULT);

    ui_Image1 = lv_img_create(ui_SpeedPanel);
    lv_img_set_src(ui_Image1, &ui_img_speedbg_png);
    lv_obj_set_width(ui_Image1, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_Image1, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_align(ui_Image1, LV_ALIGN_CENTER);
    lv_obj_add_flag(ui_Image1, LV_OBJ_FLAG_ADV_HITTEST);     /// Flags
    lv_obj_clear_flag(ui_Image1, LV_OBJ_FLAG_SCROLLABLE);      /// Flags

    ui_Slider1 = lv_slider_create(ui_Image1);
    lv_slider_set_range(ui_Slider1, 0, 250);
    lv_slider_set_value(ui_Slider1, 120, LV_ANIM_OFF);
```

```
        if(lv_slider_get_mode(ui_Slider1)    ==    LV_SLIDER_MODE_RANGE)
lv_slider_set_left_value(ui_Slider1, 0, LV_ANIM_OFF);
    lv_obj_set_width(ui_Slider1, 320);
    lv_obj_set_height(ui_Slider1, 140);
    lv_obj_set_align(ui_Slider1, LV_ALIGN_CENTER);
         lv_obj_set_style_bg_color(ui_Slider1,    lv_color_hex(0xFFFFFF),
LV_PART_MAIN | LV_STATE_DEFAULT);
          lv_obj_set_style_bg_opa(ui_Slider1,    0,    LV_PART_MAIN    |
LV_STATE_DEFAULT);

        lv_obj_set_style_radius(ui_Slider1,    0,    LV_PART_INDICATOR    |
LV_STATE_DEFAULT);
         lv_obj_set_style_bg_color(ui_Slider1,    lv_color_hex(0xFFFFFF),
LV_PART_INDICATOR | LV_STATE_DEFAULT);
        lv_obj_set_style_bg_opa(ui_Slider1,    0,    LV_PART_INDICATOR    |
LV_STATE_DEFAULT);
          lv_obj_set_style_bg_img_src(ui_Slider1,    &ui_img_speedfg_png,
LV_PART_INDICATOR | LV_STATE_DEFAULT);

          lv_obj_set_style_radius(ui_Slider1,    0,    LV_PART_KNOB    |
LV_STATE_DEFAULT);
         lv_obj_set_style_bg_color(ui_Slider1,    lv_color_hex(0xFFFFFF),
LV_PART_KNOB | LV_STATE_DEFAULT);
          lv_obj_set_style_bg_opa(ui_Slider1,    0,    LV_PART_KNOB    |
LV_STATE_DEFAULT);

    ui_SpeedLabelMin = lv_label_create(ui_Screen1);
    lv_obj_set_width(ui_SpeedLabelMin, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_SpeedLabelMin, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_x(ui_SpeedLabelMin, -147);
    lv_obj_set_y(ui_SpeedLabelMin, 56);
    lv_obj_set_align(ui_SpeedLabelMin, LV_ALIGN_CENTER);
    lv_label_set_text(ui_SpeedLabelMin, "0");
    lv_obj_set_style_text_color(ui_SpeedLabelMin, lv_color_hex(0x00FDFE),
LV_PART_MAIN | LV_STATE_DEFAULT);
      lv_obj_set_style_text_opa(ui_SpeedLabelMin,   255,   LV_PART_MAIN   |
LV_STATE_DEFAULT);
       lv_obj_set_style_text_font(ui_SpeedLabelMin,   &ui_font_Display24,
LV_PART_MAIN | LV_STATE_DEFAULT);

    ui_SpeedLabelMax = lv_label_create(ui_Screen1);
    lv_obj_set_width(ui_SpeedLabelMax, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_SpeedLabelMax, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_x(ui_SpeedLabelMax, 133);
    lv_obj_set_y(ui_SpeedLabelMax, -60);
    lv_obj_set_align(ui_SpeedLabelMax, LV_ALIGN_CENTER);
    lv_label_set_text(ui_SpeedLabelMax, "250");
```

```
    lv_obj_set_style_text_color(ui_SpeedLabelMax, lv_color_hex(0x00FDFE),
LV_PART_MAIN | LV_STATE_DEFAULT);
      lv_obj_set_style_text_opa(ui_SpeedLabelMax,  255,  LV_PART_MAIN  |
LV_STATE_DEFAULT);
       lv_obj_set_style_text_font(ui_SpeedLabelMax,  &ui_font_Display24,
LV_PART_MAIN | LV_STATE_DEFAULT);

    ui_SpeedTextPanel = lv_obj_create(ui_Screen1);
    lv_obj_set_height(ui_SpeedTextPanel, 63);
    lv_obj_set_width(ui_SpeedTextPanel, lv_pct(100));
    lv_obj_set_x(ui_SpeedTextPanel, 0);
    lv_obj_set_y(ui_SpeedTextPanel, 40);
    lv_obj_set_align(ui_SpeedTextPanel, LV_ALIGN_CENTER);
                                    lv_obj_clear_flag(ui_SpeedTextPanel,
LV_OBJ_FLAG_SCROLLABLE);      /// Flags
     lv_obj_set_style_bg_color(ui_SpeedTextPanel, lv_color_hex(0xFFFFFF),
LV_PART_MAIN | LV_STATE_DEFAULT);
       lv_obj_set_style_bg_opa(ui_SpeedTextPanel,  0,  LV_PART_MAIN  |
LV_STATE_DEFAULT);
                         lv_obj_set_style_border_color(ui_SpeedTextPanel,
lv_color_hex(0x000000), LV_PART_MAIN | LV_STATE_DEFAULT);
      lv_obj_set_style_border_opa(ui_SpeedTextPanel,  0,  LV_PART_MAIN  |
LV_STATE_DEFAULT);

    ui_Units = lv_label_create(ui_SpeedTextPanel);
    lv_obj_set_width(ui_Units, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_Units, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_x(ui_Units, 106);
    lv_obj_set_y(ui_Units, 13);
    lv_obj_set_align(ui_Units, LV_ALIGN_CENTER);
    lv_label_set_text(ui_Units, "Km/h");
         lv_obj_set_style_text_color(ui_Units,  lv_color_hex(0x00FDFE),
LV_PART_MAIN | LV_STATE_DEFAULT);
         lv_obj_set_style_text_opa(ui_Units,  255,  LV_PART_MAIN  |
LV_STATE_DEFAULT);
    lv_obj_set_style_text_font(ui_Units, &ui_font_Display20, LV_PART_MAIN
| LV_STATE_DEFAULT);

    ui_Labelback = lv_label_create(ui_SpeedTextPanel);
    lv_obj_set_width(ui_Labelback, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_Labelback, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_x(ui_Labelback, 29);
    lv_obj_set_y(ui_Labelback, -5);
    lv_obj_set_align(ui_Labelback, LV_ALIGN_CENTER);
    lv_label_set_text(ui_Labelback, "000");
       lv_obj_set_style_text_color(ui_Labelback,  lv_color_hex(0x383838),
LV_PART_MAIN | LV_STATE_DEFAULT);
```

```
        lv_obj_set_style_text_opa(ui_Labelback,    255,    LV_PART_MAIN    |
LV_STATE_DEFAULT);
          lv_obj_set_style_text_font(ui_Labelback,    &ui_font_Display70,
LV_PART_MAIN | LV_STATE_DEFAULT);

    ui_Labelfront = lv_label_create(ui_SpeedTextPanel);
    lv_obj_set_width(ui_Labelfront, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_Labelfront, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_x(ui_Labelfront, 34);
    lv_obj_set_y(ui_Labelfront, -5);
    lv_obj_set_align(ui_Labelfront, LV_ALIGN_CENTER);
    lv_label_set_text(ui_Labelfront, "120");
      lv_obj_set_style_text_color(ui_Labelfront,  lv_color_hex(0x00FDFE),
LV_PART_MAIN | LV_STATE_DEFAULT);
       lv_obj_set_style_text_opa(ui_Labelfront,    255,    LV_PART_MAIN    |
LV_STATE_DEFAULT);
         lv_obj_set_style_text_font(ui_Labelfront,    &ui_font_Display70,
LV_PART_MAIN | LV_STATE_DEFAULT);

       lv_obj_add_event_cb(ui_Screen1,   ui_event_Screen1,   LV_EVENT_ALL,
NULL);

}
```

- **ui_Screen2.c:**

```
// This file was generated by SquareLine Studio
// SquareLine Studio version: SquareLine Studio 1.3.4
// LVGL version: 8.3.6


#include "ui.h"

void ui_Screen2_screen_init(void)
{
    ui_Screen2 = lv_obj_create(NULL);
    lv_obj_clear_flag(ui_Screen2, LV_OBJ_FLAG_SCROLLABLE);      /// Flags

    ui_Panel2 = lv_obj_create(ui_Screen2);
    lv_obj_set_width(ui_Panel2, 320);
    lv_obj_set_height(ui_Panel2, 170);
    lv_obj_set_align(ui_Panel2, LV_ALIGN_CENTER);
    lv_obj_clear_flag(ui_Panel2, LV_OBJ_FLAG_SCROLLABLE);      /// Flags
          lv_obj_set_style_radius(ui_Panel2,    0,    LV_PART_MAIN    |
LV_STATE_DEFAULT);
          lv_obj_set_style_bg_color(ui_Panel2,    lv_color_hex(0x000000),
LV_PART_MAIN | LV_STATE_DEFAULT);
```

```
        lv_obj_set_style_bg_opa(ui_Panel2,    255,    LV_PART_MAIN    |
LV_STATE_DEFAULT);
       lv_obj_set_style_border_color(ui_Panel2,   lv_color_hex(0x000000),
LV_PART_MAIN | LV_STATE_DEFAULT);
        lv_obj_set_style_border_opa(ui_Panel2,    0,    LV_PART_MAIN    |
LV_STATE_DEFAULT);

    ui_TopPanel1 = lv_obj_create(ui_Panel2);
    lv_obj_set_width(ui_TopPanel1, 85);
    lv_obj_set_height(ui_TopPanel1, 38);
    lv_obj_set_x(ui_TopPanel1, -114);
    lv_obj_set_y(ui_TopPanel1, -10);
    lv_obj_set_align(ui_TopPanel1, LV_ALIGN_TOP_MID);
     lv_obj_clear_flag(ui_TopPanel1,  LV_OBJ_FLAG_SCROLLABLE);        ///
Flags
        lv_obj_set_style_radius(ui_TopPanel1,    0,    LV_PART_MAIN    |
LV_STATE_DEFAULT);
       lv_obj_set_style_bg_color(ui_TopPanel1,   lv_color_hex(0x00FDFE),
LV_PART_MAIN | LV_STATE_DEFAULT);
        lv_obj_set_style_bg_opa(ui_TopPanel1,    255,    LV_PART_MAIN    |
LV_STATE_DEFAULT);
     lv_obj_set_style_border_color(ui_TopPanel1,  lv_color_hex(0xFF0000),
LV_PART_MAIN | LV_STATE_DEFAULT);
      lv_obj_set_style_border_opa(ui_TopPanel1,   255,   LV_PART_MAIN   |
LV_STATE_DEFAULT);

    ui_Titulo1 = lv_label_create(ui_TopPanel1);
    lv_obj_set_width(ui_Titulo1, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_Titulo1, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_align(ui_Titulo1, LV_ALIGN_CENTER);
    lv_label_set_text(ui_Titulo1, "GPS");
           lv_obj_set_style_text_font(ui_Titulo1,     &ui_font_Display24,
LV_PART_MAIN | LV_STATE_DEFAULT);

    ui_TopPanel2 = lv_obj_create(ui_Panel2);
    lv_obj_set_width(ui_TopPanel2, 85);
    lv_obj_set_height(ui_TopPanel2, 38);
    lv_obj_set_x(ui_TopPanel2, 43);
    lv_obj_set_y(ui_TopPanel2, -10);
    lv_obj_set_align(ui_TopPanel2, LV_ALIGN_TOP_MID);
     lv_obj_clear_flag(ui_TopPanel2,  LV_OBJ_FLAG_SCROLLABLE);        ///
Flags
        lv_obj_set_style_radius(ui_TopPanel2,    0,    LV_PART_MAIN    |
LV_STATE_DEFAULT);
       lv_obj_set_style_bg_color(ui_TopPanel2,   lv_color_hex(0x00FDFE),
LV_PART_MAIN | LV_STATE_DEFAULT);
```

```
        lv_obj_set_style_bg_opa(ui_TopPanel2,    255,    LV_PART_MAIN    |
LV_STATE_DEFAULT);
     lv_obj_set_style_border_color(ui_TopPanel2,  lv_color_hex(0xFF0000),
LV_PART_MAIN | LV_STATE_DEFAULT);
      lv_obj_set_style_border_opa(ui_TopPanel2,   255,   LV_PART_MAIN   |
LV_STATE_DEFAULT);

    ui_Titulo2 = lv_label_create(ui_TopPanel2);
    lv_obj_set_width(ui_Titulo2, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_Titulo2, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_align(ui_Titulo2, LV_ALIGN_CENTER);
    lv_label_set_text(ui_Titulo2, "IMU");
          lv_obj_set_style_text_font(ui_Titulo2,     &ui_font_Display24,
LV_PART_MAIN | LV_STATE_DEFAULT);

    ui_Panel3 = lv_obj_create(ui_Screen2);
    lv_obj_set_width(ui_Panel3, 318);
    lv_obj_set_height(ui_Panel3, 124);
    lv_obj_set_x(ui_Panel3, 0);
    lv_obj_set_y(ui_Panel3, 21);
    lv_obj_set_align(ui_Panel3, LV_ALIGN_CENTER);
    lv_obj_clear_flag(ui_Panel3, LV_OBJ_FLAG_SCROLLABLE);      /// Flags
          lv_obj_set_style_bg_color(ui_Panel3,    lv_color_hex(0x000000),
LV_PART_MAIN | LV_STATE_DEFAULT);
         lv_obj_set_style_bg_opa(ui_Panel3,    255,    LV_PART_MAIN    |
LV_STATE_DEFAULT);
        lv_obj_set_style_border_width(ui_Panel3,    0,    LV_PART_MAIN    |
LV_STATE_DEFAULT);

    ui_Satelitespanel = lv_obj_create(ui_Panel3);
    lv_obj_set_width(ui_Satelitespanel, 69);
    lv_obj_set_height(ui_Satelitespanel, 50);
    lv_obj_set_x(ui_Satelitespanel, -116);
    lv_obj_set_y(ui_Satelitespanel, -35);
    lv_obj_set_align(ui_Satelitespanel, LV_ALIGN_CENTER);
                                lv_obj_clear_flag(ui_Satelitespanel,
LV_OBJ_FLAG_SCROLLABLE);      /// Flags
     lv_obj_set_style_bg_color(ui_Satelitespanel,  lv_color_hex(0x00FDFE),
LV_PART_MAIN | LV_STATE_DEFAULT);
      lv_obj_set_style_bg_opa(ui_Satelitespanel,   255,   LV_PART_MAIN   |
LV_STATE_DEFAULT);
                        lv_obj_set_style_border_color(ui_Satelitespanel,
lv_color_hex(0xFF002A), LV_PART_MAIN | LV_STATE_DEFAULT);
     lv_obj_set_style_border_opa(ui_Satelitespanel,  255,  LV_PART_MAIN  |
LV_STATE_DEFAULT);

    ui_Satelitestittle = lv_label_create(ui_Satelitespanel);
```

```
    lv_obj_set_width(ui_Satelitestittle, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_Satelitestittle, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_x(ui_Satelitestittle, -2);
    lv_obj_set_y(ui_Satelitestittle, -14);
    lv_obj_set_align(ui_Satelitestittle, LV_ALIGN_CENTER);
    lv_label_set_text(ui_Satelitestittle, "Satelites");
      lv_obj_set_style_text_font(ui_Satelitestittle,  &ui_font_Display16,
LV_PART_MAIN | LV_STATE_DEFAULT);

    ui_Satelitesnumero = lv_label_create(ui_Satelitespanel);
    lv_obj_set_width(ui_Satelitesnumero, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_Satelitesnumero, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_x(ui_Satelitesnumero, -3);
    lv_obj_set_y(ui_Satelitesnumero, 6);
    lv_obj_set_align(ui_Satelitesnumero, LV_ALIGN_CENTER);
    lv_label_set_text(ui_Satelitesnumero, "4");
      lv_obj_set_style_text_font(ui_Satelitesnumero,  &ui_font_Display24,
LV_PART_MAIN | LV_STATE_DEFAULT);

    ui_Aceleracionx = lv_obj_create(ui_Panel3);
    lv_obj_set_width(ui_Aceleracionx, 69);
    lv_obj_set_height(ui_Aceleracionx, 50);
    lv_obj_set_x(ui_Aceleracionx, -29);
    lv_obj_set_y(ui_Aceleracionx, -36);
    lv_obj_set_align(ui_Aceleracionx, LV_ALIGN_CENTER);
    lv_obj_clear_flag(ui_Aceleracionx, LV_OBJ_FLAG_SCROLLABLE);      ///
Flags
      lv_obj_set_style_bg_color(ui_Aceleracionx,  lv_color_hex(0x00FDFE),
LV_PART_MAIN | LV_STATE_DEFAULT);
       lv_obj_set_style_bg_opa(ui_Aceleracionx,  255,  LV_PART_MAIN  |
LV_STATE_DEFAULT);
                          lv_obj_set_style_border_color(ui_Aceleracionx,
lv_color_hex(0xFF002A), LV_PART_MAIN | LV_STATE_DEFAULT);
      lv_obj_set_style_border_opa(ui_Aceleracionx,  255,  LV_PART_MAIN  |
LV_STATE_DEFAULT);

    ui_Aceleracionxtitulo = lv_label_create(ui_Aceleracionx);
    lv_obj_set_width(ui_Aceleracionxtitulo, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_Aceleracionxtitulo, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_x(ui_Aceleracionxtitulo, -2);
    lv_obj_set_y(ui_Aceleracionxtitulo, -14);
    lv_obj_set_align(ui_Aceleracionxtitulo, LV_ALIGN_CENTER);
    lv_label_set_text(ui_Aceleracionxtitulo, "EJE X");
    lv_obj_set_style_text_font(ui_Aceleracionxtitulo, &ui_font_Display20,
LV_PART_MAIN | LV_STATE_DEFAULT);

    ui_Aceleracionxd = lv_label_create(ui_Aceleracionx);
```

```
    lv_obj_set_width(ui_Aceleracionxd, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_Aceleracionxd, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_x(ui_Aceleracionxd, -3);
    lv_obj_set_y(ui_Aceleracionxd, 6);
    lv_obj_set_align(ui_Aceleracionxd, LV_ALIGN_CENTER);
    lv_label_set_text(ui_Aceleracionxd, "360");
       lv_obj_set_style_text_font(ui_Aceleracionxd,   &ui_font_Display24,
LV_PART_MAIN | LV_STATE_DEFAULT);

    ui_Grados1 = lv_obj_create(ui_Panel3);
    lv_obj_set_width(ui_Grados1, 198);
    lv_obj_set_height(ui_Grados1, 62);
    lv_obj_set_x(ui_Grados1, -52);
    lv_obj_set_y(ui_Grados1, 24);
    lv_obj_set_align(ui_Grados1, LV_ALIGN_CENTER);
    lv_obj_clear_flag(ui_Grados1, LV_OBJ_FLAG_SCROLLABLE);      /// Flags
         lv_obj_set_style_bg_color(ui_Grados1,    lv_color_hex(0x00FDFE),
LV_PART_MAIN | LV_STATE_DEFAULT);
         lv_obj_set_style_bg_opa(ui_Grados1,    255,    LV_PART_MAIN    |
LV_STATE_DEFAULT);
      lv_obj_set_style_border_color(ui_Grados1,   lv_color_hex(0xFF002A),
LV_PART_MAIN | LV_STATE_DEFAULT);
       lv_obj_set_style_border_opa(ui_Grados1,    255,    LV_PART_MAIN    |
LV_STATE_DEFAULT);

    ui_Coordenadastitulo = lv_label_create(ui_Grados1);
    lv_obj_set_width(ui_Coordenadastitulo, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_Coordenadastitulo, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_x(ui_Coordenadastitulo, -41);
    lv_obj_set_y(ui_Coordenadastitulo, -22);
    lv_obj_set_align(ui_Coordenadastitulo, LV_ALIGN_CENTER);
    lv_label_set_text(ui_Coordenadastitulo, "Coordenadas");
     lv_obj_set_style_text_font(ui_Coordenadastitulo,  &ui_font_Display20,
LV_PART_MAIN | LV_STATE_DEFAULT);

    ui_Gradosnumero1 = lv_label_create(ui_Grados1);
    lv_obj_set_width(ui_Gradosnumero1, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_Gradosnumero1, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_x(ui_Gradosnumero1, -71);
    lv_obj_set_y(ui_Gradosnumero1, 6);
    lv_obj_set_align(ui_Gradosnumero1, LV_ALIGN_CENTER);
    lv_label_set_text(ui_Gradosnumero1, "Lat:\nLng:");
       lv_obj_set_style_text_font(ui_Gradosnumero1,   &ui_font_Display20,
LV_PART_MAIN | LV_STATE_DEFAULT);

    ui_Gradoslatitudn = lv_label_create(ui_Grados1);
    lv_obj_set_width(ui_Gradoslatitudn, LV_SIZE_CONTENT);   /// 1
```

```
    lv_obj_set_height(ui_Gradoslatitudn, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_x(ui_Gradoslatitudn, 3);
    lv_obj_set_y(ui_Gradoslatitudn, -5);
    lv_obj_set_align(ui_Gradoslatitudn, LV_ALIGN_CENTER);
    lv_label_set_text(ui_Gradoslatitudn, "-5451,35");
       lv_obj_set_style_text_font(ui_Gradoslatitudn,  &ui_font_Display20,
LV_PART_MAIN | LV_STATE_DEFAULT);

    ui_Gradoslongitudn = lv_label_create(ui_Grados1);
    lv_obj_set_width(ui_Gradoslongitudn, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_Gradoslongitudn, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_x(ui_Gradoslongitudn, 5);
    lv_obj_set_y(ui_Gradoslongitudn, 14);
    lv_obj_set_align(ui_Gradoslongitudn, LV_ALIGN_CENTER);
    lv_label_set_text(ui_Gradoslongitudn, "-5462,35");
      lv_obj_set_style_text_font(ui_Gradoslongitudn,  &ui_font_Display20,
LV_PART_MAIN | LV_STATE_DEFAULT);

    ui_Aceleraciony = lv_obj_create(ui_Panel3);
    lv_obj_set_width(ui_Aceleraciony, 69);
    lv_obj_set_height(ui_Aceleraciony, 50);
    lv_obj_set_x(ui_Aceleraciony, 45);
    lv_obj_set_y(ui_Aceleraciony, -36);
    lv_obj_set_align(ui_Aceleraciony, LV_ALIGN_CENTER);
    lv_obj_clear_flag(ui_Aceleraciony, LV_OBJ_FLAG_SCROLLABLE);      ///
Flags
      lv_obj_set_style_bg_color(ui_Aceleraciony,  lv_color_hex(0x00FDFE),
LV_PART_MAIN | LV_STATE_DEFAULT);
       lv_obj_set_style_bg_opa(ui_Aceleraciony,  255,  LV_PART_MAIN  |
LV_STATE_DEFAULT);
                          lv_obj_set_style_border_color(ui_Aceleraciony,
lv_color_hex(0xFF002A), LV_PART_MAIN | LV_STATE_DEFAULT);
      lv_obj_set_style_border_opa(ui_Aceleraciony,  255,  LV_PART_MAIN  |
LV_STATE_DEFAULT);

    ui_Aceleracionytitulo = lv_label_create(ui_Aceleraciony);
    lv_obj_set_width(ui_Aceleracionytitulo, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_Aceleracionytitulo, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_x(ui_Aceleracionytitulo, -2);
    lv_obj_set_y(ui_Aceleracionytitulo, -14);
    lv_obj_set_align(ui_Aceleracionytitulo, LV_ALIGN_CENTER);
    lv_label_set_text(ui_Aceleracionytitulo, "EJE Y");
    lv_obj_set_style_text_font(ui_Aceleracionytitulo, &ui_font_Display20,
LV_PART_MAIN | LV_STATE_DEFAULT);

    ui_aceleracionyd = lv_label_create(ui_Aceleraciony);
    lv_obj_set_width(ui_aceleracionyd, LV_SIZE_CONTENT);   /// 1
```

```
    lv_obj_set_height(ui_aceleracionyd, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_x(ui_aceleracionyd, -3);
    lv_obj_set_y(ui_aceleracionyd, 6);
    lv_obj_set_align(ui_aceleracionyd, LV_ALIGN_CENTER);
    lv_label_set_text(ui_aceleracionyd, "120");
        lv_obj_set_style_text_font(ui_aceleracionyd,   &ui_font_Display24,
LV_PART_MAIN | LV_STATE_DEFAULT);

    ui_Altitud = lv_obj_create(ui_Panel3);
    lv_obj_set_width(ui_Altitud, 94);
    lv_obj_set_height(ui_Altitud, 50);
    lv_obj_set_x(ui_Altitud, 103);
    lv_obj_set_y(ui_Altitud, 30);
    lv_obj_set_align(ui_Altitud, LV_ALIGN_CENTER);
    lv_obj_clear_flag(ui_Altitud, LV_OBJ_FLAG_SCROLLABLE);      /// Flags
         lv_obj_set_style_bg_color(ui_Altitud,    lv_color_hex(0x00FDFE),
LV_PART_MAIN | LV_STATE_DEFAULT);
          lv_obj_set_style_bg_opa(ui_Altitud,    255,    LV_PART_MAIN    |
LV_STATE_DEFAULT);
       lv_obj_set_style_border_color(ui_Altitud,   lv_color_hex(0xFF002A),
LV_PART_MAIN | LV_STATE_DEFAULT);
        lv_obj_set_style_border_opa(ui_Altitud,    255,    LV_PART_MAIN    |
LV_STATE_DEFAULT);

    ui_Altitudtitulo = lv_label_create(ui_Altitud);
    lv_obj_set_width(ui_Altitudtitulo, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_Altitudtitulo, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_x(ui_Altitudtitulo, 1);
    lv_obj_set_y(ui_Altitudtitulo, -14);
    lv_obj_set_align(ui_Altitudtitulo, LV_ALIGN_CENTER);
    lv_label_set_text(ui_Altitudtitulo, "Altitud(m)");
        lv_obj_set_style_text_font(ui_Altitudtitulo,   &ui_font_Display20,
LV_PART_MAIN | LV_STATE_DEFAULT);

    ui_Altitudt = lv_label_create(ui_Altitud);
    lv_obj_set_width(ui_Altitudt, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_Altitudt, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_x(ui_Altitudt, -3);
    lv_obj_set_y(ui_Altitudt, 6);
    lv_obj_set_align(ui_Altitudt, LV_ALIGN_CENTER);
    lv_label_set_text(ui_Altitudt, "452");
          lv_obj_set_style_text_font(ui_Altitudt,    &ui_font_Display24,
LV_PART_MAIN | LV_STATE_DEFAULT);

    ui_Aceleracionz = lv_obj_create(ui_Panel3);
    lv_obj_set_width(ui_Aceleracionz, 69);
    lv_obj_set_height(ui_Aceleracionz, 50);
```

```
    lv_obj_set_x(ui_Aceleracionz, 117);
    lv_obj_set_y(ui_Aceleracionz, -36);
    lv_obj_set_align(ui_Aceleracionz, LV_ALIGN_CENTER);
     lv_obj_clear_flag(ui_Aceleracionz, LV_OBJ_FLAG_SCROLLABLE);      ///
Flags
      lv_obj_set_style_bg_color(ui_Aceleracionz,  lv_color_hex(0x00FDFE),
LV_PART_MAIN | LV_STATE_DEFAULT);
       lv_obj_set_style_bg_opa(ui_Aceleracionz,  255,  LV_PART_MAIN  |
LV_STATE_DEFAULT);
                           lv_obj_set_style_border_color(ui_Aceleracionz,
lv_color_hex(0xFF002A), LV_PART_MAIN | LV_STATE_DEFAULT);
      lv_obj_set_style_border_opa(ui_Aceleracionz,  255,  LV_PART_MAIN  |
LV_STATE_DEFAULT);

    ui_Aceleracionztitulo = lv_label_create(ui_Aceleracionz);
    lv_obj_set_width(ui_Aceleracionztitulo, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_Aceleracionztitulo, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_x(ui_Aceleracionztitulo, -2);
    lv_obj_set_y(ui_Aceleracionztitulo, -14);
    lv_obj_set_align(ui_Aceleracionztitulo, LV_ALIGN_CENTER);
    lv_label_set_text(ui_Aceleracionztitulo, "EJE Z");
     lv_obj_set_style_text_font(ui_Aceleracionztitulo, &ui_font_Display20,
LV_PART_MAIN | LV_STATE_DEFAULT);

    ui_Aceleracionzd = lv_label_create(ui_Aceleracionz);
    lv_obj_set_width(ui_Aceleracionzd, LV_SIZE_CONTENT);   /// 1
    lv_obj_set_height(ui_Aceleracionzd, LV_SIZE_CONTENT);    /// 1
    lv_obj_set_x(ui_Aceleracionzd, -3);
    lv_obj_set_y(ui_Aceleracionzd, 6);
    lv_obj_set_align(ui_Aceleracionzd, LV_ALIGN_CENTER);
    lv_label_set_text(ui_Aceleracionzd, "120");
       lv_obj_set_style_text_font(ui_Aceleracionzd,   &ui_font_Display24,
LV_PART_MAIN | LV_STATE_DEFAULT);

      lv_obj_add_event_cb(ui_Screen2,   ui_event_Screen2,   LV_EVENT_ALL,
NULL);

}
```

- **ui_img_speedbg_png.c:**
- **ui_img_speedfg_png.c:**

En esta parte vendrían las imágenes en código binario, pero serían muchas páginas por lo que no puedo añadirlas.

- **ui_font_Display16.c:**

```
/*******************************************************************
*******
 * Size: 16 px
 * Bpp: 1
 *       Opts:       --bpp       1       --size       16       --font
C:\Users\julia\Documents\SquareLine\assets\Open   24   Display   St.ttf   -o
C:\Users\julia\Documents\SquareLine\assets\ui_font_Display16.c    --format
lvgl -r 0x20-0x7f --no-compress --no-prefilter
 *******************************************************************
******/

#include "ui.h"

#ifndef UI_FONT_DISPLAY16
#define UI_FONT_DISPLAY16 1
#endif

#if UI_FONT_DISPLAY16

/*-----------------
 *    BITMAPS
 *----------------*/

/*Store the image of the glyphs*/
static LV_ATTRIBUTE_LARGE_CONST const uint8_t glyph_bitmap[] = {
    /* U+0020 " " */
    0x0,

    /* U+0021 "!" */
    0xff, 0x60,

    /* U+0022 "\"" */
    0xf0,

    /* U+0023 "#" */
    0x3, 0x8, 0x4, 0x30, 0x18, 0xc1, 0xaf, 0x70,
    0xc2, 0x1, 0x8, 0x37, 0xfe, 0x8, 0x40, 0x31,
    0x80, 0x42, 0x0,

    /* U+0024 "$" */
    0x20, 0x8f, 0xa8, 0xa2, 0x88, 0x3e, 0x24, 0x92,
    0x41, 0x78, 0x82, 0x0,

    /* U+0025 "%" */
    0xc7, 0x34, 0x86, 0x10, 0xc2, 0x0, 0x4b, 0x38,
```

```
0x40,

/* U+0026 "&" */
0x70, 0x24, 0x12, 0x7, 0x0, 0x3, 0xc2, 0x11,
0xf, 0x84, 0x43, 0xde, 0x0,

/* U+0027 "'" */
0xc0,

/* U+0028 "(" */
0xf2, 0x49, 0x4, 0x93, 0x80,

/* U+0029 ")" */
0xc4, 0x92, 0x41, 0x27, 0x0,

/* U+002A "*" */
0x27, 0xbe, 0xe2, 0x0,

/* U+002B "+" */
0x21, 0x9, 0xf2, 0x10, 0x80,

/* U+002C "," */
0x38,

/* U+002D "-" */
0xf8,

/* U+002E "." */
0x20,

/* U+002F "/" */
0x4, 0x20, 0x84, 0x10, 0x86, 0x10, 0x42, 0x8,
0x0,

/* U+0030 "0" */
0xfe, 0x18, 0x65, 0x94, 0xca, 0x69, 0x86, 0x17,
0x80,

/* U+0031 "1" */
0x17, 0xd1, 0x10, 0x11, 0x11, 0x0,

/* U+0032 "2" */
0x7c, 0x10, 0x41, 0x7, 0xe8, 0x20, 0x82, 0x7,
0x80,

/* U+0033 "3" */
0xf8, 0x42, 0x10, 0xfc, 0x21, 0x8, 0x7c,
```

```
/* U+0034 "4" */
0x86, 0x18, 0x61, 0x85, 0xf0, 0x41, 0x4, 0x10,
0x40,

/* U+0035 "5" */
0xfa, 0x8, 0x20, 0x83, 0xe0, 0x41, 0x4, 0x17,
0x80,

/* U+0036 "6" */
0xfa, 0x8, 0x20, 0x83, 0xe8, 0x61, 0x86, 0x17,
0x80,

/* U+0037 "7" */
0xf8, 0x42, 0x10, 0x80, 0x21, 0x8, 0x42,

/* U+0038 "8" */
0xfe, 0x18, 0x61, 0x87, 0xe8, 0x61, 0x86, 0x17,
0x80,

/* U+0039 "9" */
0xfe, 0x18, 0x61, 0x85, 0xf0, 0x41, 0x4, 0x17,
0x80,

/* U+003A ":" */
0xc0, 0x0, 0x80,

/* U+003B ";" */
0xc0, 0x0, 0xe0,

/* U+003C "<" */
0x17, 0xcc, 0x71,

/* U+003D "=" */
0xf8, 0x1, 0xf0,

/* U+003E ">" */
0x86, 0x13, 0x68,

/* U+003F "?" */
0xfc, 0x10, 0x41, 0x7, 0xf0, 0x20, 0x80, 0x8,
0x0,

/* U+0040 "@" */
0xfe, 0x18, 0x69, 0x94, 0x8b, 0x69, 0x86, 0x17,
0x80,
```

```
/* U+0041 "A" */
0xfe, 0x18, 0x61, 0x87, 0xe8, 0x61, 0x86, 0x18,
0x40,

/* U+0042 "B" */
0xfe, 0x18, 0x61, 0x87, 0xe8, 0x61, 0x86, 0x17,
0x80,

/* U+0043 "C" */
0xfc, 0x21, 0x8, 0x2, 0x10, 0x84, 0x1e,

/* U+0044 "D" */
0xfe, 0x18, 0x61, 0x84, 0x8, 0x61, 0x86, 0x17,
0x80,

/* U+0045 "E" */
0xfc, 0x21, 0x8, 0x7e, 0x10, 0x84, 0x1e,

/* U+0046 "F" */
0xfc, 0x21, 0x8, 0x7e, 0x10, 0x84, 0x20,

/* U+0047 "G" */
0xfa, 0x8, 0x20, 0x80, 0x68, 0x61, 0x86, 0x17,
0x80,

/* U+0048 "H" */
0x86, 0x18, 0x61, 0x87, 0xe8, 0x61, 0x86, 0x18,
0x40,

/* U+0049 "I" */
0xfb, 0xe0,

/* U+004A "J" */
0x0, 0x10, 0x41, 0x4, 0x8, 0x61, 0x86, 0x17,
0x80,

/* U+004B "K" */
0x82, 0x18, 0xa4, 0xb3, 0xe8, 0x61, 0x86, 0x18,
0x40,

/* U+004C "L" */
0x4, 0x21, 0x8, 0x2, 0x10, 0x84, 0x1e,

/* U+004D "M" */
0xff, 0xf8, 0x41, 0x84, 0x18, 0x41, 0x84, 0x10,
0x0, 0x84, 0x18, 0x41, 0x84, 0x18, 0x41, 0x84,
0x0,
```

```
/* U+004E "N" */
0x87, 0x1e, 0x6d, 0x9c, 0x8, 0x61, 0x86, 0x18,
0x40,

/* U+004F "O" */
0xfe, 0x18, 0x61, 0x84, 0x8, 0x61, 0x86, 0x17,
0x80,

/* U+0050 "P" */
0xfe, 0x18, 0x61, 0x87, 0xe8, 0x20, 0x82, 0x8,
0x0,

/* U+0051 "Q" */
0xfe, 0x18, 0x61, 0x84, 0x8, 0x61, 0x86, 0xd7,
0x8c, 0x0,

/* U+0052 "R" */
0xfe, 0x18, 0x61, 0x87, 0xe9, 0xa2, 0x8e, 0x10,
0x0,

/* U+0053 "S" */
0xfa, 0x8, 0x20, 0x83, 0xe0, 0x41, 0x4, 0x17,
0x80,

/* U+0054 "T" */
0x7c, 0x1, 0x4, 0x10, 0x40, 0x4, 0x10, 0x41,
0x0,

/* U+0055 "U" */
0x2, 0x18, 0x61, 0x84, 0x8, 0x61, 0x86, 0x17,
0xc0,

/* U+0056 "V" */
0x86, 0x18, 0x61, 0x86, 0x10, 0x33, 0x49, 0xe3,
0x0,

/* U+0057 "W" */
0x84, 0x30, 0x86, 0x10, 0xc2, 0x18, 0x43, 0x8,
0x40, 0x6, 0x23, 0x4a, 0x4f, 0x78, 0xc6, 0x0,

/* U+0058 "X" */
0x84, 0x99, 0xe1, 0x80, 0xf, 0x80, 0x18, 0x78,
0x9b, 0x10,

/* U+0059 "Y" */
0x86, 0x10, 0x33, 0x49, 0xe3, 0x0, 0x10, 0x40,
```

```
/* U+005A "Z" */
0xf8, 0x10, 0xc6, 0x10, 0x2, 0x18, 0x41, 0x7,
0xc0,

/* U+005B "[" */
0xf2, 0x49, 0x4, 0x93, 0x80,

/* U+005C "\\" */
0x81, 0x4, 0x8, 0x20, 0x41, 0x82, 0x8, 0x10,
0x40,

/* U+005D "]" */
0xc4, 0x92, 0x41, 0x27, 0x0,

/* U+005E "^" */
0x0, 0x51, 0xb2, 0x2c, 0x60,

/* U+005F "_" */
0xf8,

/* U+0060 "`" */
0xd4,

/* U+0061 "a" */
0xfe, 0x18, 0x61, 0x87, 0xe8, 0x61, 0x86, 0x18,
0x40,

/* U+0062 "b" */
0xfe, 0x18, 0x61, 0x87, 0xe8, 0x61, 0x86, 0x17,
0x80,

/* U+0063 "c" */
0xfc, 0x21, 0x8, 0x2, 0x10, 0x84, 0x1e,

/* U+0064 "d" */
0xfe, 0x18, 0x61, 0x84, 0x8, 0x61, 0x86, 0x17,
0x80,

/* U+0065 "e" */
0xfc, 0x21, 0x8, 0x7e, 0x10, 0x84, 0x1e,

/* U+0066 "f" */
0xfc, 0x21, 0x8, 0x7e, 0x10, 0x84, 0x20,

/* U+0067 "g" */
0xfa, 0x8, 0x20, 0x80, 0x68, 0x61, 0x86, 0x17,
```

```
0x80,

/* U+0068 "h" */
0x86, 0x18, 0x61, 0x87, 0xe8, 0x61, 0x86, 0x18,
0x40,

/* U+0069 "i" */
0xfb, 0xe0,

/* U+006A "j" */
0x0, 0x10, 0x41, 0x4, 0x8, 0x61, 0x86, 0x17,
0x80,

/* U+006B "k" */
0x82, 0x18, 0xa4, 0xb3, 0xe8, 0x61, 0x86, 0x18,
0x40,

/* U+006C "l" */
0x4, 0x21, 0x8, 0x2, 0x10, 0x84, 0x1e,

/* U+006D "m" */
0xff, 0xf8, 0x41, 0x84, 0x18, 0x41, 0x84, 0x10,
0x0, 0x84, 0x18, 0x41, 0x84, 0x18, 0x41, 0x84,
0x0,

/* U+006E "n" */
0x87, 0x1e, 0x6d, 0x9c, 0x8, 0x61, 0x86, 0x18,
0x40,

/* U+006F "o" */
0xfe, 0x18, 0x61, 0x84, 0x8, 0x61, 0x86, 0x17,
0x80,

/* U+0070 "p" */
0xfe, 0x18, 0x61, 0x87, 0xe8, 0x20, 0x82, 0x8,
0x0,

/* U+0071 "q" */
0xfe, 0x18, 0x61, 0x84, 0x8, 0x61, 0x86, 0xd7,
0x8c, 0x0,

/* U+0072 "r" */
0xfe, 0x18, 0x61, 0x87, 0xe9, 0xa2, 0x8e, 0x10,
0x0,

/* U+0073 "s" */
0xfa, 0x8, 0x20, 0x83, 0xe0, 0x41, 0x4, 0x17,
```

```
    0x80,

    /* U+0074 "t" */
    0x7c, 0x1, 0x4, 0x10, 0x40, 0x4, 0x10, 0x41,
    0x0,

    /* U+0075 "u" */
    0x2, 0x18, 0x61, 0x84, 0x8, 0x61, 0x86, 0x17,
    0xc0,

    /* U+0076 "v" */
    0x86, 0x18, 0x61, 0x86, 0x10, 0x33, 0x49, 0xe3,
    0x0,

    /* U+0077 "w" */
    0x84, 0x30, 0x86, 0x10, 0xc2, 0x18, 0x43, 0x8,
    0x40, 0x6, 0x23, 0x4a, 0x4f, 0x78, 0xc6, 0x0,

    /* U+0078 "x" */
    0x84, 0x99, 0xe1, 0x80, 0xf, 0x80, 0x18, 0x78,
    0x9b, 0x10,

    /* U+0079 "y" */
    0x86, 0x10, 0x33, 0x49, 0xe3, 0x0, 0x10, 0x40,

    /* U+007A "z" */
    0xf8, 0x10, 0xc6, 0x10, 0x2, 0x18, 0x41, 0x7,
    0xc0,

    /* U+007B "{" */
    0x24, 0x93, 0x89, 0x24, 0x80,

    /* U+007C "|" */
    0xfb, 0xdf,

    /* U+007D "}" */
    0x92, 0x48, 0xe4, 0x92, 0x0,

    /* U+007E "~" */
    0x66, 0x60
};


/*--------------------
 *  GLYPH DESCRIPTION
 *--------------------*/
```

```
static const lv_font_fmt_txt_glyph_dsc_t glyph_dsc[] = {
    {.bitmap_index = 0, .adv_w = 0, .box_w = 0, .box_h = 0, .ofs_x = 0,
.ofs_y = 0} /* id = 0 reserved */,
    {.bitmap_index = 0, .adv_w = 113, .box_w = 1, .box_h = 1, .ofs_x = 0,
.ofs_y = 0},
    {.bitmap_index = 1, .adv_w = 39, .box_w = 1, .box_h = 11, .ofs_x = 1,
.ofs_y = 0},
    {.bitmap_index = 3, .adv_w = 51, .box_w = 2, .box_h = 2, .ofs_x = 0,
.ofs_y = 14},
    {.bitmap_index = 4, .adv_w = 263, .box_w = 15, .box_h = 10, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 23, .adv_w = 113, .box_w = 6, .box_h = 15, .ofs_x =
0, .ofs_y = -2},
    {.bitmap_index = 35, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 44, .adv_w = 158, .box_w = 9, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 57, .adv_w = 26, .box_w = 1, .box_h = 2, .ofs_x = 0,
.ofs_y = 14},
    {.bitmap_index = 58, .adv_w = 75, .box_w = 3, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 63, .adv_w = 76, .box_w = 3, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 68, .adv_w = 93, .box_w = 5, .box_h = 5, .ofs_x = 0,
.ofs_y = 5},
    {.bitmap_index = 72, .adv_w = 101, .box_w = 5, .box_h = 7, .ofs_x =
0, .ofs_y = 2},
    {.bitmap_index = 77, .adv_w = 38, .box_w = 2, .box_h = 4, .ofs_x = 0,
.ofs_y = -2},
    {.bitmap_index = 78, .adv_w = 84, .box_w = 5, .box_h = 1, .ofs_x = 0,
.ofs_y = 6},
    {.bitmap_index = 79, .adv_w = 37, .box_w = 2, .box_h = 2, .ofs_x = 0,
.ofs_y = 0},
    {.bitmap_index = 80, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 89, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 98, .adv_w = 76, .box_w = 4, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 104, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 113, .adv_w = 113, .box_w = 5, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 120, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 129, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
```

```
    {.bitmap_index = 138, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 147, .adv_w = 113, .box_w = 5, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 154, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 163, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 172, .adv_w = 37, .box_w = 2, .box_h = 9, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 175, .adv_w = 70, .box_w = 2, .box_h = 11, .ofs_x =
0, .ofs_y = -2},
    {.bitmap_index = 178, .adv_w = 87, .box_w = 4, .box_h = 6, .ofs_x =
0, .ofs_y = 2},
    {.bitmap_index = 181, .adv_w = 88, .box_w = 5, .box_h = 4, .ofs_x =
0, .ofs_y = 4},
    {.bitmap_index = 184, .adv_w = 87, .box_w = 4, .box_h = 6, .ofs_x =
0, .ofs_y = 2},
    {.bitmap_index = 187, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 196, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 205, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 214, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 223, .adv_w = 113, .box_w = 5, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 230, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 239, .adv_w = 113, .box_w = 5, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 246, .adv_w = 113, .box_w = 5, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 253, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 262, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 271, .adv_w = 35, .box_w = 1, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 273, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 282, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 291, .adv_w = 113, .box_w = 5, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
```

```
    {.bitmap_index = 298, .adv_w = 204, .box_w = 12, .box_h = 11, .ofs_x
= 0, .ofs_y = 0},
    {.bitmap_index = 315, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 324, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 333, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 342, .adv_w = 113, .box_w = 6, .box_h = 13, .ofs_x =
0, .ofs_y = -2},
    {.bitmap_index = 352, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 361, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 370, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 379, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 388, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 397, .adv_w = 193, .box_w = 11, .box_h = 11, .ofs_x
= 0, .ofs_y = 0},
    {.bitmap_index = 413, .adv_w = 113, .box_w = 7, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 423, .adv_w = 113, .box_w = 6, .box_h = 10, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 431, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 440, .adv_w = 75, .box_w = 3, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 445, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 454, .adv_w = 76, .box_w = 3, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 459, .adv_w = 131, .box_w = 7, .box_h = 5, .ofs_x =
1, .ofs_y = 8},
    {.bitmap_index = 464, .adv_w = 81, .box_w = 5, .box_h = 1, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 465, .adv_w = 128, .box_w = 2, .box_h = 3, .ofs_x =
3, .ofs_y = 9},
    {.bitmap_index = 466, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 475, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 484, .adv_w = 113, .box_w = 5, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
```

```
    {.bitmap_index = 491, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 500, .adv_w = 113, .box_w = 5, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 507, .adv_w = 113, .box_w = 5, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 514, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 523, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 532, .adv_w = 35, .box_w = 1, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 534, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 543, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 552, .adv_w = 113, .box_w = 5, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 559, .adv_w = 204, .box_w = 12, .box_h = 11, .ofs_x
= 0, .ofs_y = 0},
    {.bitmap_index = 576, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 585, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 594, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 603, .adv_w = 113, .box_w = 6, .box_h = 13, .ofs_x =
0, .ofs_y = -2},
    {.bitmap_index = 613, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 622, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 631, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 640, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 649, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 658, .adv_w = 193, .box_w = 11, .box_h = 11, .ofs_x
= 0, .ofs_y = 0},
    {.bitmap_index = 674, .adv_w = 113, .box_w = 7, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 684, .adv_w = 113, .box_w = 6, .box_h = 10, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 692, .adv_w = 113, .box_w = 6, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
```

```
    {.bitmap_index = 701, .adv_w = 75, .box_w = 3, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 706, .adv_w = 37, .box_w = 1, .box_h = 16, .ofs_x =
0, .ofs_y = -2},
    {.bitmap_index = 708, .adv_w = 75, .box_w = 3, .box_h = 11, .ofs_x =
0, .ofs_y = 0},
    {.bitmap_index = 713, .adv_w = 125, .box_w = 6, .box_h = 2, .ofs_x =
0, .ofs_y = 8}
};

/*---------------------
 *  CHARACTER MAPPING
 *--------------------*/



/*Collect the unicode lists and glyph_id offsets*/
static const lv_font_fmt_txt_cmap_t cmaps[] =
{
    {
        .range_start = 32, .range_length = 95, .glyph_id_start = 1,
         .unicode_list = NULL, .glyph_id_ofs_list = NULL, .list_length =
0, .type = LV_FONT_FMT_TXT_CMAP_FORMAT0_TINY
    }
};



/*--------------------
 *  ALL CUSTOM DATA
 *--------------------*/

#if LV_VERSION_CHECK(8, 0, 0)
/*Store all the custom data of the font*/
static  lv_font_fmt_txt_glyph_cache_t cache;
static const lv_font_fmt_txt_dsc_t font_dsc = {
#else
static lv_font_fmt_txt_dsc_t font_dsc = {
#endif
    .glyph_bitmap = glyph_bitmap,
    .glyph_dsc = glyph_dsc,
    .cmaps = cmaps,
    .kern_dsc = NULL,
    .kern_scale = 0,
    .cmap_num = 1,
    .bpp = 1,
    .kern_classes = 0,
```

```
    .bitmap_format = 0,
#if LV_VERSION_CHECK(8, 0, 0)
    .cache = &cache
#endif
};


/*-----------------
 *  PUBLIC FONT
 *----------------*/

/*Initialize a public general font descriptor*/
#if LV_VERSION_CHECK(8, 0, 0)
const lv_font_t ui_font_Display16 = {
#else
lv_font_t ui_font_Display16 = {
#endif
    .get_glyph_dsc = lv_font_get_glyph_dsc_fmt_txt,    /*Function pointer
to get glyph's data*/
    .get_glyph_bitmap = lv_font_get_bitmap_fmt_txt,    /*Function pointer
to get glyph's bitmap*/
    .line_height = 18,          /*The maximum line height required by the
font*/
    .base_line = 2,              /*Baseline measured from the bottom of
the line*/
#if !(LVGL_VERSION_MAJOR == 6 && LVGL_VERSION_MINOR == 0)
    .subpx = LV_FONT_SUBPX_NONE,
#endif
#if LV_VERSION_CHECK(7, 4, 0) || LVGL_VERSION_MAJOR >= 8
    .underline_position = -2,
    .underline_thickness = 1,
#endif
    .dsc = &font_dsc           /*The custom font data. Will be accessed
by `get_glyph_bitmap/dsc` */
};



#endif /*#if UI_FONT_DISPLAY16*/
```

- **ui_font_Display20.c:**
- **ui_font_Display24.c:**
- **ui_font_Display70.c:**

Estas fuentes el código es similar al del tamaño 16 pero con ligeras variaciones por lo que para no ser repetitivo no se han añadido al Anexo.